

(Multi-Input) FE for Randomized Functionalities, Revisited

Pratish Datta*

Jiaxin Guan[†]

Alexis Korb[‡]

Amit Sahai[§]

Abstract

Randomized functional encryption (rFE) generalizes functional encryption (FE) by incorporating randomized functionalities. Randomized multi-input functional encryption (rMIFE) extends rFE to accommodate multi-input randomized functionalities.

In this paper, we reassess the framework of rFE/rMIFE enhancing our understanding of this primitive and laying the groundwork for more secure and flexible constructions in this field. Specifically, we make three key contributions:

- **New definition:** We identify critical gap in the existing indistinguishability-based (IND) security definition for rFE/rMIFE. Notably, current definition fails to adequately address security against malicious encryptors—a crucial requirement for rFE/rMIFE since their introduction. We propose a novel, robust IND security definition that not only addresses threats from malicious decryptors but also quantifies the security against malicious encryptors effectively.
- **Counterexample:** To illustrate the importance of this definitional gap, we provide a counterexample of an insecure rFE scheme that meets IND security under the previous definition but explicitly fails in a natural setting (and where this failure would be precluded by our enhanced definition). Our counterexample scheme is non-trivial and meticulously designed using standard cryptographic tools, namely FE for deterministic functions, pseudorandom function (PRF), public key encryption (PKE), and simulation-sound non-interactive zero-knowledge (NIZK) proof systems.
- **Adaptive unbounded-message secure construction:** The only viable prior construction of rMIFE by Goldwasser et al. [EUROCRYPT 2014] (which uses indistinguishability obfuscation ($i\mathcal{O}$) and other standard assumptions) has significant limitations: it permits only a pre-defined number of messages per encryption slot and operates under selective-security constraints, requiring adversaries to declare challenge ciphertext queries and “corrupted” encryption keys in advance. We address these shortcomings by employing sub-exponentially secure $i\mathcal{O}$. Technically, we build on and adapt methods developed by Goyal et al. [ASIACRYPT 2016] for deterministic MIFE.

Keywords: Functional encryption, randomized functionalities, multi-input, simulation-based security, indistinguishability-based security

*This article is the full version of the article under the same name appearing at TCC 2025, submitted by the authors to the IACR and to Springer-Verlag on September 25, 2025. The version published by Springer-Verlag is available at https://doi.org/10.1007/978-3-032-12293-3_10.

*NTT Research. Email: pratish.datta@ntt-research.com. ORCID: 0000-0002-3938-7594.

[†]Zellic. Email: jiaxin@guan.io. ORCID: 0000-0003-1823-8845. This work was done when the author was affiliated with New York University.

[‡]UCLA. Email: alexiskorb@cs.ucla.edu. ORCID: 0000-0001-6888-5296.

[§]UCLA. Email: sahai@cs.ucla.edu. ORCID: 0000-0003-2216-9600.

Contents

1	Introduction	3
2	Technical Overview	8
2.1	New Security Definitions for rFE/rMIFE	8
2.2	Counterexample	10
2.3	The sketch of proposed rMIFE scheme	12
3	Preliminaries	15
3.1	Indistinguishability Obfuscation	18
3.2	Functional Encryption	18
3.3	Non-Interactive Zero Knowledge Proof Systems	20
4	Improved Security Definitions for rMIFE	21
4.1	Indistinguishability-Based Definition of rMIFE	23
4.2	Simulation-Based Definition of rMIFE	25
5	Counterexample	28
5.1	Definition in [GJKS15]	28
5.2	Construction of Counterexample	29
5.3	Proof of (Insufficient) Security	31
5.4	Issue with Counterexample Construction (Construction 1)	47
6	Constructing Adaptively Secure rMIFE	48
6.1	Parameters	48
6.2	Construction	48
6.3	Security against malicious decryptors	50
6.4	Security against malicious encryptors	67
7	Acknowledgements	71
8	References	72

1 Introduction

Functional Encryption. Functional Encryption (FE) [BSW11,O’N10] refines public-key encryption by allowing a decryptor to learn only $f(x)$ —and nothing else—given a function key for f and a ciphertext of x . A central authority holds the master secret key, issues function keys, and publishes the master public key for encryption by any party. Research has since focused on which functionalities FE can support and under which assumptions. A long line of work on deterministic functions culminated in Jain–Lin–Sahai’s construction of FE for all polynomial-size circuits from well-founded hardness assumptions [JLS21,JLS22]. Yet many practical tasks rely on randomized functionalities—needs that deterministic FE cannot address directly.

FE for Randomized Functionalities. To address those challenges, Alwen et al. [ABF⁺13] and Goyal et al. [GJKS15] introduced randomized functional encryption (rFE), where a key for a randomized function f lets the holder obtain a single sample from the distribution of $f(x)$ given a ciphertext of x . Across many keys for $f_1, \dots, f_q$ and ciphertexts of $x_1, \dots, x_n$, each (f_i, x_j) pair must reveal only an independent sample of $f_i(x_j)$ and nothing more. rFE underpins diverse applications – privacy-aware auditing, differentially-private data release [GJKS15], delegation of encrypted contents [AW17], fully homomorphic encryption [ABF⁺13], controlled homomorphic encryption [DIPV17], and even FE for deterministic functionalities [GGHZ16]. Acknowledging the vast potential of rFE, numerous studies [ABF⁺13,GJKS15,GGHZ16,ITZ16,AW17,KS15,DIPV17,LZ20,BKMT21] have explored rFE from both definitional and construction perspectives.

Security Against Malicious Decryptors and Encryptors in rFE. Unlike deterministic FE, where security primarily focuses on restricting the decryptor’s view, rFE must also defend against malicious encryptors. Specifically, rFE must ensure that encryptors cannot produce malformed ciphertexts for a message x that, when decrypted under a secret key for a function f , yield outputs that deviate significantly from the true distribution of $f(x)$. Consider the privacy-aware auditing use case from [GJKS15]: a regulatory agency oversees financial institutions but cannot be given unrestricted access to their sensitive databases. Granting partial access is also risky, as institutions could cherry-pick favorable disclosures. rFE enables a better solution: institutions encrypt their data under rFE, and auditors receive keys allowing random sampling of unbiased record subsets. For such audits to be trustworthy, three properties must hold: (1) distinct function keys applied to the same ciphertext should yield independent samples; (2) the same key applied to different ciphertexts should also yield independent outputs; and (3) institutions must be unable to bias these outputs by submitting faulty or malformed ciphertexts. Failure to satisfy any of these conditions compromises the audit’s integrity. For further discussion, see [GJKS15].

Simulation-Based (SIM) Security Definition for rFE. The two core security requirements for rFE—protecting against both malicious decryptors and encryptors—have been formally captured using a unified simulation-based (SIM) framework, introduced in [GJKS15] and refined in [AW17].¹ As in standard FE for deterministic functionalities [O’N10,BSW11,AGVW13], the SIM security experiment for rFE pits a real-world adversary—interacting with honest ciphertext and key generation—against a simulator that only sees minimal leakage and must emulate this interaction. To model security against malicious encryptors, [GJKS15,AW17] introduced a decryption oracle, akin to that in IND-CCA2 security [RS92]. Given a ciphertext ct and function f , the oracle behaves differently in each world: in the real world, it derives the key for f and decrypts ct normally; in the ideal world, it uses a simulator which outputs either a value x or $\perp$, and the oracle returns a random sample from

¹Some prior works on rFE [ABF⁺13,GGHZ16,ITZ16,KS15] omit protection against malicious encryptors, as it is unnecessary for their target applications.

the output distribution $f(x)$ or $\perp$ accordingly. The refinement in [AW17] strengthens this oracle to accept a polynomial-sized batch of ciphertexts and a function f . In the real world, the same secret key is used to decrypt all ciphertexts. In the ideal world, the simulator can query an evaluation oracle once per ciphertext, receiving fresh, independent outputs from $f(x)$. This enhancement addresses a key limitation in [GJKS15], where adversaries could still craft ciphertext batches yielding correlated decryptions—an attack precluded under the refined definition.

Indistinguishability-Based (IND) Security for rFE. While simulation-based (SIM) security offers the strongest guarantees for rFE [O’N10, BSW11, AGVW13], it suffers from a key limitation: it only supports a bounded number of challenge ciphertext and key queries made before the challenge ciphertext [O’N10, BSW11, AGVW13, DIJ⁺13, DI13]. To overcome this, Goyal et al. [GJKS15] introduced an indistinguishability-based (IND) security notion for rFE, extending the analogous framework from deterministic FE [O’N10, BSW11], while still aiming to protect against both malicious decryptors and encryptors. The IND security experiment tasks an adversary with distinguishing the encryptions of two messages, given access to secret keys for randomized functions whose outputs on those messages are statistically close.² To address malicious encryptors, the IND framework includes a decryption oracle—just as in SIM security. This oracle accepts a ciphertext ct and a function f as input, generates a secret key for f , and decrypts ct using this key—much like in the real-world setting of the SIM security definition. A major advantage of IND security is its flexibility: it imposes no limit on the number or timing of ciphertext and key queries. In contrast, while SIM security for a bounded number of ciphertexts implies IND security with unbounded ciphertext queries, this implication does not extend to secret key queries. In fact, known transformations from SIM to IND security [O’N10, BSW11, GJKS15] inherit the same bound on function key queries. Therefore, in applications of rFE that demand unbounded ciphertext and key queries—possibly both before and after the ciphertext queries—the strongest achievable notion is IND security. Moreover, as observed in deterministic FE, IND security can often be achieved under weaker assumptions than those required for full SIM security.

Limitations of the Existing IND Security Definition for rFE. The primary purpose of incorporating a decryption oracle in the rFE security model is to address threats from malicious encryptors. In the case of the SIM security definition, this goal is effectively realized, as shown in [AW17, GJKS15]. In the ideal world, the decryption oracle simulates honest behavior by returning independent samples from the correct output distribution of the queried function applied to the message encrypted in the ciphertext. Consequently, if the real-world decryption oracle is indistinguishable from its ideal counterpart, adversaries cannot bias or correlate outputs through malformed ciphertexts. In contrast, this safeguard breaks down in the current IND security setting. Since there is no ideal-world counterpart to anchor the behavior of the decryption oracle, there is no assurance that the adversary cannot craft malicious ciphertexts that cause the oracle to output biased or correlated values—even if a fresh secret key is used for each query. Therefore, simply providing a decryption oracle in the IND security game—otherwise designed to target malicious decryptors—fails to effectively capture security against malicious encryptors. This raises a fundamental question:

Open Problem 1. *Can we define a strengthened IND security notion for rFE that effectively captures adversaries with both malicious decryption and encryption capabilities? Further, can we construct an rFE scheme that provably satisfies this enhanced notion?*

Multi-Input Functional Encryption. Multi-Input Functional Encryption (MIFE), introduced by Goldwasser et al. [GGG⁺14], generalizes the notion of FE to support functionalities over multiple

²Computationally close output distributions can also be supported, but only when all function queries occur before the master public key is issued. Otherwise, the security definition becomes vacuous; see Remark 2.8 in [GJKS15].

inputs. In a MIFE scheme, a secret key for an n -input function (where $n > 1$ may be polynomial in the security parameter) enables a user holding n ciphertexts—each encrypting a message $x_1, \dots, x_n$ —to compute $f(x_1, \dots, x_n)$ through joint decryption of these ciphertexts, while revealing nothing beyond the final output. MIFE naturally fits into many real-world applications where aggregate computation over multiple encrypted data sources is essential. Examples include SQL queries on encrypted databases, encrypted stream analytics, non-interactive differentially-private data release, order-revealing encryption, and multi-client delegated computation. Motivated by this broad utility, the cryptographic community has devoted great effort to developing MIFE schemes and their extensions—both for general functionalities [BGJS15, AJ15, GJO16, BKS16, BGJS16, Lin17, CMR17, KS17, AJS18, KNT18] and for targeted classes of functions with practical relevance [BLR⁺15, CLWW16, BZ16, LW16, AGRW17, KLM⁺18, BJK⁺18, CDG⁺18a, CDG⁺18b, DOT18, ACF⁺18, ABKW19, ABG19, LT19, Tom19, AGW20, ACGU20, ACF⁺20, ABM⁺20, CDSG⁺20, AFS21, AGT21, AGT22, AYY22, NPP22, ATY23, FFMV23, SV23, NPP23a, NPP23b, ARYY23, FFMV24]. Despite this progress, all these schemes are restricted to deterministic functionalities.

MIFE for Randomized Functionalities. As with single-input FE, many natural applications of MIFE demand support for randomized functionalities. Examples include privacy-preserving joint audits across multiple organizations, salary surveys of citizens within a country, randomized SQL queries, and so on. To support such scenarios, the syntax and security notions of deterministic MIFE can be extended to define randomized MIFE (rMIFE), mirroring the extensions used for single-input rFE. Goldwasser et al. [GGG⁺14] initiated this direction by formulating a SIM-based security definition for rMIFE, generalizing the corresponding notion for single-input rFE [GJKS15]. This definition can naturally incorporate the correlated ciphertext handling capabilities of [AW17]. Although [GGG⁺14] did not explicitly define an IND-based security framework for rMIFE, such a definition can be analogously derived from the IND notion for rFE [GJKS15]. However, as in the single-input setting, the resulting IND definition fails to adequately capture security against malicious encryptors. Therefore, developing a robust IND notion that properly accounts for both malicious encryptors and decryptors is equally crucial for rMIFE.

Importance of IND Security for rMIFE. IND security plays an even more critical role in the setting of MIFE and rMIFE. Goldwasser et al. [GGG⁺14] showed that achieving SIM security for MIFE is impossible for general deterministic functionalities—even in the secret-key setting, with constant arity and just two ciphertext queries per slot. Specifically, they proved that such a scheme would imply virtual black-box (VBB) obfuscation, which is known to be impossible for general functionalities [BGI⁺01]. Since every deterministic function can be viewed as a randomized one with a singleton output distribution, this impossibility result directly extends to SIM security for rMIFE. Thus, in any meaningful general setting, IND security is the only attainable notion for rMIFE.

Existing rMIFE scheme and its limitations. The only known rMIFE construction is the one briefly outlined by Goldwasser et al. [GGG⁺14] as an extension of their deterministic MIFE scheme, based on differing-inputs obfuscation (*diO*) [BGI⁺01, BCP14]. However, *diO* is widely regarded as implausible in general [GGHW14], rendering their proposed MIFE and rMIFE constructions generally infeasible. Moreover, Goldwasser et al. [GGG⁺14] did not provide any formal security proof or even a proof sketch for their proposed rMIFE construction.

In the same work, Goldwasser et al. [GGG⁺14] also suggested that their deterministic MIFE construction based on indistinguishability obfuscation (*iO*) [BGI⁺01, GGH⁺13, JLS21, JLS22] could be generalized to an rMIFE scheme as well in a manner similar to their *diO*-based construction, though no concrete construction or proof sketch was provided. The original *iO*-based deterministic MIFE construction was proven to achieve IND security, and thus, the rMIFE construction extended

from it is expected to achieve IND security according to the multi-input extension of the definition in [GJKS15]. However, since this definition does not fully capture security against malicious encryptors, it is unclear whether that generalized rMIFE construction would provide such guarantees.

Additionally, the rMIFE construction inherits other significant limitations from the original deterministic MIFE scheme. These include security guarantees only against a bounded number of ciphertext queries per encryption slot and selective adversaries, who must declare all challenge ciphertext queries and the encryption keys they wish to corrupt at the beginning of the security experiment. These challenges lead to the following natural question.

Open Problem 2. *Can we construct a viable rMIFE scheme that achieves the following security properties:*

- IND security under an enhanced definition that effectively captures security against both malicious decryptors and encryptors;
- Protection against fully adaptive adversaries, who are allowed to make ciphertext, secret key, and corruption queries in any arbitrary order, at any point during the security experiment;
- Support for an unbounded polynomial number of secret key and ciphertext queries per encryption slot.

Our Results

We affirmatively resolve the aforementioned open problems through three key contributions:

1. **Enhanced IND Security Definition for rFE/rMIFE:** We propose a robust IND security definition for rFE, effectively capturing threats from both malicious decryptors and encryptors. Unlike previous unified frameworks [GJKS15], our approach separates these threats into two distinct security experiments. The first experiment, analogous to existing IND definitions for deterministic FE [BSW11, O’N10, GGG⁺14], ensures indistinguishability against malicious decryptors. The second experiment introduces a new oracle structure, employing a two-mode decryption oracle, rigorously capturing security against malicious encryptors. For detailed discussion, refer to Section 2.1.1. This definition naturally generalizes to the multi-input setting, providing a comprehensive IND security notion for rMIFE. In fact, we formally define our IND security notion for rMIFE (Section 4), with the single-input rFE definitions as a special case.

Additionally, we identify a subtle yet significant gap in existing SIM-based security definitions [GGG⁺14, GJKS15, AW17], specifically concerning reusing secret keys. Such reuse can be exploited by malicious encryptors to influence decryption outcomes, compromising security in real-life applications. For instance, in the privacy-aware auditing application, an auditor may use the same key to decrypt encrypted datasets from several institutions within a single audit. An institution could exploit the audit results of others to tailor its encryption and bias its own audit outcome. Existing SIM definitions do not capture this threat. Our refined SIM framework addresses this vulnerability, protecting against these advanced attack vectors (see Section 2.1.2 and 4.2).

Fortunately however, as detailed in Remark 4.8 and Lemma 4.10, the existing rFE/rMIFE constructions from [GGG⁺14, GJKS15, AW17] although were originally analyzed under earlier formulations, also satisfy the strengthened security notions we introduce in this work.

2. **Counterexample Demonstrating the Gap Between New and Existing IND Security Definitions:** We present a counterexample to highlight the significance of the gap between

our IND security definition and the existing definition [GJKS15]. Specifically, we construct an insecure rFE scheme that satisfies IND security criteria of the previous definition but fails to do so in our new definition. The counterexample relies solely on standard cryptographic primitives, including functional encryption (FE) for deterministic functionalities, pseudorandom function (PRF), standard public key encryption (PKE), and simulation-sound non-interactive zero-knowledge (NIZK) proof systems [Sah99]. Our construction is non-trivial, and its security analysis requires careful consideration. We elaborate more on this in Section 2.2.

3. ***Adaptively IND secure rMIFE scheme with unbounded message security:*** Our final contribution is an adaptively IND secure rMIFE scheme for general randomized functionalities, based on sub-exponentially secure $i\mathcal{O}$. As demonstrated by Goldwasser et al. [GGG⁺14], $i\mathcal{O}$ is already necessary for MIFE for general deterministic functionalities, even in the secret key setting, only supporting selective IND security with a single secret key query and at most two ciphertext queries per encryption slot.

We analyze the IND security of our construction within the new robust security model introduced in this work. Moreover, our scheme is the first plausible construction that supports an unbounded polynomial number of secret key and challenge ciphertext queries per encryption slot. The construction relies on three key components: (a) sub-exponentially secure $i\mathcal{O}$, (b) sub-exponentially secure injective one-way functions, and (c) standard public-key encryption (PKE). Here, “sub-exponential security” means that the advantage of any efficient adversary is sub-exponentially small. For the injective one-way functions, this security should additionally hold against adversaries operating in sub-exponential time.

A few clarifications are in order. First, in our case, the required level of security of the underlying primitives varies based on the function’s arity, but it does not depend on the number of challenge messages. As Goldwasser et al. noted, the selective security of their deterministic MIFE scheme based on $i\mathcal{O}$ (though not bounded-message security, which pertains to their use of statistically sound non-interactive proofs) can be overcome via standard complexity leveraging. This should similarly apply to their rMIFE construction generalized from it. However, in their case, the required security level depends on the number of challenge messages, leading to significantly larger parameters than our scheme, especially since the number of challenge messages is typically much larger than the function’s arity in practical applications.

Secondly, although our security proof utilizes a sub-exponentially secure injective one-way function (primitive (b)), this function is not needed in the scheme itself. Therefore, the existence of such an injective one-way function is sufficient for the security of our rMIFE scheme, even without the knowledge of an explicit candidate. At a technical level, we build on the methods developed by Goyal et al. [GJO16], who achieved similar results for deterministic MIFE. We give an overview of our construction in Section 2.3.

One caveat in our adaptive rMIFE scheme is that the output distributions of the queried functions on the queried sets of inputs need to be computationally indistinguishable with a sub-exponentially small advantage. Overcoming the sub-exponential barrier remains an interesting open problem.

Challenges in Utilizing $i\mathcal{O}$. $i\mathcal{O}$ is a very powerful cryptographic tool, yet harnessing it to develop new primitives remains highly challenging even when ideal obfuscation would make the task straightforward. A striking example is the long-standing effort to construct adaptively secure SNARGs for NP from $i\mathcal{O}$. Despite extensive research, this milestone was only recently reached [WW24a, WW24b], underscoring the deep complexity of using $i\mathcal{O}$ to build cryptographic primitives, even in well-established areas.

In contrast, some transformations that seem feasible under ideal obfuscation turn out to be fundamentally unattainable with $i\mathcal{O}$ alone. A notable case is the "Upgrade any PKE to FE" problem [BKS^W18], which, despite being simple to achieve using ideal obfuscation, has been proven impossible even using $i\mathcal{O}$ [BKS^W18].

In fact, after more than a decade of study, many fundamental questions about $i\mathcal{O}$ remain open. One major hurdle is constructing $i\mathcal{O}$ for Turing machines capable of handling inputs of arbitrary length—an unsolved problem that continues to challenge researchers, despite being easy given ideal obfuscation for circuits. Expanding our understanding of both the potential and the limitations of $i\mathcal{O}$ is a crucial pursuit in cryptographic foundations. Our results and techniques contribute to this long-standing effort, pushing the field closer to resolving these enduring challenges.

2 Technical Overview

Here we outline our three technical contributions, as previewed in Section 1.

2.1 New Security Definitions for rFE/rMIFE

We introduce a novel IND security definition of rFE/rMIFE. We also present a robust SIM security formulation for rFE/rMIFE. In this technical overview, we focus on single-input functionality for the ease of exposition. However, the formal definitions in Sections 4 and 4.2 extend to multi-input functionalities. Unlike prior works [GJKS15, GGG⁺14], our definitions are designed for adaptive adversaries, who are not required to submit their challenge ciphertext queries upfront.

2.1.1 New IND Security Definition for rFE

As highlighted earlier, we address the IND security of rFE through two distinct indistinguishability-based security experiments, which are formalized below.

IND Security Against Malicious Decryptors. This security experiment extends the IND security framework for deterministic FE [O’N10, BSW11] to randomized functionalities, closely following the definition in [GJKS15] (Definition 2.6), with one key distinction: we omit providing the adversary with a decryption oracle, as this experiment is not concerned with malicious encryptors.

- **Setup:** The challenger runs the **Setup** algorithm to generate the master public/secret key pair (mpk, msk) and provides mpk to the adversary.
- **Query Phase 1:** The adversary can adaptively request any polynomial number of secret keys for randomized functions within the function space. For each query function f , the challenger runs **KeyGen** with the master secret key to generate a secret key sk_f and hands it to the adversary.
- **Challenge:** The adversary submits two challenge messages x_0 and x_1 . The challenger selects a random bit $b \leftarrow \{0, 1\}$ and encrypts x_b under mpk to generate the ciphertext ct , which is then sent to the adversary.
- **Query Phase 2:** The adversary can continue to adaptively request secret keys as in **Query Phase 1**, and the challenger responds accordingly.
- **Guess:** The adversary outputs its guess $b' \in \{0, 1\}$ and wins if $b' = b$.

The adversary must satisfy an admissibility condition: for any queried function f and any pair of challenge messages x_0, x_1 , the output distributions $f(x_0)$ and $f(x_1)$ must be indistinguishable. This definition can readily be generalized to handle multiple challenge ciphertexts. In fact, it can be shown that security against single and multiple ciphertexts are essentially equivalent [GJKS15].

IND Security Against Malicious Encryptors. This security experiment models the behavior of malicious encryptors attempting to influence functional outputs by generating faulty ciphertexts. The experiment proceeds as follows:

- **Setup:** The challenger runs the **Setup** algorithm to generate the master public/secret key pair (mpk, msk) and provides mpk to the adversary. A random bit $b \leftarrow \{0, 1\}$ is also sampled.
- **Query Phase 1:** The adversary can adaptively make the following queries:
 - *Secret Key Query:* The adversary requests secret keys for any number of randomized functions from the underlying function family. For each function f , the challenger generates the secret key sk_f by running the **KeyGen** algorithm and provides it to the adversary.
 - *Secret Key Store Query:* The adversary requests the challenger to store secret keys for certain randomized functions. For each function g , the challenger generates sk_g using **KeyGen** algorithm and stores (g, sk_g) in a register **KeyReg**.
 - *Decryption Query:* The adversary submits ciphertexts for decryption. If $b = 0$, the challenger decrypts the ciphertext using all stored keys in **KeyReg**. If $b = 1$, the challenger extracts the plaintext using msk , applies all stored functions g to it, and returns independently sampled random outputs from the resulting distributions to the adversary. Each additionally stores these outputs along with the queried ciphertext in an output register **OutReg**, and if the adversary requests decryption for the same ciphertext once again, it simply outputs the stored values.
- **Guess:** The adversary outputs a guess $b' \in \{0, 1\}$ and wins if $b' = b$.

Old vs. New IND Definition. The IND security definition in [GJKS15] closely resembles our first definition, which addresses malicious decryptors, but with a key difference: their model includes a decryption oracle, similar to IND-CCA2 security. This oracle decrypts ciphertexts using honestly generated secret keys for the functions under which decryption is sought. While [GJKS15] claims this approach ensures security against malicious encryptors, it actually falls short. Observe that, unlike the **SIM** security model, the **IND** setting lacks an ideal functionality. The decryption oracle merely runs the decryption algorithm of the underlying **rFE** scheme, allowing adversaries to submit malicious ciphertexts and obtain biased or correlated outputs.

Our new IND security definition, specifically designed for malicious encryptors, addresses this issue by introducing two decryption modes for the decryption oracle: one for the real decryption algorithm and the other for an ideal decryption functionality, similar to the **SIM** security model. Indistinguishability between these modes ensures that adversaries cannot craft ciphertexts to influence the decryption oracle’s outputs in ways that deviate from the intended functionality. Additionally, note that this is true even when the same secret keys are used by the decryption oracle repeatedly for decrypting the queried ciphertext over time.

2.1.2 New SIM Security Definition for rFE

In existing SIM security definitions [GJKS15, AW17], each decryption query—whether in the real or ideal world—generates fresh, independent samples: new secret keys in the real world and new samples from the function’s outputs distribution in the ideal world. Thus, these definitions only guarantee security in scenarios where repeated decryptions under the same function yield independent outputs. To address this limitation, we propose an enhanced SIM security definition for rFE/rMIFE that additionally protects against adversaries exploiting repeated secret key usage. Our model introduces a **KeyStore** oracle that stores one secret key per function. In the real world, the decryption oracle uses these fixed keys. In the ideal world, it freshly samples and stores outputs per ciphertext-function pair and reuses stored results for repeated queries. This design thwarts adaptive ciphertext crafting that targets correlation across decryptions under persistent keys. For further details, please see Section 4.2.

2.2 Counterexample

To highlight the shortcomings of the IND security definition in [GJKS15], we present a counterexample. Specifically, we construct an rFE scheme that satisfies the IND security requirements of [GJKS15] but is, in fact, insecure. We demonstrate that this scheme fails to meet the criteria of our proposed IND security definition. An overview of the rFE scheme is provided below.

The rFE Scheme. Our counterexample rFE scheme leverages the following cryptographic tools: (a) an FE scheme for general deterministic functionalities, (b) a pseudorandom function (PRF), (c) a public key encryption (PKE) scheme, (d) a symmetric key encryption (SKE) scheme, and (e) a simulation-sound non-interactive zero-knowledge (NIZK) proof system. The scheme operates as follows:

- **Setup:** The **Setup** algorithm runs the **Setup** for the FE, PKE, SKE, and NIZK systems, generating keys: $(\text{FE.mpk}, \text{FE.msk})$ for FE, $(\text{PKE.pk}, \text{PKE.sk})$ for PKE, SKE.sk for SKE and a common reference string crs for NIZK. The master public key is $\text{mpk} = (\text{FE.mpk}, \text{PKE.pk}, \text{crs})$, and the master secret key is $\text{msk} = (\text{FE.msk}, \text{SKE.sk})$. The algorithm outputs mpk and retains msk .
- **Enc:** Given the message x , the encryption algorithm proceeds as follows:
 - It samples a PRF key K and generates an FE ciphertext FE.ct encrypting $(x, K, 0, \perp)$ under FE.mpk where $\perp$ is special string of appropriate length.
 - It then creates a PKE ciphertext PKE.ct encrypting $(x, K, 0, \perp)$ along with FE.ct under PKE.pk .
 - Next, it computes an NIZK proof π attesting that both ciphertexts encrypt the same values $(x, K, \alpha, \hat{\text{sk}})$, and that PKE.ct also encrypts the FE ciphertext. Here, $\alpha \in \{0, 1\}$, and $\hat{\text{sk}}$ is the same length as the SKE key.
 - Finally, the encryption outputs the ciphertext $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$.
- **KeyGen:** The algorithm takes the master secret key msk and a randomized function f and proceeds as follows.
 1. It samples a seed s and a SKE.ct encrypting a random message with the same length as the output of f to construct a function $G[f, s, \text{SKE.ct}]$.

2. The function G takes as input $(x, K, \alpha, \widehat{\mathbf{sk}})$ and operates in two modes depending on α :
 - If $\alpha = 0$, it computes randomness $\text{PRF.Eval}(K, s)$ and uses it to compute f on x ;
 - If $\alpha = 1$, it decrypts the SKE ciphertext using $\widehat{\mathbf{sk}}$ and output the corresponding message.
 3. The algorithm generates an FE secret key FE.sk_{G_f} for $G[f, s, \text{SKE.ct}]$ using FE.msk and outputs SK_f .
- **Dec:** The Dec algorithm takes as input the NIZK common reference string crs , a secret key $\text{SK}_f = \text{FE.sk}_{G_f}$ for the randomized function f and a ciphertext $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$. It verifies the NIZK proof π . If valid, it decrypts FE.ct using the secret key FE.sk_{G_f} and outputs the result; otherwise, it aborts and outputs $\perp$.

Insecurity of the constructed rFE scheme We demonstrate that the rFE scheme described above is insecure against malicious encryptors. Specifically, consider a pseudorandom function (PRF) with a key of the form $K = (K', 0, \perp)$, where K' is the key for another pseudorandom function PRF' , and $\perp$ is a special symbol. Given an input seed s , this PRF outputs $\text{PRF}'(K', s)$. It is easy to verify that this PRF behaves as a valid pseudorandom function. However, suppose the evaluation algorithm of this PRF includes a trojan branch. For keys of the form $K = (K', 1, r)$, where K' is the key for PRF' and r is a fixed string (matching the length of the output of PRF'), the evaluation algorithm bypasses PRF' entirely and directly outputs the string r .

Now, consider instantiating the above rFE scheme with this specially crafted PRF and its evaluation algorithm. If the encryptor generates two ciphertexts ct_1 and ct_2 for the same message x , using PRF keys $K_1 = (K'_1, 1, r)$ and $K_2 = (K'_2, 1, r)$ with freshly sampled K'_1 and K'_2 but using the same string r , the decryption using a secret key FE.sk_{G_f} for a randomized function f will yield the same output $f(x; r)$ in both cases. This occurs because the trojan evaluation branch outputs the fixed string r , regardless of the seed.

In a secure rFE scheme, the decryption results should be independent, uniformly sampled outputs from the distribution of $f(x)$. However, by carefully selecting PRF keys, a malicious encryptor can introduce arbitrary correlations between the decryption outputs of different ciphertexts for the same message. This attack can be further extended to enforce correlations among ciphertexts for different messages, breaking the intended security guarantees of the rFE scheme.

Specifically, to see how this rFE scheme fails our new IND based definition, consider an adversary that first makes a Secret Key Store Query for some randomized function f , causing the challenger to store (sk_f, f) in KeyReg . The adversary then submits multiple Decryption Queries with ciphertexts ct_1 and ct_2 that encrypt the same message x but use PRF keys $K_1 = (K'_1, 1, r)$ and $K_2 = (K'_2, 1, r)$ respectively (with the same fixed string r). If $b = 0$, the adversary will observe that the two ciphertexts decrypt to the same output, as the same randomness r is hardcoded; but if $b = 1$, the outputs will be independently sampled from the output distribution and hence be different.

Proving that our rFE scheme achieves [GJKS15] IND security definition. We provide an intuitive outline of why the rFE scheme above satisfies the IND security definition from [GJKS15]. Observe that, this security definition is similar to the one against malicious decryptors but with an additional decryption oracle, akin to IND-CCA2 security. For simplicity, here we ignore the decryption oracle and focus on the case where the ciphertext consists solely of the underlying FE ciphertext. The other two ciphertext components are mostly designed to handle the decryption oracle. More precisely, those components are used to make the decryption oracle not use any secret

key for FE while answering the adversary’s decryption queries during the hybrid proof. For further details, see Section 5.

At a high level, our goal is to reduce the IND security of the rFE scheme to the security of the underlying FE scheme, the SKE scheme, and the PRF. We outline the key steps in the proof through a series of hybrid arguments:

1. *Initial Setup:* We start with the security game where the FE ciphertext encrypts $(x_0, K, 0, \perp)$.
2. *Hardcoding the Output:* We now change the SKE ciphertext in the KeyGen queries from encrypting a random message to encrypting the output of $f(x_0; r)$, where r is the PRF output used as the randomness.
3. *Modifying the Ciphertext:* We update the FE ciphertext to encrypt $(\perp, \perp, 1, \text{SKE.sk})$. Since $\alpha = 1$, the function $G[f, s, \text{SKE.ct}]$ will output the result of decrypting SKE.ct using SKE.sk, which yields exactly $f(x_0; r)$. So the output of $G[f, s, \text{SKE.ct}]$ remains unchanged, and thus this transition is indistinguishable by the security of FE scheme.
4. *Switching PRF Output:* Next, we use the security of the PRF to replace r with a uniform random string, instead of the output of the PRF. This transition is indistinguishable due to the security of the pseudorandom function PRF, as the PRF key K is hidden and unused.
5. *Switching to $f(x_1)$:* In the next hybrid, we switch $f(x_0; r)$ to $f(x_1; r)$, which is a uniform sample from the output distribution of $f(x_1)$. This change is indistinguishable due to the indistinguishability of $f(x_0)$ and $f(x_1)$, as required by the IND security game restriction for rFE.
6. *Final Reversion:* Once this transition is made, we can reverse the previous steps, ultimately encrypting $(x_1, K, 0, \perp)$, completing the proof.

2.3 The sketch of proposed rMIFE scheme

We now outline the main technical ideas behind our adaptively secure rMIFE scheme, which supports an unbounded number of challenge ciphertext queries per encryption slot. Inspired by [GJO16], we build upon the adaptively secure deterministic MIFE construction of Goldwasser et al. [GGG⁺14], based on $di\mathcal{O}$.

In their construction, the encryption key for each index $i \in [n]$ (where n is the function’s arity) consists of a pair of public keys $(\text{pk}_i^0, \text{pk}_i^1)$ from an underlying public key encryption (PKE) scheme. The ciphertext at index i includes encryptions of the plaintext under both pk_i^0 and pk_i^1 , a simulation-sound NIZK proof asserting equality of the encrypted messages, and a one-time signature over the two PKE ciphertexts and the proof, signed under a freshly generated one-time verification key.

The secret key for a function f is an obfuscated program that processes n ciphertext pairs, each with associated proofs, one-time signatures, and verification keys. This program has the function f and a key K for a puncturable pseudorandom function (PRF) [SW14, BGI14, BW13, KPTZ13] hardwired. The program takes as input ciphertext pairs $(c_1^0, c_1^1, \pi_1, \text{vk}_1, \sigma_1), \dots, (c_n^0, c_n^1, \pi_n, \text{vk}_n, \sigma_n)$ and first verifies the one-time signatures and proofs. If all checks pass, the program decrypts each ciphertext using the corresponding secret key. It then evaluates the puncturable PRF with key K on the entire ciphertext tuple to generate randomness r , which is finally used to apply f to the decrypted plaintexts.

Goldwasser et al. [GGG⁺14] showed that security against dishonest decryptors follows similarly to their deterministic scheme based on $di\mathcal{O}$. They further mention that security against malicious encryptors is ensured by using one-time signatures, which guarantee the uniqueness of each

ciphertext, preventing adversaries from modifying honestly generated ciphertexts to manipulate decryption queries.

Unfortunately, as highlighted in our previous discussion, merely preventing the adversary from modifying honestly generated ciphertexts for decryption queries is insufficient to guarantee IND security against malicious encryptors. In fact, an adversary could generate “bad” ciphertexts for slots where it has corrupted the encryption keys, then combine these faulty ciphertexts with honestly generated ones from uncorrupted slots to extract non-uniform or correlated outputs from the decryption oracle. Therefore, a more robust approach is required to construct an rMIFE scheme that meets the stronger IND security definition. Furthermore, a key assumption behind security proof of the above rMIFE scheme by [GGG⁺14] is the use of $di\mathcal{O}$. Specifically, when function keys are switched to decrypt the second ciphertext in each pair, an adversary capable of detecting this change could exploit it to produce a false proof.

To address these issues, we introduce modifications to the scheme, allowing us to leverage a result from [BCP14], which shows that any indistinguishability obfuscator is, in fact, a $di\mathcal{O}$ for circuits that differ on polynomially many points. Fortunately, [CGJS15] recently demonstrated that the result of [BCP14] extends to our setting. Thus, we begin with a sub-exponentially secure indistinguishability obfuscator, which forms the basis of our enhanced approach to achieving adaptive IND security in the presence of malicious encryptors.

Specifically, to ensure that the proofs of well-formedness are unique for each ciphertext pair—and to limit the number of differing input points in the corresponding hybrids of our security proofs to a polynomial amount—we design novel proof strategy using $i\mathcal{O}$ and puncturable PRFs. Here’s how it works:

We include in the public parameters an obfuscated program that takes two ciphertexts and a witness proving they are well-formed (i.e., generated using the same message and randomness). If this check succeeds, the program outputs a puncturable PRF evaluation on those ciphertexts. The secret key for a function f is then an obfuscated program that has hardwired the PRF keys and verifies the proofs of well-formedness by checking the correctness of the PRF evaluations. As in the construction by [GGG⁺14], the program also contains another puncturable PRF key K , which is sampled during key generation. Once all PRF evaluations are verified, this puncturable PRF with key K is applied to the entire collection of ciphertexts to generate the randomness needed for the functional output.

In the security proof, we introduce a key enhancement by performing the verification through an injective one-way function applied to the PRF values, rather than directly comparing the PRF outputs. This approach ensures that extracting a differing input at this stage of the proof corresponds to inverting the injective one-way function. Without this mechanism, we would need to hardcode the correct PRF evaluation into the obfuscated secret key, making it difficult to argue security effectively.

Security against malicious decryptors. We now sketch the sequence of hybrids in our IND security proof against malicious decryptors. The proof starts from a hybrid where each challenge ciphertext encrypts x_i^0 for $i \in [n]$. Then we switch to a hybrid where each c_i^1 is an encryption of x_i^1 instead. These two hybrids are indistinguishable due to security of the PKE scheme. Let s denote the length of a ciphertext. For each index $i \in [n]$ we define hybrids indexed by w , for all $w \in [2^{2sn}]$, in which function key SK_f decrypts the first ciphertext in the pair using SK_i^0 when $(c_1^0, c_1^1, \dots, c_n^0, c_n^1) < w$ and decrypts the second ciphertext in the pair using SK_i^1 otherwise. Parse $w = (w_1^0, w_1^1, \dots, w_n^0, w_n^1)$.

Hybrids indexed by w and $w + 1$ can be proven indistinguishable as follows: We first switch to

sub-hybrids that puncture the PRF key at w_i^0, w_i^1 , change a function key SK_f to check correctness of an PRF value by applying an injective one-way function as described above, and hardcode the output of the injective one-way function as the PRF evaluation at the punctured point. We also puncture the hardwired puncturable PRF key K used for generating output randomness as the point $(w_1^0, w_1^1, \dots, w_n^0, w_n^1)$. Roughly speaking, if the two hybrids differ at an input of the form $(w_1^0, w_1^1, u_1, \dots, w_n^0, w_n^1, u_n)$ where u_i is some fixed value (a PRF evaluation of (w_i^0, w_i^1)), extracting the differing input can be used to invert the injective one-way function on random input (namely u_i). The formal security argument is a bit subtle at this point since unlike deterministic MIFE, we do not have exactly equality of the functional values corresponding to 0 and 1 cases. Instead, we carefully leverage the sub-exponentially small computational distance between the functional output distribution corresponding to the 0 and 1 cases. Please refer to our formal proof in the sequel. Finally, we note that exponentially many hybrids are indexed by all possible ciphertext vectors that could be input to decryption (i.e., vectors of length the arity of the functionality) and not all possible challenge ciphertext vectors. This allows us to handle any unbounded (polynomial) number of ciphertexts for every index. Also, by deterministically traversing over all possible ciphertexts, we can support adaptive adversary since the reduction does not need to know what challenge ciphertexts the adversary will be querying during the hardwiring at different stages of the security proof.

We now sketch the hybrid sequence used for our IND security proof against malicious decryptors.

- In the first hybrid, each challenge ciphertext encrypts x_i^0 for $i \in [n]$.
- We then switch to a hybrid where each c_i^1 encrypts x_i^1 instead. These two hybrids are indistinguishable due to the security of the underlying PKE scheme.
- Let s denote the length of a ciphertext. For each index $i \in [n]$, we define hybrids indexed by w , for all $w \in [2^{2sn}]$. In these hybrids, the function key SK_f decrypts the first ciphertext in each pair using SK_i^0 when $(c_1^0, c_1^1, \dots, c_n^0, c_n^1) < w$, and decrypts the second ciphertext in each pair using SK_i^1 otherwise. The index w is parsed as $(w_1^0, w_1^1, \dots, w_n^0, w_n^1)$.
- Hybrids indexed by w and $w + 1$ can be proven indistinguishable as follows:
 - We start with sub-hybrids that puncture the PRF key at points w_i^0, w_i^1 .
 - We then modify the function key SK_f to check the correctness of a PRF value by applying an injective one-way function, as described earlier, and hardcode the output of this function at the punctured points. Additionally, we puncture the hardcoded PRF key K , used for generating output randomness, at the point $(w_1^0, w_1^1, \dots, w_n^0, w_n^1)$.
 - Roughly speaking, if the two hybrids differ on an input of the form $(w_1^0, w_1^1, u_1, \dots, w_n^0, w_n^1, u_n)$ —where u_i is the result of a PRF evaluation on (w_i^0, w_i^1) —extracting the differing input would allow us to invert the injective one-way function on random input, i.e., the u_i . The formal security argument is subtle here, as unlike in deterministic MIFE, we do not have exact equality between functional values in the 0 and 1 cases. Instead, we carefully exploit the sub-exponentially small computational distance between the output distributions of the function in these two cases. For full details, please refer to our formal proof.

Lastly, we note that the exponentially many hybrids are indexed by all possible ciphertext vectors that could be input to decryption (i.e., vectors of length equal to the function's arity), rather than just the challenge ciphertext vectors. This allows us to handle any unbounded (polynomial)

number of ciphertexts per index. By deterministically traversing all possible ciphertexts, we support adaptive adversaries without needing to know in advance which challenge ciphertexts the adversary will query at different stages of the security proof.

Security against malicious encryptors. Finally, to argue security against malicious encryptors, we observe that the randomness used in function evaluation within the secret key program is derived from the hardwired PRF key, which is applied to the entire tuple of input ciphertexts. As a result, the encryptor has no control over the randomness used to generate the function output. This is because due to the pseudorandom properties of the PRF, whenever a different collection of ciphertexts is provided to the secret key program, the program generates a fresh random string for evaluating the output. This ensures that encryptors cannot bias the randomness, preserving the integrity of function evaluation.

3 Preliminaries

Throughout, we will use λ to denote the security parameter.

Notation

- We say that a function $f(\lambda)$ is negligible in λ if $f(\lambda) = \lambda^{-\omega(1)}$, and we denote it by $f(\lambda) = \text{negl}(\lambda)$.
- We say that a function $g(\lambda)$ is polynomial in λ if $g(\lambda) = p(\lambda)$ for some fixed polynomial p , and we denote it by $g(\lambda) = \text{poly}(\lambda)$.
- For $n \in \mathbb{N}$, we use $[n]$ to denote $\{1, \dots, n\}$.
- If R is a random variable, then $r \leftarrow R$ denotes sampling r from R . If T is a set, then $i \leftarrow T$ denotes sampling i uniformly at random from T .

Definition 3.1 (Statistical Distance). *Let D_1 and D_2 be two distributions with support in X . The statistical distance between D_1 and D_2 is*

$$\Delta(D_1, D_2) = \frac{1}{2} \sum_{x \in X} \left| \Pr[D_1 = x] - \Pr[D_2 = x] \right|$$

Notation Let A and B be two random variables with support in X . We use $\Delta(A, B)$ to denote the statistical distance $\Delta(P_A, P_B)$ between the underlying distributions of the random variables.

Definition 3.2 (Pseudorandom Function (PRF)). *A pseudorandom function family (PRF) with key space $\mathcal{K} = \{\mathcal{K}_{\lambda,n,m}\}_{\lambda,n,m \in \mathbb{N}}$ is a tuple of PPT algorithms $\text{PRF} = (\text{PRF.Setup}, \text{PRF.Eval})$ where*

- $\text{PRF.Setup}(1^\lambda, 1^n, 1^m)$ is a randomized algorithm that takes as input the security parameter λ , an input length n , and an output length m , and outputs a key $K \in \mathcal{K}_{\lambda,n,m}$.
- $\text{PRF.Eval}(K, x)$ is a deterministic algorithm that takes as input a key $K \in \mathcal{K}_{\lambda,n,m}$ and an input $x \in \{0, 1\}^n$, and outputs a value $y \in \{0, 1\}^m$.

Security requires that there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and all PPT adversaries $\mathcal{A}$,

$$\left| \Pr[\text{Expt}_{\mathcal{A}}^{\text{PRF}}(1^\lambda, 0) = 1] - \Pr[\text{Expt}_{\mathcal{A}}^{\text{PRF}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{Expt}_{\mathcal{A}}^{\text{PRF}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ and outputs an input size 1^n and an output size 1^m .
2. **Setup:**
 - (a) If $b = 0$, sample $K \leftarrow \text{PRF.Setup}(1^\lambda, 1^n, 1^m)$.
 - (b) If $b = 1$, sample $R \leftarrow \mathcal{R}_{n,m}$ where $\mathcal{R}_{n,m}$ is the set of all functions from $\{0,1\}^n$ to $\{0,1\}^m$.
3. **PRF Queries:** The following can be repeated any polynomial number of times:
 - (a) $\mathcal{A}$ outputs a value $x \in \{0,1\}^n$.
 - (b) If $b = 0$, send $y = \text{PRF.Eval}(K, x)$ to $\mathcal{A}$.
 - (c) If $b = 1$, send $y = R(x)$ to $\mathcal{A}$.
4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Definition 3.3 (Puncturable Pseudorandom Function (PPRF)). A puncturable pseudorandom function family with key space $\mathcal{K} = \{\mathcal{K}_{\lambda,n,m}\}_{\lambda,n,m \in \mathbb{N}}$ is a tuple of PPT algorithms $\text{PPRF} = (\text{PPRF.Setup}, \text{PPRF.Eval}, \text{PPRF.Punc}, \text{PPRF.EvalPunc})$ where

- $\text{PPRF.Setup}(1^\lambda, 1^n, 1^m)$ is a randomized algorithm that takes as input the security parameter λ , an input length n , and an output length m , and outputs a key $K \in \mathcal{K}_{\lambda,n,m}$.
- $\text{PPRF.Eval}(K, x)$ is a deterministic algorithm that takes as input a key $K \in \mathcal{K}_{\lambda,n,m}$ and an input $x \in \{0,1\}^n$, and outputs a value $y \in \{0,1\}^m$.
- $\text{PPRF.Punc}(K, x^*)$ is a randomized algorithm that takes as input a key $K \in \mathcal{K}_{\lambda,n,m}$ and an input $x^* \in \{0,1\}^n$, and outputs a punctured key $K[x^*]$.
- $\text{PPRF.EvalPunc}(K[x^*], x)$ is a deterministic algorithm that takes as input a punctured key $K[x^*]$ and an input $x \in \{0,1\}^n$, and outputs either a value $y \in \{0,1\}^m$ or $\perp$.

We require correctness under puncturing, and selective pseudorandomness at punctured points.

Remark 3.4. For convenience, we will sometimes combine PPRF.Eval and PPRF.EvalPunc into one algorithm. This can be done by having the combined algorithm run PPRF.Eval if it receives a regular key K and run PPRF.EvalPunc if it receives a punctured key $K[x^*]$ since the two types of keys are easily distinguishable in the construction from [SW14]. When using the combined algorithm, we will overload notation and refer to it simply by PPRF.Eval .

Definition 3.5 (Correctness under Puncturing). For all $\lambda, n, m \in \mathbb{N}$ and all $x^* \in \{0,1\}^n$, if $K \leftarrow \text{PPRF.Setup}(1^\lambda, 1^n, 1^m)$ and $K[x^*] \leftarrow \text{PPRF.Punc}(K, x^*)$, then

$$\text{PPRF.EvalPunc}(K[x^*], x) = \begin{cases} \text{PPRF.Eval}(K, x) & \text{if } x \neq x^* \\ \perp & \text{else} \end{cases}$$

Definition 3.6 (Selective Pseudorandomness at Punctured Points). There exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and all PPT adversaries $\mathcal{A}$,

$$\left| \Pr[\text{Expt}_{\mathcal{A}}^{\text{PPRF}}(1^\lambda, 0) = 1] - \Pr[\text{Expt}_{\mathcal{A}}^{\text{PPRF}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0,1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{Expt}_{\mathcal{A}}^{\text{PPRF}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs an input size 1^n , an output size 1^m , and a message $x^* \in \{0, 1\}^n$.
2. **Compute Values:**
 - (a) $K \leftarrow \text{PPRF.Setup}(1^\lambda, 1^n, 1^m)$.
 - (b) $K[x^*] \leftarrow \text{PPRF.Punc}(K, x^*)$.
 - (c) If $b = 0$, send $(y, K[x^*])$ to $\mathcal{A}$ where $y = \text{PPRF.Eval}(K, x^*)$.
 - (d) If $b = 1$, send $(r, K[x^*])$ to $\mathcal{A}$ where $r \leftarrow \{0, 1\}^m$.
3. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Definition 3.7 (Symmetric Key Encryption (SKE)). A symmetric key encryption scheme with key space $\mathcal{K} = \{\mathcal{K}_{\lambda,n}\}_{\lambda,n \in \mathbb{N}}$ and ciphertext size $m(\cdot)$ is a tuple of PPT algorithms $\text{SKE} = (\text{SKE.Setup}, \text{SKE.Enc}, \text{SKE.Dec})$ where

- $\text{SKE.Setup}(1^\lambda, 1^n)$ is a randomized algorithm that takes as input the security parameter λ and an input length n and outputs a secret key $k \in \mathcal{K}_{\lambda,n}$
- $\text{SKE.Enc}(k, x)$ is a randomized algorithm that takes as input a secret key $k \in \mathcal{K}_{\lambda,n}$ and a message $x \in \{0, 1\}^n$ and outputs an encryption $\text{ct} \in \{0, 1\}^{m(\lambda,n)}$ of x .
- $\text{SKE.Dec}(k, \text{ct})$ is a deterministic algorithm that takes as input a secret key $k \in \mathcal{K}_{\lambda,n}$ and a ciphertext $\text{ct} \in \{0, 1\}^{m(\lambda,n)}$ and outputs a value $y \in \{0, 1\}^n$.

Correctness requires that for all $\lambda, n \in \mathbb{N}$ and every $x \in \{0, 1\}^n$,

$$\Pr \left[\text{SKE.Dec}(k, \text{SKE.Enc}(k, x)) = x : k \leftarrow \text{SKE.Setup}(1^\lambda, 1^n) \right] = 1$$

Security requires that there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and all PPT adversaries $\mathcal{A}$,

$$\left| \Pr[\text{Expt}_{\mathcal{A}}^{\text{SKE}}(1^\lambda, 0) = 1] - \Pr[\text{Expt}_{\mathcal{A}}^{\text{SKE}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{Expt}_{\mathcal{A}}^{\text{SKE}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ and outputs an input size 1^n .
2. **Setup:** $k \leftarrow \text{SKE.Setup}(1^\lambda, 1^n)$
3. **Challenge Message Queries:** The following can be repeated any polynomial number of times:
 - (a) $\mathcal{A}$ outputs a challenge message pair (x_0, x_1) where $x_0, x_1 \in \{0, 1\}^n$.
 - (b) $\text{ct}_b \leftarrow \text{SKE.Enc}(k, x_b)$
 - (c) Sent ct_b to $\mathcal{A}$.

4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

Definition 3.8 (Public Key Encryption (PKE)). *A public-key encryption (PKE) scheme is a tuple of PPT algorithms $\text{PKE} = (\text{PKE.Setup}, \text{PKE.Enc}, \text{PKE.Dec})$ with the following syntax:*

- $(\text{pk}, \text{sk}) \leftarrow \text{Setup}(1^\lambda)$: Takes as input the security parameter and samples a public/private key pair.
- $\text{ct} \leftarrow \text{Enc}(\text{pk}, m)$: Takes as input the public key and a message, and outputs a ciphertext encrypting the corresponding message.
- $m \leftarrow \text{Dec}(\text{sk}, \text{ct})$: Takes as input the private key and a ciphertext, and outputs a decrypted message.

Correctness requires that for all $\lambda \in \mathbb{N}$ and every m ,

$$\Pr \left[\text{PKE.Dec}(\text{sk}, \text{PKE.Enc}(\text{pk}, m)) = m : (\text{pk}, \text{sk}) \leftarrow \text{PKE.Setup}(1^\lambda) \right] = 1.$$

A PKE scheme PKE is IND-CPA secure if for all PPT adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, we have

$$\Pr \left[\begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{Setup}(1^\lambda); \\ (m_0, m_1, \text{st}) \leftarrow \mathcal{A}_1(\text{pk}); \\ b \leftarrow \{0, 1\}, \text{ct}^* \leftarrow \text{Enc}(\text{pk}, m_b); \\ b' \leftarrow \mathcal{A}_2(\text{st}, \text{ct}^*) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

3.1 Indistinguishability Obfuscation

The recent work of [JLS22] shows how to construct $i\mathcal{O}$ for P/Poly from well-established computational assumptions.

Definition 3.9 (Indistinguishability Obfuscation ($i\mathcal{O}$) for Circuits [JLS21]). *A uniform PPT algorithm $i\mathcal{O}$ is an indistinguishability obfuscator for polynomial-sized circuits if the following holds:*

- **Completeness:** For every $\lambda \in \mathbb{N}$, every circuit C with input length n , and every input $x \in \{0, 1\}^n$,

$$\Pr[C'(x) = C(x) : C' \leftarrow i\mathcal{O}(1^\lambda, C)] = 1$$

- **Indistinguishability:** For every two ensembles $\{C_{0,\lambda}\}, \{C_{1,\lambda}\}$ of polynomial-sized circuits that have the same size, input length, and output length, and are functionally equivalent, that is, $\forall \lambda \in \mathbb{N}, C_{0,\lambda}(x) = C_{1,\lambda}(x)$ for every input x , then for all polynomial-time, non-uniform adversaries $\mathcal{A}$, there exists a negligible function μ , such that for all λ ,

$$\left| \Pr[\mathcal{A}(1^\lambda, i\mathcal{O}(1^\lambda, C_{0,\lambda}))] = 1 - \Pr[\mathcal{A}(1^\lambda, i\mathcal{O}(1^\lambda, C_{1,\lambda}))] = 1 \right| \leq \mu(\lambda)$$

3.2 Functional Encryption

Here we give some fundamental definitions for functional encryption (FE) schemes. First, we define a class of functions parameterized by function size, input length, and output length.

Definition 3.10 (Function Class). *The function class $\mathcal{F}[n, \ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{R}}, \ell_{\mathcal{Y}}]$ is the set of all functions f that have a description $\hat{f} \in \{0, 1\}^{\ell_{\mathcal{F}}}$, take inputs in $\{0, 1\}^{\ell_{\mathcal{X}}}$, and output values in $\{0, 1\}^{\ell_{\mathcal{Y}}}$.*

Definition 3.11 (Public-Key Functional Encryption). A public-key functional encryption scheme for P/Poly is a tuple of PPT algorithms $\text{FE} = (\text{Setup}, \text{Enc}, \text{KeyGen}, \text{Dec})$ defined as follows:³

- $\text{Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}})$: takes the security parameter λ , a function size $\ell_{\mathcal{F}}$, an input size $\ell_{\mathcal{X}}$, and an output size $\ell_{\mathcal{Y}}$, and outputs the master public key mpk and the master secret key msk .
- $\text{Enc}(\text{mpk}, x)$: takes as input the master public key mpk and a message $x \in \{0, 1\}^{\ell_{\mathcal{X}}}$, and outputs an encryption ct of x .
- $\text{KeyGen}(\text{msk}, f)$: takes as input the master secret key msk and a function $f \in \mathcal{F}[n, \ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{R}}, \ell_{\mathcal{Y}}]$, and outputs a function key sk_f .
- $\text{Dec}(\text{sk}_f, \text{ct})$: takes a function key sk_f and a ciphertext ct , and outputs a value $y \in \{0, 1\}^{\ell_{\mathcal{Y}}}$.

FE satisfies **correctness** if for all polynomials p , there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, all $\ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{Y}} \leq p(\lambda)$, all $x \in \{0, 1\}^{\ell_{\mathcal{X}}}$, and all $f \in \mathcal{F}[n, \ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{R}}, \ell_{\mathcal{Y}}]$,

$$\Pr \left[\begin{array}{l} (\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}) \\ \text{ct}_x \leftarrow \text{Enc}(\text{mpk}, x) \\ \text{sk}_f \leftarrow \text{KeyGen}(\text{msk}, f) \end{array} : \text{Dec}(\text{sk}_f, \text{ct}_x) = f(x) \right] \geq 1 - \mu(\lambda).$$

We now define adaptive security.

Definition 3.12 (Adaptive Security for Public-Key FE). A public-key functional encryption scheme FE for P/Poly is **adaptively secure** if there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$ and every PPT adversary $\mathcal{A}$,

$$\left| \Pr[\text{Expt}_{\mathcal{A}}^{\text{FE-Adaptive}}(1^\lambda, 0) = 1] - \Pr[\text{Expt}_{\mathcal{A}}^{\text{FE-Adaptive}}(1^\lambda, 1) = 1] \right| \leq \mu(\lambda)$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{Expt}_{\mathcal{A}}^{\text{FE-Adaptive}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs a function size $1^{\ell_{\mathcal{F}}}$, an input size $1^{\ell_{\mathcal{X}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$.
2. **Setup:** Compute $(\text{mpk}, \text{msk}) \leftarrow \text{FE.Setup}(1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}})$.
3. **Public Key:** Send mpk to $\mathcal{A}$.
4. **Function Queries Phase 1:** The following is repeated any polynomial number of times:
 - (a) $\mathcal{A}$ outputs a function query $f \in \mathcal{F}[n, \ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{R}}, \ell_{\mathcal{Y}}]$
 - (b) $\text{sk}_f \leftarrow \text{FE.KeyGen}(\text{msk}, f)$
 - (c) Send sk_f to $\mathcal{A}$
5. **Challenge Message Query:**
 - (a) $\mathcal{A}$ outputs a challenge message pair (x_0, x_1) where $x_0, x_1 \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
 - (b) $\text{ct} \leftarrow \text{FE.Enc}(\text{mpk}, x_b)$
 - (c) Send ct to $\mathcal{A}$.

³We also allow Enc , KeyGen , and Dec to additionally receive parameters $1^\lambda, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{Y}}}$ as input, but omit them from our notation for convenience.

6. **Function Queries Phase 2:** This is identical to Function Queries Phase 1.

7. **Experiment Outcome:** A outputs a bit b' which is the output of the experiment.

Additionally, when running the experiment, we immediately halt and output 0 if the adversary ever aborts or if it at any point $f(x_0) \neq f(x_1)$ for some message query (x_0, x_1) and function query f submitted by the adversary.

Definition 3.13 (Other Public-Key FE Security Definitions). *There are many variations of the security definition. We list a few below:*

- **Semi-Adaptive Security:** The adversary is required to make the message query before the function queries. This is identical to adaptive security, except that we remove Function Queries Phase 1 from the security game.
- **Function-Selective Semi-Adaptive Security:** The adversary is required to make all function queries before the message query. This is identical to adaptive security, except that we remove Function Queries Phase 2 from the security game.
- **Selective Security:** The adversary is required to make the message query at the beginning of the experiment before receiving the master public key. This is similar to adaptive security, except that in the security game, we move the Challenge Message Query step so that it now lies between the Setup step and the Public Key step. Note that the two function query phases are now adjacent and can thus be merged into one step.
- **Function-Selective Security:** The adversary is required to make the function queries at the beginning of the experiment before receiving the master public key. This is similar to adaptive security, except that in the security game, we move the two function query steps so that they now lie between the Setup step and the Public Key step. Note that the two function query phases are now adjacent and can thus be merged into one step.

3.3 Non-Interactive Zero Knowledge Proof Systems

Definition 3.14 (Non-Interactive Zero Knowledge Proof). *Let $L \in \text{NP}$ and R_L be the corresponding NP relation. Let $\lambda \in \mathbb{N}$ be the security parameter. A Non-Interactive Zero Knowledge (NIZK) Proof is a tuple of algorithms $\Pi = (\text{Setup}, \text{Prove}, \text{Verify})$ with the following syntax:*

- $\text{crs} \leftarrow \text{Setup}(1^\lambda)$: Takes as input the security parameter 1^λ and outputs the common reference string crs .
- $\pi \leftarrow \text{Prove}(\text{crs}, x, w)$: Takes as input the common reference string crs , a statement x , and a witness w , and outputs a proof π .
- $1/0 \leftarrow \text{Verify}(\text{crs}, x, \pi)$: Takes as input the common reference string crs , a statement x and a proof π , outputs a single bit $1/0$ signaling whether the proof π is valid for the statement x .

We require the following properties of a NIZK scheme:

- **Perfect Completeness:** For all security parameters $\lambda \in \mathbb{N}$, and all $(x, w) \in R_L$, we have

$$\Pr \left[\text{Verify}(\text{crs}, x, \pi) = 1 : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda); \\ \pi \leftarrow \text{Prove}(\text{crs}, x, w) \end{array} \right] = 1.$$

- **Computational Adaptive Soundness:** For all PPT adversaries $\mathcal{A}$, we have

$$\Pr[\text{ExptSound}_{\Pi, \mathcal{A}}(1^\lambda) = 1] \leq \text{negl}(\lambda),$$

where the experiment ExptSound is defined as below:

$\text{ExptSound}_{\Pi, \mathcal{A}}(\lambda)$:

1. $\text{crs} \leftarrow \text{Setup}(1^\lambda)$;
2. $(x, \pi) \leftarrow \mathcal{A}(1^\lambda, \text{crs})$;
3. The experiment outputs 1 if and only if $x \notin L \wedge \text{Verify}(\text{crs}, x, \pi) = 1$.

- **Computational Zero-Knowledge:** There exists a pair of PPT simulators $\text{Sim} = (\text{Sim}_1, \text{Sim}_2)$ where $\text{Sim}_1(1^\lambda)$ outputs $(\widetilde{\text{crs}}, \tau)$ and $\text{Sim}_2(\widetilde{\text{crs}}, \tau, x)$ outputs a simulated proof $\tilde{\pi}$ such that for all non-uniform PPT adversaries $\mathcal{A}$:

$$\left| \Pr \left[\mathcal{A}^{\mathcal{O}_1(\text{crs}, \cdot, \cdot)}(\text{crs}) = 1 : \text{crs} \leftarrow \text{Setup}(1^\lambda) \right] - \Pr \left[\mathcal{A}^{\mathcal{O}_2(\text{crs}, \tau, \cdot, \cdot)}(\widetilde{\text{crs}}) = 1 : (\widetilde{\text{crs}}, \tau) \leftarrow \text{Sim}_1(1^\lambda) \right] \right| \leq \text{negl}(\lambda),$$

where $\mathcal{O}_1, \mathcal{O}_2$ on input (x, w) returns $\perp$ if $(x, w) \notin R_L$. Otherwise, $\mathcal{O}_1$ returns $\text{Prove}(\text{crs}, x, w)$ and $\mathcal{O}_2$ returns $\text{Sim}_2(\widetilde{\text{crs}}, \tau, x)$.

We can also require a NIZK to have *Simulation Soundness*, saying that the protocol still has soundness after the adversary sees *simulated* proofs.

Definition 3.15 (Unbounded Simulation Soundness). Let $\text{Sim} = (\text{Sim}_1, \text{Sim}_2)$ be the simulators for a NIZK protocol $\Pi = (\text{Setup}, \text{Prove}, \text{Verify})$ as defined in Computational Zero-Knowledge. We say Π has (unbounded) simulation soundness if for all PPT adversaries $\mathcal{A}$, we have

$$\Pr \left[(x, \pi) \notin Q \wedge x \notin \mathcal{L} \wedge \text{Verify}(\widetilde{\text{crs}}, x, \pi) = 1 : \begin{array}{l} (\widetilde{\text{crs}}, \tau) \leftarrow \text{Sim}_1(1^\lambda); \\ (x, \pi) \leftarrow \mathcal{A}^{\text{Sim}_2(\widetilde{\text{crs}}, \tau, \cdot)}(\widetilde{\text{crs}}) \end{array} \right] \leq \text{negl}(\lambda),$$

where Q denotes the set of all Sim_2 queries by $\mathcal{A}$ and their corresponding responses $(x_i, \tilde{\pi}_i)$.

4 Improved Security Definitions for Randomized (Multi-Input) Functional Encryption (rMIFE)

The syntax of rMIFE follows naturally from that of MIFE and FE for randomized functionalities.

Definition 4.1 (Randomized Function Class). The randomized function class $\mathcal{F}[n, \ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{R}}, \ell_{\mathcal{Y}}]$ is the set of all functions $f : (\{0, 1\}^{\ell_{\mathcal{X}}})^n \times \{0, 1\}^{\ell_{\mathcal{R}}} \rightarrow \{0, 1\}^{\ell_{\mathcal{Y}}}$ that have a description $\hat{f} \in \{0, 1\}^{\ell_{\mathcal{F}}}$. We interpret each function f as a randomized function with arity n that takes as input values $x_1, \dots, x_n$ where each $x_i \in \{0, 1\}^{\ell_{\mathcal{X}}}$ and randomness $r \in \{0, 1\}^{\ell_{\mathcal{R}}}$, and outputs a value $y \in \{0, 1\}^{\ell_{\mathcal{Y}}}$.

Definition 4.2 (rMIFE). A randomized multi-input functional encryption (rMIFE) scheme for P/Poly is a tuple of PPT algorithms $\text{rMIFE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ defined as follows:

- $\text{Setup}(1^\lambda, 1^n, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{R}}}, 1^{\ell_{\mathcal{Y}}})$: takes as input the security parameter λ , a function arity n , a function size $\ell_{\mathcal{F}}$, an input size $\ell_{\mathcal{X}}$, a randomness size $\ell_{\mathcal{R}}$, and an output size $\ell_{\mathcal{Y}}$, out outputs n encryption keys $\text{EK}_1, \dots, \text{EK}_n$ and the master secret key MSK .
- $\text{KeyGen}(\text{MSK}, f)$: takes as input the master secret key MSK and a function $f \in \mathcal{F}[n, \ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{R}}, \ell_{\mathcal{Y}}]$ and outputs a function key sk_f .
- $\text{Enc}(\text{EK}_j, x_j)$: takes as input an encryption key EK_j and an input $x_j \in \{0, 1\}^{\ell_{\mathcal{X}}}$ and outputs an encryption ct_j of x_j .
- $\text{Dec}(\text{sk}_f, \text{ct}_1, \dots, \text{ct}_n)$: takes as input a function key sk_f and ciphertexts $\text{ct}_1, \dots, \text{ct}_n$ and outputs a value $y \in \{0, 1\}^{\ell_{\mathcal{Y}}}$.

rMIFE satisfies **correctness** if for all polynomials p , there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, all $n, \ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{R}}, \ell_{\mathcal{Y}}, q \leq p(\lambda)$, all $\{f_k\}_{k \in [q]}$ where each $f_k \in \mathcal{F}[n, \ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{R}}, \ell_{\mathcal{Y}}]$, all $\{x_{i,1}, \dots, x_{i,n}\}_{i \in [q]}$ where each $x_{i,j} \in \{0, 1\}^{\ell_{\mathcal{X}}}$, and all PPT adversaries $\mathcal{A}$,

$$\left| \Pr[\text{Real}_{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Ideal}_{\mathcal{A}}(1^\lambda) = 1] \right| \leq \mu(1^\lambda)$$

where we define

- $\text{Real}_{\mathcal{A}}(1^\lambda)$
 1. $(\text{EK}_1, \dots, \text{EK}_n, \text{MSK}) \leftarrow \text{Setup}(1^\lambda)$.
 2. For $k \in [q]$, $\text{sk}_k \leftarrow \text{KeyGen}(\text{MSK}, f_k)$.
 3. For $i \in [q], j \in [n]$, $\text{ct}_{i,j} \leftarrow \text{Enc}(\text{EK}_j, x_{i,j})$.
 4. For $k, i_1, \dots, i_n \in [q]$, $y_{k,i_1, \dots, i_n} = \text{Dec}(\text{sk}_k, \text{ct}_{i_1,1}, \dots, \text{ct}_{i_n,n})$.
 5. Output $\mathcal{A}^{\mathcal{O}(\cdot)}(1^\lambda)$ where $\mathcal{O}(k, i_1, \dots, i_n) = y_{k,i_1, \dots, i_n}$.
- $\text{Ideal}_{\mathcal{A}}(1^\lambda)$
 1. For $k, i_1, \dots, i_n \in [q]$,
 - (a) $r_{k,i_1, \dots, i_n} \leftarrow \{0, 1\}^{\ell_{\mathcal{R}}}$
 - (b) $y_{k,i_1, \dots, i_n} = f_k(x_{i_1,1}, \dots, x_{i_n,n}; r_{k,i_1, \dots, i_n})$
 2. Output $\mathcal{A}^{\mathcal{O}(\cdot)}(1^\lambda)$ where $\mathcal{O}(k, i_1, \dots, i_n) = y_{k,i_1, \dots, i_n}$.

Definition 4.3 ($((\mathcal{A}, \epsilon)\text{-}\mathcal{I}\text{-randomized-compatible})$). Let $n, \ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{R}}, \ell_{\mathcal{Y}}, q_1, q_2 \in \mathbb{N}$.

- Let $\mathcal{I} \subseteq [n]$.
- Let $\{(x_{i,j}^{(0)}, x_{i,j}^{(1)})\}_{i \in [n], j \in [q_1]}$ be a set of inputs where each $x_{i,j}^{(b)} \in \{0, 1\}^{\ell_{\mathcal{X}}}$.
- Let $\{f_k\}_{k \in [q_2]}$ be a set of functions where each $f_k \in \mathcal{F}[n, \ell_{\mathcal{F}}, \ell_{\mathcal{X}}, \ell_{\mathcal{R}}, \ell_{\mathcal{Y}}]$.

We say that $\{x_{i,j}^{(0)}, x_{i,j}^{(1)}\}_{i \in [n], j \in [q_1]}$ is $\epsilon\text{-}\mathcal{I}\text{-randomized-compatible}$ with $\{f_k\}_{k \in [q_2]}$ if

- For all $U \subseteq \mathcal{I}$, all $\{x'_u\}_{u \in U}$ where each $x'_u \in \{0, 1\}^{\ell_{\mathcal{X}}}$, all $\{j_t\}_{t \in [n] \setminus U}$ where each $j_t \in [q_1]$, and all $k \in [q_2]$,

$$\left| \Pr[\mathcal{A}(f_k, U, \{x'_u\}_{u \in U}, \{x_{t,j_t}^{(0)}\}_{t \in [n] \setminus U}, f_k(\langle \{x'_u\}_{u \in U}, \{x_{t,j_t}^{(0)}\}_{t \in [n] \setminus U} \rangle)) = 1] \right. \\ \left. - \Pr[\mathcal{A}(f_k, U, \{x'_u\}_{u \in U}, \{x_{t,j_t}^{(1)}\}_{t \in [n] \setminus U}, f_k(\langle \{x'_u\}_{u \in U}, \{x_{t,j_t}^{(1)}\}_{t \in [n] \setminus U} \rangle)) = 1] \right| \leq \epsilon$$

where $\langle \{x'_u\}_{u \in U}, \{x_{t,j_t}^{(b)}\}_{t \in [n] \setminus U} \rangle$ is a permutation $(x_1, \dots, x_n)$ such that

$$x_i = \begin{cases} x'_i & \text{if } i \in U \\ x_{i,j_i}^{(b)} & \text{if } i \in [n] \setminus U \end{cases}$$

4.1 Indistinguishability-Based Definition of rMIFE

Definition 4.4 (Indistinguishability Based Security Against Malicious decryptors for rMIFE). A randomized multi-input functional encryption scheme $\text{rMIFE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ for $\mathcal{P}/\text{Poly}$ is *IND* secure against malicious receivers for $\epsilon = \epsilon(\lambda)$ -distinguishable distributions if for all sufficiently large λ , and all PPT adversaries $\mathcal{A}$,

$$\Pr[\text{Expt}_{\mathcal{A}}^{\text{rMIFE-Decryptors}}(1^\lambda) = 1] \leq \frac{1}{2} + \text{negl}(\lambda)$$

where we define

$\text{Expt}_{\mathcal{A}}^{\text{rMIFE-Decryptors}}(1^\lambda) :$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs an arity 1^n , a function size $1^{\ell_{\mathcal{F}}}$, an input size $1^{\ell_{\mathcal{X}}}$, a randomness size $1^{\ell_{\mathcal{R}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$.
2. **Setup:** $(\text{EK}_1, \dots, \text{EK}_n, \text{MSK}) \leftarrow \text{Setup}(1^\lambda, 1^n, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{R}}}, 1^{\ell_{\mathcal{Y}}})$.
We define $\vec{\text{EK}} = (\text{EK}_1, \dots, \text{EK}_n)$.
3. **Challenge Bit:** $b \leftarrow \{0, 1\}$.
4. **Adversary's Output:** $b' \leftarrow \mathcal{A}^{\text{OGetEK}(\vec{\text{EK}}, \cdot), \text{OEncLR}(\vec{\text{EK}}, b, \cdot), \text{KeyGen}(\text{MSK}, \cdot)}$.
5. **Experiment Output:** Output 1 if $b = b'$ and if $\{(x_{i,j}^{(0)}, x_{i,j}^{(1)})\}_{i \in [n], j \in [q_1]}$ are $(\mathcal{A}, \epsilon) - \mathcal{I}$ -randomized-compatible with $\{f_k\}_{k \in [q_2]}$ where
 - $\mathcal{I}$ are the set of queries made to OGetEK by $\mathcal{A}$.
 - $\{(x_{i,j}^{(0)}, x_{i,j}^{(1)})\}_{i \in [n], j \in [q_1]}$ are the set of queries made to OEncLR by $\mathcal{A}$.
 - $\{f_k\}_{k \in [q_2]}$ are the set of queries made to KeyGen by $\mathcal{A}$.

and where we define

$\text{OGetEK}(\vec{\text{EK}}, j) :$ Output EK_j .

$\text{OEncLR}(\vec{\text{EK}}, b, \{(x_i^{(0)}, x_i^{(1)})\}_{i \in [n]})$

1. For $i \in [n]$,
 - (a) $\text{ct}_i^{(b)} \leftarrow \text{MIFE.Enc}(\text{EK}_i, x_i^{(b)})$.
2. Output $\text{CT}_b = \{\text{ct}_i^{(b)}\}_{i \in [n]}$.

Definition 4.5 (Indistinguishability Based Security Against Malicious Encryptors for rMIFE). A randomized multi-input functional encryption scheme $\text{rMIFE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ for $\mathcal{P}/\text{Poly}$ is IND secure against malicious encryptors if there exists a PPT extractor Extr such that for all $\lambda \in \mathbb{N}$, and all PPT adversaries $\mathcal{A}$,

$$\Pr[\text{Expt}_{\mathcal{A}}^{\text{rMIFE-Encryptors}}(1^\lambda) = 1] \leq \frac{1}{2} + \text{negl}(\lambda)$$

where we define

$\text{Expt}_{\mathcal{A}}^{\text{rMIFE-Encryptors}}(1^\lambda) :$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ , and outputs an arity 1^n , a function size $1^{\ell_{\mathcal{F}}}$, an input size $1^{\ell_{\mathcal{X}}}$, a randomness size $1^{\ell_{\mathcal{R}}}$, and an output size $1^{\ell_{\mathcal{Y}}}$.
2. **Setup:** $(\text{EK}_1, \dots, \text{EK}_n, \text{MSK}) \leftarrow \text{Setup}(1^\lambda, 1^n, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{R}}}, 1^{\ell_{\mathcal{Y}}})$.
We define $\vec{\text{EK}} = (\text{EK}_1, \dots, \text{EK}_n)$.
3. **Challenge Bit:** $b \leftarrow \{0, 1\}$.
4. **Adversary's Output:**
 $b' \leftarrow \mathcal{A}^{\text{OGetEK}(\vec{\text{EK}}, \cdot), \text{OEnc}(\vec{\text{EK}}, \cdot), \text{KeyGen}(\text{MSK}, \cdot), \text{OKeyStore}(\text{MSK}, \cdot), \text{ODec}(\text{MSK}, b, \cdot)}$.
5. **Experiment Output:** Output 1 if $b = b'$.

and where we define

$\text{OGetEK}(\vec{\text{EK}}, j) :$ Output EK_j .

$\text{OEnc}(\vec{\text{EK}}, \{x_i\}_{i \in [n]})$

1. For $i \in [n]$,
(a) $\text{ct}_i \leftarrow \text{Enc}(\text{EK}_i, x_i)$.
2. Output $\text{CT} = \{\text{ct}_i\}$.

$\text{OKeyStore}(\text{MSK}, g) :$

1. $\text{sk}_g \leftarrow \text{KeyGen}(\text{MSK}, g)$.
2. Store (g, sk_g) in register KeyReg .
3. Output $\perp$.

$\text{ODec}(\text{MSK}, b, \{\text{ct}_i\}_{i \in [n]}) :$

1. For each (g_j, sk_{g_j}) stored in register KeyReg :
(a) If $(j, \{\text{ct}_i\}_{i \in [n]}, y)$ is stored in register OutReg for some y , output y .

- (b) Else if $b = 0$,
- i. $y_j = \text{Dec}(\text{sk}_{g_j}, \{\text{ct}_i\}_{i \in [n]})$.
 - ii. Store $(j, \{\text{ct}_i\}_{i \in [n]}, y_j)$ in OutReg.
- (c) Else if $b = 1$,
- i. For $i \in [n]$, $x_i \leftarrow \text{Extr}(\text{MSK}, \text{ct}_i)$.
 - ii. $r_j \leftarrow \{0, 1\}^{\ell_{\mathcal{R}}}$.
 - iii. $y_j \leftarrow g_j(\{x_i\}_{i \in [n]}; r_j)$.
 - iv. Store $(j, \{\text{ct}_i\}_{i \in [n]}, y_j)$ in OutReg.
2. Output $\{y_j\}$.

4.2 Simulation-Based Definition of rMIFE

Definition 4.6 (Simulation-based Security for rMIFE). *A randomized multi-input functional encryption scheme $\text{rMIFE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ for P/Poly is (q_1, q_c, q_2) simulation-secure against both malicious encryptors and decryptors if there exists a PPT simulator $S = (S_1, S_2, \dots, S_5)$ such that for all sufficiently large λ , and all PPT adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ where $\mathcal{A}_1$ makes at most q_1 key-generation queries and $\mathcal{A}_2$ makes at most q_2 key-generation queries, we have*

$$\left| \Pr[\text{Expt}_{\mathcal{A}}^{\text{rMIFE}, \text{Real}}(1^\lambda) = 1] - \Pr[\text{Expt}_{\mathcal{A}, S}^{\text{rMIFE}, \text{Ideal}}(1^\lambda) = 1] \right| \leq \text{negl}(\lambda)$$

where we define,

- $\text{Expt}_{\mathcal{A}}^{\text{rMIFE}, \text{Real}}(1^\lambda)$
 1. $(\text{EK}_1, \dots, \text{EK}_n, \text{MSK}) \leftarrow \text{Setup}(1^\lambda, 1^n, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{R}}}, 1^{\ell_{\mathcal{Y}}})$.
We define $\vec{\text{EK}} = (\text{EK}_1, \dots, \text{EK}_n)$
 2. $(\{(x_{i,j})\}_{i \in [n], j \in [q_c]}, st) \leftarrow \mathcal{A}_1^{\text{OGetEK}(\vec{\text{EK}}, \cdot), \text{KeyGen}(\text{MSK}, \cdot), \text{OKeyStore}(\text{MSK}, \cdot), \text{ODec}(\text{MSK}, \cdot)}(1^\lambda)$.
 3. $\text{ct}_{i,j}^* \leftarrow \text{Enc}(\text{EK}_i, x_{i,j})$ for all $i \in [n], j \in [q_c]$.
 4. $\alpha \leftarrow \mathcal{A}_2^{\text{OGetEK}(\vec{\text{EK}}, \cdot), \text{KeyGen}(\text{MSK}, \cdot), \text{OKeyStore}(\text{MSK}, \cdot), \text{ODec}(\text{MSK}, \cdot)}(\{\text{ct}_{i,j}^*\}_{i \in [n], j \in [q_c]}, st)$.
 5. **Output** $(\{(x_{i,j})\}_{i \in [n], j \in [q_c]}, \{f_k\}_{k \in [q_1 + q_2]}, \{g\}, \{y\}, \alpha)$.
- $\text{Expt}_{\mathcal{A}, S}^{\text{rMIFE}, \text{Ideal}}(1^\lambda)$
 1. $st' \leftarrow S_1(1^\lambda, 1^n, 1^{\ell_{\mathcal{F}}}, 1^{\ell_{\mathcal{X}}}, 1^{\ell_{\mathcal{R}}}, 1^{\ell_{\mathcal{Y}}})$.
 2. $(\{(x_{i,j})\}_{i \in [n], j \in [q_c]}, st) \leftarrow \mathcal{A}_1^{\text{O'GetEK}(st', \cdot), \text{O'KeyGen}_1(st', \cdot), \text{O'KeyStore}(st', \cdot), \text{ODec}'(st', \cdot)}(1^\lambda)$.
 - Let $\{f_k\}_{k \in [q_1]}$ be $\mathcal{A}_1$'s oracle queries to $\text{O'KeyGen}_1(st', \cdot)$.
 - Pick $r_{j_1, \dots, j_n, k} \leftarrow \{0, 1\}^{\ell_{\mathcal{R}}}$ and let $y_{j_1, \dots, j_n, k} = f_k(x_{1, j_1}, \dots, x_{n, j_n}; r_{j_1, \dots, j_n, k})$ for all $j_1, \dots, j_n \in [q_c], k \in [q_1]$.
 3. $(\{ct_{i,j}^*\}_{i \in [n], j \in [q_c]}, st') \leftarrow S_3(st', \{y_{j_1, \dots, j_n, k}\}_{j_1, \dots, j_n \in [q_c], k \in [q_1]})$.
 4. $\alpha \leftarrow \mathcal{A}_2^{\text{O'GetEK}(st', \cdot), \text{O'KeyGen}_2(st', \cdot), \text{O'KeyStore}(st', \cdot), \text{ODec}'(st', \cdot)}(\{\text{ct}_{i,j}^*\}_{i \in [n], j \in [q_c]}, st)$.
- **Output** $(\{(x_{i,j})\}_{i \in [n], j \in [q_c]}, \{f_k\}_{k \in [q_1 + q_2]}, \{g\}, \{y'\}, \alpha)$.

and where we define

In Real Experiment:

- $\text{OGetEK}(\overrightarrow{\text{EK}}, i)$: Output EK_i .
- $\text{OKeyStore}(\text{MSK}, g)$:
 1. $\text{sk}_g \leftarrow \text{KeyGen}(\text{MSK}, g)$.
 2. Store (g, sk_g) in register **KeyReg**.
 3. Output $\perp$.
- $\text{ODec}(\{\text{ct}_{i,j}^*\}_{i \in [n], j \in [q_c]}, \{\text{ct}_i\}_{i \in [n]})$:
 1. For each (g_l, sk_{g_l}) stored in register **KeyReg**: $y_l = \text{Dec}(\text{sk}_{g_l}, \{\text{ct}_i\}_{i \in [n]})$.
 2. Output $\{y_l\}$.

In Ideal Experiment:

- $\text{O'GetEK}(\text{st}', i)$: Output EK'_i
- $\text{O'KeyGen}_1(\text{st}', f_k)$
 1. $\text{sk}'_{f_k} \leftarrow S_2(\text{st}', f_k)$
 2. Output $\{\text{sk}'_{f_k}\}$
- $\text{O'KeyGen}_2(\text{st}', f_k)$
 1. $\text{sk}'_{f_k} \leftarrow S_4(\text{st}', f_k, \text{KeyIdeal}(\{x_{i,j}\}_{i \in [n], j \in [q_c]}))$
 2. Output sk'_{f_k}
 3. $\text{KeyIdeal}(\{x_{i,j}\}_{i \in [n], j \in [q_c]})$
 - (a) $r_{j_1, \dots, j_n, k} \leftarrow \{0, 1\}^{\ell_{\mathcal{R}}}, j_n \in [q_c]$
 - (b) $y_{j_1, \dots, j_n, k} = f_k(x_{1, j_1}, \dots, x_{n, j_n}; r_{j_1, \dots, j_n, k})$ for all $j_1, \dots, j_n \in [q_c]$
 - (c) Output $\{y_{j_1, \dots, j_n, k}\}$ for all $j_1, \dots, j_n \in [q_c]$.
- $\text{O'KeyStore}(\text{st}', g)$:
 1. Store $(g, \perp)$ in register **KeyReg**.
 2. Output $\perp$.
- $\text{O'Dec}(\text{st}', \{\text{ct}_{i,j}^*\}_{i \in [n], j \in [q_c]}, \{\text{ct}_i\}_{i \in [n]})$:
 1. For each $(g_l, \perp)$ stored in register **KeyReg**:
 - (a) If $(l, \{\text{ct}_i\}_{i \in [n]}, y)$ is stored in register **OutReg** for some y , output y .
 - (b) For $i \in [n]$, $x_i = S_5(\text{st}', \text{ct}_i)$.
 - (c) $r_l \leftarrow \{0, 1\}^{\ell_{\mathcal{R}}}$.

- (d) $y_l \leftarrow g_l(\{x_i\}_{i \in [n]}; r_l)$.
(e) Store $(l, \{\text{ct}_i\}_{i \in [n]}, y_l)$ in OutReg.
2. Output $\{y_l\}$.

Remark 4.7 (On Imposing Equivalence Relations over Ciphertexts.). The work of [AW17] define randomized (single-input) functional encryption with respect to “admissible ciphertext equivalence relations.”

In more detail, in their scheme (and some prior schemes such as [GJKS15]), it may be the case that $\text{Dec}(\text{sk}_f, \text{ct}) = \text{Dec}(\text{sk}_f, \text{ct}')$ even though $\text{ct} \neq \text{ct}'$. This could occur for example if the ciphertext $\text{ct} = (c, \pi)$ where π only serves to prove the validity of c , but does not otherwise contribute to the decryption output. Thus, if we were to generate a different proof π' attesting to the validity of c , then we could have two different valid ciphertexts $\text{ct} = (c, \pi)$ and $\text{ct}' = (c, \pi')$ that decrypt to the same value on every function key.

However, this means that their scheme would be insecure if we define security against malicious encryptors in the manner which we have done. This is because the decryption oracle ODec' in the ideal world will output independently generated values whenever it decrypts two different ciphertexts ct and ct' . However, in the real world, the decryption of these two ciphertexts may be the same value, leading to a trivial distinguisher between the real and ideal worlds.

To handle this, [AW17] propose the notion of an admissible ciphertext equivalence relation which is an efficiently checkable relation where two ciphertexts are considered equivalent if they decrypt to the same value.⁴⁵ Then, in their security game, they require that the adversary does not query the decryption oracle on two different ciphertexts from the same equivalence class. This restriction allows them to prove the security of their scheme under some reasonable notion of security.

Although our definition of security is different from that of [AW17], we believe that both [AW17] and [GJKS15] will be secure under our definitions if we additionally add the restriction that an adversary cannot query the decryption oracle on two different ciphertexts from the same admissible equivalence class.

We further remark that the rMIFE scheme we construct in this paper does not require this restriction or any notion of ciphertext equivalence relations and can be proven secure as per our main definitions given above.

Remark 4.8 (On SIM Security of [GGG⁺14, GJKS15, AW17] under our new definition.). As mentioned above, the existing rFE/rMIFE constructions from [GGG⁺14, GJKS15, AW17], which were proven secure under the previous SIM-security definition, also achieve SIM-security under our new definition (of course with the equivalence relations restriction mentioned in 4.7). Observe that the key distinction between our new definition and the prior one lies in the handling of secret keys within the decryption oracle. In previous definitions, the decryption oracle generates a fresh secret key for the function each time a decryption is performed involving that function. In contrast, under our new definition, the secret key for the function is generated once and reused for subsequent decryptions involving that function. Importantly, the security proofs in the prior works [GGG⁺14, GJKS15, AW17] do not rely on the generation of fresh secret keys for each decryption by the decryption oracle. As a result, their security proofs should naturally extend to our new definition without modification.

⁴⁵In more detail, consider an equivalence relation $\sim$ on the ciphertext space. We say that $\sim$ is admissible if it is efficiently checkable and if for every two ciphertexts $\text{ct}^{(0)}$ and $\text{ct}^{(1)}$, then $\text{ct}^{(0)} \sim \text{ct}^{(1)}$ if and only if for any honestly generated function key sk_f , one of the following holds: (1) $\text{Dec}(\text{sk}_f, \text{ct}^{(0)}) = \perp$ or $\text{Dec}(\text{sk}_f, \text{ct}^{(1)}) = \perp$, or (2) $\text{Dec}(\text{sk}_f, \text{ct}^{(0)}) = \text{Dec}(\text{sk}_f, \text{ct}^{(1)})$.

⁵This notion can also be extended to the multi-input setting by defining an equivalence relation on sets $\{\text{ct}_i\}_{i \in [n]}$ of ciphertext queries where n is the arity of the function.

Remark 4.9 (On submitting multiple ciphertexts to the decryption oracle.). Recall that [AW17] extends the original SIM security definition from [GJKS15] by allowing the adversary to submit multiple ciphertexts to the decryption oracle simultaneously, rather than one at a time. As noted in [AW17], this modification captures security against adversarially generated correlated ciphertexts, which could potentially influence the decryption outputs.

In contrast, our definition, similar to [GJKS15], allows the adversary to submit a single ciphertext (or a single tuple of ciphertexts in the multi-input case) to the decryption oracle at a time. However, this does not make our model more restrictive compared to [AW17]. In their model, the decryption oracle generates a fresh secret key for each randomized function every time a decryption query is made, and this key is used to decrypt the set of ciphertexts submitted along with the function. In our definition, the same secret keys are used consistently for decrypting all ciphertexts submitted over time.

Therefore, our model naturally addresses attacks involving adversarially generated correlated ciphertexts, even though we submit one ciphertext at a time to the decryption oracle.

Lemma 4.10 (SIM-security implies IND-security for rFE/rMIFE.). *Let $\text{rMIFE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ be SIM-secure as per Definition 4.6. Then, rMIFE is IND-secure against both malicious encryptors and malicious decryptors as per Definitions 4.4 and 4.5, respectively.*

The proof that SIM-security of rMIFE as per definition 4.6 implies IND-security against malicious decryptors (definition 4.4) is essentially the same as the one in [GJKS15]. Please refer to the proof of Lemma 2.9, Appendix C in [GJKS15] for details.

To show that SIM-security implies IND-security against malicious encryptors (Definition 4.5), we proceed as follows. Observe that $\text{Expt}_{\mathcal{A}}^{\text{rMIFE-Encryptors}}(1^\lambda)$ with challenge bit $b = 0$ is equivalent to $\text{Expt}_{\mathcal{A}}^{\text{rMIFE,Real}}(1^\lambda)$ where the adversary issues a zero-challenge ciphertext query. Similarly, $\text{Expt}_{\mathcal{A}}^{\text{rMIFE-Encryptors}}(1^\lambda)$ with challenge bit $b = 1$ is identical to $\text{Expt}_{\mathcal{A},\mathcal{S}}^{\text{rMIFE,Ideal}}(1^\lambda)$ under the same zero-challenge query.

Since $\text{Expt}_{\mathcal{A}}^{\text{rMIFE,Real}}(1^\lambda)$ and $\text{Expt}_{\mathcal{A},\mathcal{S}}^{\text{rMIFE,Ideal}}(1^\lambda)$ are computationally indistinguishable under the SIM-security guarantee, it follows that $\text{Expt}_{\mathcal{A}}^{\text{rMIFE-Encryptors}}(1^\lambda)$ with $b = 0$ and $b = 1$ are also computationally indistinguishable. Hence, SIM-security implies IND-security against malicious encryptors.

5 Counterexample

In this section, we provide a counterexample showcasing the issue with the *indistinguishability-based* definition of Functional Encryption for Randomized Functionalities as presented in the work by Goyal, Jain, Koppula, and Sahai [GJKS15]. This definition has been used in a line of follow-up works, and therefore we think it is crucial that the insufficiency of the definition be pointed out.

5.1 Definition in [GJKS15]

First, we recall the indistinguishability-based definition in [GJKS15]. [GJKS15] provides two IND definitions, corresponding to whether the adversary needs to submit the function queries before or after receiving the master public key mpk . Here we focus on the more adaptive IND_{post} security, where the adversary sends function queries after receiving the master public key.

Definition 5.1 (IND-based Definition of rFE as in [GJKS15]). *A functional encryption scheme for randomized functionalities $\text{rFE} = (\text{Setup}, \text{Enc}, \text{KeyGen}, \text{Dec})$ is IND-secure if for all PPT adversary*

$\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)^6$, we have

$$\Pr[\text{Expt}_{\mathcal{A}}^{\text{rFE}}(1^\lambda) = 1] \leq \frac{1}{2} + \text{negl}(\lambda),$$

where $\text{Expt}_{\mathcal{A}}^{\text{rFE}}$ is defined as below.

$\text{Expt}_{\mathcal{A}}^{\text{rFE}}(1^\lambda) :$

1. $(x_0, x_1, \text{st}) \leftarrow \mathcal{A}_1(1^\lambda)$.
2. Sample $b \leftarrow \{0, 1\}$ and $(\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda)$, and compute $\text{CT}^* \leftarrow \text{Enc}(\text{mpk}, x_b)$.
3. $b' \leftarrow \mathcal{A}_2^{\text{KeyGen}(\text{msk}, \cdot), \mathcal{O}(\text{msk}, \cdot, \cdot)}(\text{mpk}, \text{CT}^*, \text{st})$, where KeyGen is the key generation oracle, and $\mathcal{O}$ is defined as below.
4. Let $\{f\}$ be the set of function queries by $\mathcal{A}_2$ to the key generation oracle. We require that the distributions $(\text{mpk}, \text{st}, \{f(x_0)\})$ and $(\text{mpk}, \text{st}, \{f(x_1)\})$ are statistically indistinguishable^a. If not, the experiment aborts and outputs 0.
5. If $b' = b$, the adversary wins and the game outputs 1. Otherwise, the adversary loses and the game outputs 0.

^aStatistical indistinguishability is necessary here to prevent circularity. If we only require *computational* indistinguishability, $\mathcal{A}_2$ can submit a query $f = \text{Enc}(\text{mpk}, \cdot)$. Then, the indistinguishability requirement on these two distributions will be the same as the desired security requirement for the challenge ciphertext, making this a vacuous definition.

$\mathcal{O}_{\text{CT}^*}(\text{msk}, \text{CT}, g) :$

1. If $\text{CT} = \text{CT}^*$, return $\perp$.
2. Compute $\text{SK}_g \leftarrow \text{KeyGen}(\text{msk}, g)$.
3. Return $\text{Dec}(\text{SK}_g, \text{CT})$.

5.2 Construction of Counterexample

Now, we present our construction of a rFE scheme that satisfies Definition 5.1, but is insecure. We use a number of tools, the definitions of which can be found in Section 3.

Construction 1 (Counterexample rFE). Let $\text{PKE} = (\text{Setup}, \text{Enc}, \text{Dec})$ be a CPA-secure PKE scheme, $\text{SKE} = (\text{Setup}, \text{Enc}, \text{Dec})$ be a CPA-secure symmetric key encryption, $\text{NIZK} = (\text{Setup}, \text{Prove}, \text{Verify})$ be a NIZK with simulation soundness, $\text{FE} = (\text{Setup}, \text{Enc}, \text{KeyGen}, \text{Dec})$ be a plain Functional Encryption scheme with selective security, and PRF be a secure PRF with output length $\ell_{\text{PRF}} = \text{poly}(\lambda)$. We construct our randomized FE scheme $\text{rFE} = (\text{Setup}, \text{Enc}, \text{KeyGen}, \text{Dec})$ as follows:

- $\text{Setup}(1^\lambda) :$

⁶The original definition in [GJKS15] considers non-uniform attackers to allow for a more general definition, but the non-uniformity is never utilized anywhere in the proofs. So here we present the definition and counterexample against uniform attackers, but bear in mind that it can be easily adapted to allow for non-uniform attackers if one desires so.

1. $(\text{PKE.pk}, \text{PKE.sk}) \leftarrow \text{PKE.Setup}(1^\lambda)$.
2. $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
3. $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
4. $\text{crs} \leftarrow \text{NIZK.Setup}(1^\lambda)$.
5. Output $(\text{mpk}, \text{msk}) = ((\text{FE.mpk}, \text{PKE.pk}, \text{crs}), (\text{FE.mpk}, \text{PKE.pk}, \text{crs}, \text{FE.msk}, \text{PKE.sk}, \text{SKE.sk}))$.

• $\text{Enc}(\text{mpk}, x)$:

1. Parse $\text{mpk} = (\text{FE.mpk}, \text{PKE.pk}, \text{crs})$.
2. $K \leftarrow \text{PRF.Setup}(1^\lambda)$.
3. $\text{FE.ct} \leftarrow \text{FE.Enc}(\text{FE.mpk}, (x, K, 0, \perp))$.
4. $\text{PKE.ct} \leftarrow \text{PKE.Enc}(\text{PKE.pk}, (x, K, 0, \perp, \text{FE.ct}))$.
5. $\pi \leftarrow \text{NIZK.Prove}(\text{crs}, z, w)$ where z is the following statement on FE.ct and PKE.ct :
“PKE.ct correctly encrypts $(x, K, \alpha, \hat{\text{sk}}, \text{FE.ct})$, where $(x, K, \alpha, \hat{\text{sk}})$ is encrypted in FE.ct”
The witness w for the statement is the tuple $(x, K, \alpha = 0, \hat{\text{sk}} = \perp)$.
6. Output $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$.

• $\text{KeyGen}(\text{msk}, f)$:

1. Parse $\text{msk} = (\text{FE.mpk}, \text{PKE.pk}, \text{crs}, \text{FE.msk}, \text{PKE.sk}, \text{SKE.sk})$.
2. $s \leftarrow \{0, 1\}^\lambda$.
3. $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|f(\cdot)|}$ is randomly sampled.
4. $\text{FE.sk}_{G_f} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[f, s, \text{SKE.ct}])$.
5. Output $\text{SK}_f = (\text{crs}, \text{FE.sk}_{G_f})$.

$G[f, s, \text{SKE.ct}](x, K, \alpha, \hat{\text{sk}})$:

1. If $\alpha = 0$:
 - (a) $r \leftarrow \text{PRF.Eval}(K, s)$.
 - (b) Output $f(x; r)$.
2. If $\alpha = 1$:
 - (a) Output $\text{SKE.Dec}(\hat{\text{sk}}, \text{SKE.ct})$.

• $\text{Dec}(\text{SK}_f, \text{CT})$:

1. Parse $\text{SK}_f = (\text{crs}, \text{FE.sk}_{G_f})$ and $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$.
2. If $\text{NIZK.Verify}(\text{crs}, (\text{FE.ct}, \text{PKE.ct}), \pi) = 0$, abort and output $\perp$.
3. Output $y = \text{FE.Dec}(\text{FE.sk}_{G_f}, \text{FE.ct})$.

Correctness follows from the correctness of the underlying FE scheme.

5.3 Proof of (Insufficient) Security

In this section, we prove (insufficient) security of Construction 1 according to Definition 5.1.

Theorem 5.2. *If PKE is a CPA-secure PKE, SKE is a CPA-secure SKE, NIZK is a NIZK with simulation soundness, FE is a selectively-secure FE scheme, and PRF is a secure PRF, then Construction 1 is IND-secure per Definition 5.1.*

We prove this through a hybrid proof. We first lay out the sequence of hybrids, followed by proofs of each hybrid argument.

Sequence of Hybrids

Hybrid₀^A: The same as $\text{Expt}_{\mathcal{A}}^{\text{rFE}}$ where the challenge bit b is fixed to be 0. More specifically:

1. $(x_0, x_1, \text{st}) \leftarrow \mathcal{A}_1(1^\lambda)$.
2. Sample $(\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda)$ as follows:
 - (a) $(\text{PKE.pk}, \text{PKE.sk}) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 - (b) $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 - (c) $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (d) $\text{crs} \leftarrow \text{NIZK.Setup}(1^\lambda)$.
 - (e) $(\text{mpk}, \text{msk}) := ((\text{FE.mpk}, \text{PKE.pk}, \text{crs}), (\text{FE.mpk}, \text{PKE.pk}, \text{crs}, \text{FE.msk}, \text{PKE.sk}, \text{SKE.sk}))$.
3. Compute $\text{CT}^* \leftarrow \text{Enc}(\text{mpk}, x_0)$ as follows:
 - (a) $K \leftarrow \text{PRF.Setup}(1^\lambda)$.
 - (b) $\text{FE.ct}^* \leftarrow \text{FE.Enc}(\text{FE.mpk}, (x_0, K, 0, \perp))$.
 - (c) $\text{PKE.ct}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, (x_0, K, 0, \perp, \text{FE.ct}^*))$.
 - (d) $\pi^* \leftarrow \text{NIZK.Prove}(\text{crs}, z, w)$ where z is the following statement on FE.ct^* and PKE.ct^* :
 “ PKE.ct^* correctly encrypts $(x, K, \alpha, \hat{\text{sk}}, \text{FE.ct}^*)$, where $(x, K, \alpha, \hat{\text{sk}})$ is encrypted in FE.ct^* ”

The witness w is the tuple $(x_0, K, 0, \perp)$.
- (e) Set $\text{CT}^* = (\text{FE.ct}^*, \text{PKE.ct}^*, \pi^*)$.
4. $b' \leftarrow \mathcal{A}_2^{\text{KeyGen}(\text{msk}, \cdot), \mathcal{O}(\text{msk}, \cdot, \cdot)}(\text{mpk}, \text{CT}^*, \text{st})$, where KeyGen and $\mathcal{O}$ are oracles defined as below:
 - $\text{KeyGen}(\text{msk}, f)$:
 - (a) $s \leftarrow \{0, 1\}^\lambda$.
 - (b) $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|f(\cdot)|}$ is randomly sampled.
 - (c) $\text{FE.sk}_{G_f} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[f, s, \text{SKE.ct}])$.
 - (d) Output $\text{SK}_f = (\text{crs}, \text{FE.sk}_{G_f})$.
 - $\mathcal{O}(\text{msk}, \text{CT}, g)$:
 - (a) If $\text{CT} = \text{CT}^*$, return $\perp$.
 - (b) $s \leftarrow \{0, 1\}^\lambda$.
 - (c) $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|g(\cdot)|}$ is randomly sampled.
 - (d) $\text{FE.sk}_{G_g} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[g, s, \text{SKE.ct}])$.

- (e) Parse $CT = (FE.ct, PKE.ct, \pi)$.
 - (f) If $NIZK.Verify(crs, (FE.ct, PKE.ct), \pi) = 0$, return $\perp$.
 - (g) Return $FE.Dec(SK_{G_g}, FE.ct)$.
5. Let $\{f\}$ be the set of function queries by $\mathcal{A}_2$ to the **KeyGen** oracle. We require that the distributions $(mpk, st, \{f(x_0)\})$ and $(mpk, st, \{f(x_1)\})$ are statistically indistinguishable. If not, the experiment aborts and outputs 0.
 6. If $b' = b$, the adversary wins and the game outputs 1. Otherwise, the adversary loses and the game outputs 0.

Hybrid₁^A: Instead of sampling crs using $NIZK.Setup$, now sample a simulated $\widetilde{crs}$ using $NIZK.Sim_1(1^\lambda)$. And later replace $NIZK.Prove$ with $NIZK.Sim_2$. This step follows from the computational zero-knowledge of the underlying NIZK.

1. $(x_0, x_1, st) \leftarrow \mathcal{A}_1(1^\lambda)$.
2. Sample $(mpk, msk) \leftarrow Setup(1^\lambda)$ as follows:
 - (a) $(PKE.pk, PKE.sk) \leftarrow PKE.Setup(1^\lambda)$.
 - (b) $SKE.sk \leftarrow SKE.Setup(1^\lambda)$.
 - (c) $(FE.mpk, FE.msk) \leftarrow FE.Setup(1^\lambda)$.
 - (d) $(\widetilde{crs}, \tau) \leftarrow NIZK.Sim_1(1^\lambda)$.
 - (e) $(mpk, msk) := ((FE.mpk, PKE.pk, \widetilde{crs}), (FE.mpk, PKE.pk, \widetilde{crs}, FE.msk, PKE.sk, SKE.sk))$.
3. Compute $CT^* \leftarrow Enc(mpk, x_0)$ as follows:
 - (a) $K \leftarrow PRF.Setup(1^\lambda)$.
 - (b) $FE.ct^* \leftarrow FE.Enc(FE.mpk, (x_0, K, 0, \perp))$.
 - (c) $PKE.ct^* \leftarrow PKE.Enc(PKE.pk, (x_0, K, 0, \perp, FE.ct^*))$.
 - (d) $\pi^* \leftarrow NIZK.Sim_2(\widetilde{crs}, \tau, z)$ where z is the following statement on $FE.ct^*$ and $PKE.ct^*$:
 “ $PKE.ct^*$ correctly encrypts $(x, K, \alpha, \widehat{sk}, FE.ct^*)$, where $(x, K, \alpha, \widehat{sk})$ is encrypted in $FE.ct^*$ ”
~~The witness w is the tuple $(x_0, K, 0, \perp)$.~~
 - (e) Set $CT^* = (FE.ct^*, PKE.ct^*, \pi^*)$.
4. $b' \leftarrow \mathcal{A}_2^{KeyGen(msk, \cdot), \mathcal{O}(msk, \cdot, \cdot)}(mpk, CT^*, st)$, where **KeyGen** and $\mathcal{O}$ are oracles defined as below:
 - **KeyGen**(msk, f):
 - (a) $s \leftarrow \{0, 1\}^\lambda$.
 - (b) $SKE.ct \leftarrow SKE.Enc(SKE.sk, y)$, where $y \leftarrow \{0, 1\}^{|f(\cdot)|}$ is randomly sampled.
 - (c) $FE.sk_{G_f} \leftarrow FE.KeyGen(FE.msk, G[f, s, SKE.ct])$.
 - (d) Output $SK_f = (\widetilde{crs}, FE.sk_{G_f})$.
 - $\mathcal{O}(msk, CT, g)$:
 - (a) If $CT = CT^*$, return $\perp$.

- (b) $s \leftarrow \{0, 1\}^\lambda$.
 - (c) $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|g(\cdot)|}$ is randomly sampled.
 - (d) $\text{FE.sk}_{G_g} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[g, s, \text{SKE.ct}])$.
 - (e) Parse $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$.
 - (f) If $\text{NIZK.Verify}(\widetilde{\text{crs}}, (\text{FE.ct}, \text{PKE.ct}), \pi) = 0$, return $\perp$.
 - (g) Return $\text{FE.Dec}(\text{SK}_{G_g}, \text{FE.ct})$.
5. Let $\{f\}$ be the set of function queries by $\mathcal{A}_2$ to the **KeyGen** oracle. We require that the distributions $(\text{mpk}, \text{st}, \{f(x_0)\})$ and $(\text{mpk}, \text{st}, \{f(x_1)\})$ are statistically indistinguishable. If not, the experiment aborts and outputs 0.
 6. If $b' = b$, the adversary wins and the game outputs 1. Otherwise, the adversary loses and the game outputs 0.

Hybrid₂⁴: When generating the challenge ciphertext, change PKE.ct^* into an encryption of $\perp$. This step follows from CPA security of the underlying PKE scheme.

1. $(x_0, x_1, \text{st}) \leftarrow \mathcal{A}_1(1^\lambda)$.
2. Sample $(\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda)$ as follows:
 - (a) $(\text{PKE.pk}, \text{PKE.sk}) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 - (b) $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 - (c) $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (d) $(\widetilde{\text{crs}}, \tau) \leftarrow \text{NIZK.Sim}_1(1^\lambda)$.
 - (e) $(\text{mpk}, \text{msk}) := ((\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}), (\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}, \text{FE.msk}, \text{PKE.sk}, \text{SKE.sk}))$.
3. Compute $\text{CT}^* \leftarrow \text{Enc}(\text{mpk}, x_0)$ as follows:
 - (a) $K \leftarrow \text{PRF.Setup}(1^\lambda)$.
 - (b) $\text{FE.ct}^* \leftarrow \text{FE.Enc}(\text{FE.mpk}, (x_0, K, 0, \perp))$.
 - (c) $\text{PKE.ct}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, \perp)$.
 - (d) $\pi^* \leftarrow \text{NIZK.Sim}_2(\widetilde{\text{crs}}, \tau, z)$ where z is the following statement on FE.ct^* and PKE.ct^* :
 “ PKE.ct^* correctly encrypts $(x, K, \alpha, \widehat{\text{sk}}, \text{FE.ct}^*)$, where $(x, K, \alpha, \widehat{\text{sk}})$ is encrypted in FE.ct^* ”
 - (e) Set $\text{CT}^* = (\text{FE.ct}^*, \text{PKE.ct}^*, \pi^*)$.
4. $b' \leftarrow \mathcal{A}_2^{\text{KeyGen}(\text{msk}, \cdot), \mathcal{O}(\text{msk}, \cdot, \cdot)}(\text{mpk}, \text{CT}^*, \text{st})$, where **KeyGen** and $\mathcal{O}$ are oracles defined as below:
 - **KeyGen**(msk, f):
 - (a) $s \leftarrow \{0, 1\}^\lambda$.
 - (b) $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|f(\cdot)|}$ is randomly sampled.
 - (c) $\text{FE.sk}_{G_f} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[f, s, \text{SKE.ct}])$.
 - (d) Output $\text{SK}_f = (\widetilde{\text{crs}}, \text{FE.sk}_{G_f})$.
 - $\mathcal{O}(\text{msk}, \text{CT}, g)$:

- (a) If $CT = CT^*$, return $\perp$.
 - (b) $s \leftarrow \{0, 1\}^\lambda$.
 - (c) $SKE.ct \leftarrow SKE.Enc(SKE.sk, y)$, where $y \leftarrow \{0, 1\}^{|g(\cdot)|}$ is randomly sampled.
 - (d) $FE.sk_{G_g} \leftarrow FE.KeyGen(FE.msk, G[g, s, SKE.ct])$.
 - (e) Parse $CT = (FE.ct, PKE.ct, \pi)$.
 - (f) If $NIZK.Verify(\widetilde{crs}, (FE.ct, PKE.ct), \pi) = 0$, return $\perp$.
 - (g) Return $FE.Dec(SK_{G_g}, FE.ct)$.
5. Let $\{f\}$ be the set of function queries by $\mathcal{A}_2$ to the **KeyGen** oracle. We require that the distributions $(mpk, st, \{f(x_0)\})$ and $(mpk, st, \{f(x_1)\})$ are statistically indistinguishable. If not, the experiment aborts and outputs 0.
6. If $b' = b$, the adversary wins and the game outputs 1. Otherwise, the adversary loses and the game outputs 0.

Hybrid₃⁴: In $\mathcal{O}$, now instead of returning FE decryption of $FE.ct$ (which results in evaluating $G[g, s, SKE.ct, pad]$ on the tuple encrypted under FE), use the PKE decryption of $PKE.ct$ to help compute $G[g, s, SKE.ct, pad]$ directly. This step follows from the simulation soundness of the underlying NIZK.

1. $(x_0, x_1, st) \leftarrow \mathcal{A}_1(1^\lambda)$.
2. Sample $(mpk, msk) \leftarrow Setup(1^\lambda)$ as follows:
 - (a) $(PKE.pk, PKE.sk) \leftarrow PKE.Setup(1^\lambda)$.
 - (b) $SKE.sk \leftarrow SKE.Setup(1^\lambda)$.
 - (c) $(FE.mpk, FE.msk) \leftarrow FE.Setup(1^\lambda)$.
 - (d) $(\widetilde{crs}, \tau) \leftarrow NIZK.Sim_1(1^\lambda)$.
 - (e) $(mpk, msk) := ((FE.mpk, PKE.pk, \widetilde{crs}), (FE.mpk, PKE.pk, \widetilde{crs}, FE.msk, PKE.sk, SKE.sk))$.
3. Compute $CT^* \leftarrow Enc(mpk, x_0)$ as follows:
 - (a) $K \leftarrow PRF.Setup(1^\lambda)$.
 - (b) $FE.ct^* \leftarrow FE.Enc(FE.mpk, (x_0, K, 0, \perp))$.
 - (c) $PKE.ct^* \leftarrow PKE.Enc(PKE.pk, \perp)$.
 - (d) $\pi^* \leftarrow NIZK.Sim_2(\widetilde{crs}, \tau, z)$ where z is the following statement on $FE.ct^*$ and $PKE.ct^*$:
“ $PKE.ct^*$ correctly encrypts $(x, K, \alpha, \widehat{sk}, FE.ct^*)$, where $(x, K, \alpha, \widehat{sk})$ is encrypted in $FE.ct^*$ ”
 - (e) Set $CT^* = (FE.ct^*, PKE.ct^*, \pi^*)$.
4. $b' \leftarrow \mathcal{A}_2^{KeyGen(msk, \cdot), \mathcal{O}(msk, \cdot, \cdot)}(mpk, CT^*, st)$, where **KeyGen** and $\mathcal{O}$ are oracles defined as below:
 - **KeyGen**(msk, f):
 - (a) $s \leftarrow \{0, 1\}^\lambda$.
 - (b) $SKE.ct \leftarrow SKE.Enc(SKE.sk, y)$, where $y \leftarrow \{0, 1\}^{|f(\cdot)|}$ is randomly sampled.
 - (c) $FE.sk_{G_f} \leftarrow FE.KeyGen(FE.msk, G[f, s, SKE.ct])$.

- (d) Output $SK_f = (\widetilde{crs}, FE.sk_{G_f})$.
- $\mathcal{O}(msk, CT, g)$:
 - (a) If $CT = CT^*$, return $\perp$.
 - (b) $s \leftarrow \{0, 1\}^\lambda$.
 - (c) $SKE.ct \leftarrow SKE.Enc(SKE.sk, y)$, where $y \leftarrow \{0, 1\}^{|g(\cdot)|}$ is randomly sampled.
 - (d) ~~$FE.sk_{G_g} \leftarrow FE.KeyGen(FE.msk, G[g, s, SKE.ct])$.~~
 - (e) Parse $CT = (FE.ct, PKE.ct, \pi)$.
 - (f) If $NIZK.Verify(\widetilde{crs}, (FE.ct, PKE.ct), \pi) = 0$, return $\perp$.
 - (g) **Compute $(x, K, \alpha, \widehat{sk}, FE.ct') \leftarrow PKE.Dec(PKE.sk, PKE.ct)$, and return $G[g, s, SKE.ct](x, K, \alpha, \widehat{sk})$.**
- 5. Let $\{f\}$ be the set of function queries by $\mathcal{A}_2$ to the KeyGen oracle. We require that the distributions $(mpk, st, \{f(x_0)\})$ and $(mpk, st, \{f(x_1)\})$ are statistically indistinguishable. If not, the experiment aborts and outputs 0.
- 6. If $b' = b$, the adversary wins and the game outputs 1. Otherwise, the adversary loses and the game outputs 0.

Hybrid₄^A: When answering KeyGen queries, we now compute $r \leftarrow PRF.Eval(K, s)$, and set $SKE.ct$ to encrypt $\hat{y} \leftarrow f(x_0; r)$ instead of a randomly sampled y . This step follows from SKE security.

1. $(x_0, x_1, st) \leftarrow \mathcal{A}_1(1^\lambda)$.
2. Sample $(mpk, msk) \leftarrow Setup(1^\lambda)$ as follows:
 - (a) $(PKE.pk, PKE.sk) \leftarrow PKE.Setup(1^\lambda)$.
 - (b) $SKE.sk \leftarrow SKE.Setup(1^\lambda)$.
 - (c) $(FE.mpk, FE.msk) \leftarrow FE.Setup(1^\lambda)$.
 - (d) $(\widetilde{crs}, \tau) \leftarrow NIZK.Sim_1(1^\lambda)$.
 - (e) $(mpk, msk) := ((FE.mpk, PKE.pk, \widetilde{crs}), (FE.mpk, PKE.pk, \widetilde{crs}, FE.msk, PKE.sk, SKE.sk))$.
3. Compute $CT^* \leftarrow Enc(mpk, x_0)$ as follows:
 - (a) $K \leftarrow PRF.Setup(1^\lambda)$.
 - (b) $FE.ct^* \leftarrow FE.Enc(FE.mpk, (x_0, K, 0, \perp))$.
 - (c) $PKE.ct^* \leftarrow PKE.Enc(PKE.pk, \perp)$.
 - (d) $\pi^* \leftarrow NIZK.Sim_2(\widetilde{crs}, \tau, z)$ where z is the following statement on $FE.ct^*$ and $PKE.ct^*$:
“PKE.ct correctly encrypts $(x, K, \alpha, \widehat{sk}, FE.ct^*)$, where $(x, K, \alpha, \widehat{sk})$ is encrypted in FE.ct*”*
 - (e) Set $CT^* = (FE.ct^*, PKE.ct^*, \pi^*)$.
4. $b' \leftarrow \mathcal{A}_2^{KeyGen(msk, \cdot), \mathcal{O}(msk, \cdot, \cdot)}(mpk, CT^*, st)$, where KeyGen and $\mathcal{O}$ are oracles defined as below:
 - KeyGen(msk, f):
 - (a) $s \leftarrow \{0, 1\}^\lambda$.

- (b) $r \leftarrow \text{PRF.Eval}(K, s)$.
 - (c) $\hat{y} \leftarrow f(x_0; r)$.
 - (d) $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, \hat{y})$.
 - (e) $\text{FE.sk}_{G_f} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[f, s, \text{SKE.ct}])$.
 - (f) Output $\text{SK}_f = (\widetilde{\text{crs}}, \text{FE.sk}_{G_f})$.
 - $\mathcal{O}(\text{msk}, \text{CT}, g)$:
 - (a) If $\text{CT} = \text{CT}^*$, return $\perp$.
 - (b) $s \leftarrow \{0, 1\}^\lambda$.
 - (c) $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|g(\cdot)|}$ is randomly sampled.
 - (d) Parse $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$.
 - (e) If $\text{NIZK.Verify}(\widetilde{\text{crs}}, (\text{FE.ct}, \text{PKE.ct}), \pi) = 0$, return $\perp$.
 - (f) Compute $(x, K, \alpha, \hat{\text{sk}}, \text{FE.ct}') \leftarrow \text{PKE.Dec}(\text{PKE.sk}, \text{PKE.ct})$, and return $G[g, s, \text{SKE.ct}]$
 $(x, K, \alpha, \hat{\text{sk}})$.
5. Let $\{f\}$ be the set of function queries by $\mathcal{A}_2$ to the KeyGen oracle. We require that the distributions $(\text{mpk}, \text{st}, \{f(x_0)\})$ and $(\text{mpk}, \text{st}, \{f(x_1)\})$ are statistically indistinguishable. If not, the experiment aborts and outputs 0.
6. If $b' = b$, the adversary wins and the game outputs 1. Otherwise, the adversary loses and the game outputs 0.

Hybrid₅⁴: Now we invoke FE selective security to encrypt $(\perp, \perp, 1, \text{SKE.sk})$ instead in FE.ct^* .

1. $(x_0, x_1, \text{st}) \leftarrow \mathcal{A}_1(1^\lambda)$.
2. Sample $(\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda)$ as follows:
 - (a) $(\text{PKE.pk}, \text{PKE.sk}) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 - (b) $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 - (c) $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (d) $(\widetilde{\text{crs}}, \tau) \leftarrow \text{NIZK.Sim}_1(1^\lambda)$.
 - (e) $(\text{mpk}, \text{msk}) := ((\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}), (\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}, \text{FE.msk}, \text{PKE.sk}, \text{SKE.sk}))$.
3. Compute $\text{CT}^* \leftarrow \text{Enc}(\text{mpk}, x_0)$ as follows:
 - (a) $K \leftarrow \text{PRF.Setup}(1^\lambda)$.
 - (b) $\text{FE.ct}^* \leftarrow \text{FE.Enc}(\text{FE.mpk}, (\perp, \perp, 1, \text{SKE.sk}))$.
 - (c) $\text{PKE.ct}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, \perp)$.
 - (d) $\pi^* \leftarrow \text{NIZK.Sim}_2(\widetilde{\text{crs}}, \tau, z)$ where z is the following statement on FE.ct^* and PKE.ct^* :
“PKE.ct correctly encrypts $(x, K, \alpha, \hat{\text{sk}}, \text{FE.ct}^*)$, where $(x, K, \alpha, \hat{\text{sk}})$ is encrypted in FE.ct*”*
 - (e) Set $\text{CT}^* = (\text{FE.ct}^*, \text{PKE.ct}^*, \pi^*)$.
4. $b' \leftarrow \mathcal{A}_2^{\text{KeyGen}(\text{msk}, \cdot), \mathcal{O}(\text{msk}, \cdot, \cdot)}(\text{mpk}, \text{CT}^*, \text{st})$, where KeyGen and $\mathcal{O}$ are oracles defined as below:
 - $\text{KeyGen}(\text{msk}, f)$:

- (a) $s \leftarrow \{0, 1\}^\lambda$.
 - (b) $r \leftarrow \text{PRF.Eval}(K, s)$.
 - (c) $\hat{y} \leftarrow f(x_0; r)$.
 - (d) $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, \hat{y})$.
 - (e) $\text{FE.sk}_{G_f} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[f, s, \text{SKE.ct}])$.
 - (f) Output $\text{SK}_f = (\widetilde{\text{crs}}, \text{FE.sk}_{G_f})$.
 - $\mathcal{O}(\text{msk}, \text{CT}, g)$:
 - (a) If $\text{CT} = \text{CT}^*$, return $\perp$.
 - (b) $s \leftarrow \{0, 1\}^\lambda$.
 - (c) $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|g(\cdot)|}$ is randomly sampled.
 - (d) Parse $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$.
 - (e) If $\text{NIZK.Verify}(\widetilde{\text{crs}}, (\text{FE.ct}, \text{PKE.ct}), \pi) = 0$, return $\perp$.
 - (f) Compute $(x, K, \alpha, \widehat{\text{sk}}, \text{FE.ct}') \leftarrow \text{PKE.Dec}(\text{PKE.sk}, \text{PKE.ct})$, and return $G[g, s, \text{SKE.ct}](x, K, \alpha, \widehat{\text{sk}})$.
5. Let $\{f\}$ be the set of function queries by $\mathcal{A}_2$ to the **KeyGen** oracle. We require that the distributions $(\text{mpk}, \text{st}, \{f(x_0)\})$ and $(\text{mpk}, \text{st}, \{f(x_1)\})$ are statistically indistinguishable. If not, the experiment aborts and outputs 0.
6. If $b' = b$, the adversary wins and the game outputs 1. Otherwise, the adversary loses and the game outputs 0.

Hybrid₆^A: When responding to **KeyGen** queries, now sample a uniformly random $\hat{r}$ instead of computing $r \leftarrow \text{PRF.Eval}(K, s)$. This follows from PRF security, since the PRF key K is now completely hidden and unused.

1. $(x_0, x_1, \text{st}) \leftarrow \mathcal{A}_1(1^\lambda)$.
2. Sample $(\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda)$ as follows:
 - (a) $(\text{PKE.pk}, \text{PKE.sk}) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 - (b) $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 - (c) $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (d) $(\widetilde{\text{crs}}, \tau) \leftarrow \text{NIZK.Sim}_1(1^\lambda)$.
 - (e) $(\text{mpk}, \text{msk}) := ((\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}), (\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}, \text{FE.msk}, \text{PKE.sk}, \text{SKE.sk}))$.
3. Compute $\text{CT}^* \leftarrow \text{Enc}(\text{mpk}, x_0)$ as follows:
 - (a) $K \leftarrow \text{PRF.Setup}(1^\lambda)$.
 - (b) $\text{FE.ct}^* \leftarrow \text{FE.Enc}(\text{FE.mpk}, (\perp, \perp, 1, \text{SKE.sk}))$.
 - (c) $\text{PKE.ct}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, \perp)$.
 - (d) $\pi^* \leftarrow \text{NIZK.Sim}_2(\widetilde{\text{crs}}, \tau, z)$ where z is the following statement on FE.ct^* and PKE.ct^* :
“ PKE.ct^* correctly encrypts $(x, K, \alpha, \widehat{\text{sk}}, \text{FE.ct}^*)$, where $(x, K, \alpha, \widehat{\text{sk}})$ is encrypted in FE.ct^* ”
 - (e) Set $\text{CT}^* = (\text{FE.ct}^*, \text{PKE.ct}^*, \pi^*)$.

4. $b' \leftarrow \mathcal{A}_2^{\text{KeyGen}(\text{msk}, \cdot), \mathcal{O}(\text{msk}, \cdot)}(\text{mpk}, \text{CT}^*, \text{st})$, where KeyGen and $\mathcal{O}$ are oracles defined as below:
 - $\text{KeyGen}(\text{msk}, f)$:
 - (a) $s \leftarrow \{0, 1\}^\lambda$.
 - (b) $\hat{r} \leftarrow \{0, 1\}^{\ell_{\text{PRF}}}$.
 - (c) $\hat{y} \leftarrow f(x_0; \hat{r})$.
 - (d) $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, \hat{y})$.
 - (e) $\text{FE.sk}_{G_f} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[f, s, \text{SKE.ct}])$.
 - (f) Output $\text{SK}_f = (\widetilde{\text{crs}}, \text{FE.sk}_{G_f})$.
 - $\mathcal{O}(\text{msk}, \text{CT}, g)$:
 - (a) If $\text{CT} = \text{CT}^*$, return $\perp$.
 - (b) $s \leftarrow \{0, 1\}^\lambda$.
 - (c) $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|g(\cdot)|}$ is randomly sampled.
 - (d) Parse $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$.
 - (e) If $\text{NIZK.Verify}(\widetilde{\text{crs}}, (\text{FE.ct}, \text{PKE.ct}), \pi) = 0$, return $\perp$.
 - (f) Compute $(x, K, \alpha, \hat{\text{sk}}, \text{FE.ct}') \leftarrow \text{PKE.Dec}(\text{PKE.sk}, \text{PKE.ct})$, and return $G[g, s, \text{SKE.ct}]$
 $(x, K, \alpha, \hat{\text{sk}})$.
5. Let $\{f\}$ be the set of function queries by $\mathcal{A}_2$ to the KeyGen oracle. We require that the distributions $(\text{mpk}, \text{st}, \{f(x_0)\})$ and $(\text{mpk}, \text{st}, \{f(x_1)\})$ are statistically indistinguishable. If not, the experiment aborts and outputs 0.
6. If $b' = b$, the adversary wins and the game outputs 1. Otherwise, the adversary loses and the game outputs 0.

Hybrid₇^A: Now we change the challenge bit b from 0 to 1. This step follows from the fact that for all functions f queried by the adversary to the KeyGen oracle, $(\text{mpk}, \text{st}, \{f(x_0)\})$ and $(\text{mpk}, \text{st}, \{f(x_1)\})$ are statistically indistinguishable.

1. $(x_0, x_1, \text{st}) \leftarrow \mathcal{A}_1(1^\lambda)$.
2. Sample $(\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda)$ as follows:
 - (a) $(\text{PKE.pk}, \text{PKE.sk}) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 - (b) $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 - (c) $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
 - (d) $(\widetilde{\text{crs}}, \tau) \leftarrow \text{NIZK.Sim}_1(1^\lambda)$.
 - (e) $(\text{mpk}, \text{msk}) := ((\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}), (\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}, \text{FE.msk}, \text{PKE.sk}, \text{SKE.sk}))$.
3. Compute $\text{CT}^* \leftarrow \text{Enc}(\text{mpk}, x_1)$ as follows:
 - (a) $K \leftarrow \text{PRF.Setup}(1^\lambda)$.
 - (b) $\text{FE.ct}^* \leftarrow \text{FE.Enc}(\text{FE.mpk}, (\perp, \perp, 1, \text{SKE.sk}))$.
 - (c) $\text{PKE.ct}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, \perp)$.

- (d) $\pi^* \leftarrow \text{NIZK.Sim}_2(\widetilde{\text{crs}}, \tau, z)$ where z is the following statement on FE.ct^* and PKE.ct^* :
“ PKE.ct^* correctly encrypts $(x, K, \alpha, \widehat{\text{sk}}, \text{FE.ct}^*)$, where $(x, K, \alpha, \widehat{\text{sk}})$ is encrypted in FE.ct^* ”
- (e) Set $\text{CT}^* = (\text{FE.ct}^*, \text{PKE.ct}^*, \pi^*)$.
4. $b' \leftarrow \mathcal{A}_2^{\text{KeyGen}(\text{msk}, \cdot), \mathcal{O}(\text{msk}, \cdot, \cdot)}(\text{mpk}, \text{CT}^*, \text{st})$, where KeyGen and $\mathcal{O}$ are oracles defined as below:
- $\text{KeyGen}(\text{msk}, f)$:
 - (a) $s \leftarrow \{0, 1\}^\lambda$.
 - (b) $\hat{r} \leftarrow \{0, 1\}^{\ell_{\text{PRF}}}$.
 - (c) $\hat{y} \leftarrow f(\textcolor{red}{x}_1; \hat{r})$.
 - (d) $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, \hat{y})$.
 - (e) $\text{FE.sk}_{G_f} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[f, s, \text{SKE.ct}])$.
 - (f) Output $\text{SK}_f = (\widetilde{\text{crs}}, \text{FE.sk}_{G_f})$.
 - $\mathcal{O}(\text{msk}, \text{CT}, g)$:
 - (a) If $\text{CT} = \text{CT}^*$, return $\perp$.
 - (b) $s \leftarrow \{0, 1\}^\lambda$.
 - (c) $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|g(\cdot)|}$ is randomly sampled.
 - (d) Parse $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$.
 - (e) If $\text{NIZK.Verify}(\widetilde{\text{crs}}, (\text{FE.ct}, \text{PKE.ct}), \pi) = 0$, return $\perp$.
 - (f) Compute $(x, K, \alpha, \widehat{\text{sk}}, \text{FE.ct}') \leftarrow \text{PKE.Dec}(\text{PKE.sk}, \text{PKE.ct})$, and return $G[g, s, \text{SKE.ct}](x, K, \alpha, \widehat{\text{sk}})$.
5. Let $\{f\}$ be the set of function queries by $\mathcal{A}_2$ to the KeyGen oracle. We require that the distributions $(\text{mpk}, \text{st}, \{f(x_0)\})$ and $(\text{mpk}, \text{st}, \{f(x_1)\})$ are statistically indistinguishable. If not, the experiment aborts and outputs 0.
6. If $b' = b$, the adversary wins and the game outputs 1. Otherwise, the adversary loses and the game outputs 0.

Hybrid_8^A , Hybrid_9^A , Hybrid_{10}^A , Hybrid_{11}^A , Hybrid_{12}^A , and Hybrid_{13}^A will revert the changes introduced in Hybrid_6^A , Hybrid_5^A , Hybrid_4^A , Hybrid_3^A , Hybrid_2^A and Hybrid_1^A respectively, with Hybrid_{13}^A being the same as $\text{Expt}_{\mathcal{A}}^{\text{rFE}}$ where the challenge bit is fixed to be 1.

Proof of Hybrid Arguments

Lemma 5.3. *If NIZK has computational zero-knowledge, then no PPT adversary $\mathcal{A}$ can distinguish between Hybrid_0^A and Hybrid_1^A with non-negligible probability.*

Proof. We prove this by reduction to the zero-knowledgeness of NIZK. Specifically, we show how an adversary $\mathcal{A}$ that distinguishes between Hybrid_0^A and Hybrid_1^A can be used to construct an adversary $\mathcal{B}$ that breaks zero-knowledgeness of NIZK.

$\mathcal{B}^{\mathcal{O}_{\text{NIZK}}(\cdot, \cdot)}(\text{crs}) :$

- $\mathcal{B}$ runs $\mathcal{A}_1(1^\lambda)$ to obtain (x_0, x_1, st) .
- Sample (mpk, msk) as follows:

- $(\text{PKE.pk}, \text{PKE.sk}) \leftarrow \text{PKE.Setup}(1^\lambda)$.
- $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
- $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
- $(\text{mpk}, \text{msk}) := ((\text{FE.mpk}, \text{PKE.pk}, \text{crs}), (\text{FE.mpk}, \text{PKE.pk}, \text{crs}, \text{FE.msk}, \text{PKE.sk}, \text{SKE.sk}))$.
- Compute CT^* as follows:
 - $K \leftarrow \text{PRF.Setup}(1^\lambda)$.
 - $\text{FE.ct}^* \leftarrow \text{FE.Enc}(\text{FE.mpk}, (x_0, K, 0, \perp))$.
 - $\text{PKE.ct}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, (x_0, K, 0, \perp, \text{FE.ct}^*))$.
 - Let z be the following statement on FE.ct^* and PKE.ct^* :

“ PKE.ct^ correctly encrypts $(x, K, \alpha, \hat{\text{sk}}, \text{FE.ct}^*)$, where $(x, K, \alpha, \hat{\text{sk}})$ is encrypted in FE.ct^* ”*

and the witness w be the tuple $(x_0, K, 0, \perp)$. Query the oracle $\mathcal{O}_{\text{NIZK}}$ with (z, w) and receive π^* .
 - Set $\text{CT}^* = (\text{FE.ct}^*, \text{PKE.ct}^*, \pi^*)$.
- Run $\mathcal{A}_2^{\text{KeyGen}(\text{msk}, \cdot), \mathcal{O}(\text{msk}, \cdot, \cdot)}(\text{mpk}, \text{CT}^*, \text{st})$ while simulating the following two oracles:
 - $\text{KeyGen}(\text{msk}, f)$:
 - * $s \leftarrow \{0, 1\}^\lambda$.
 - * $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|f(\cdot)|}$ is randomly sampled.
 - * $\text{FE.sk}_{G_f} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[f, s, \text{SKE.ct}])$.
 - * Output $\text{SK}_f = (\text{crs}, \text{FE.sk}_{G_f})$.
 - $\mathcal{O}(\text{msk}, \text{CT}, g)$:
 - * If $\text{CT} = \text{CT}^*$, return $\perp$.
 - * $s \leftarrow \{0, 1\}^\lambda$.
 - * $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|g(\cdot)|}$ is randomly sampled.
 - * $\text{FE.sk}_{G_g} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[g, s, \text{SKE.ct}])$.
 - * Parse $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$.
 - * If $\text{NIZK.Verify}(\text{crs}, (\text{FE.ct}, \text{PKE.ct}), \pi) = 0$, return $\perp$.
 - * Return $\text{FE.Dec}(\text{FE.sk}_{G_g}, \text{FE.ct})$.
- If $\mathcal{A}$ output it's in $\mathbf{Hybrid}_0^{\mathcal{A}}$, output 0. Otherwise, output 1.

Note that if $\mathcal{A}$ successfully distinguishes $\mathbf{Hybrid}_0^{\mathcal{A}}$ and $\mathbf{Hybrid}_1^{\mathcal{A}}$, then $\mathcal{B}$ also successfully distinguishes whether it's interacting with the real NIZK or the simulators NIZK.Sim_1 and NIZK.Sim_2 . $\square$

Lemma 5.4. *If PKE has IND-CPA security, then no PPT adversary $\mathcal{A}$ can distinguish between $\mathbf{Hybrid}_1^{\mathcal{A}}$ and $\mathbf{Hybrid}_2^{\mathcal{A}}$ with non-negligible probability.*

Proof. We show this by a reduction to IND-CPA security of PKE. Specifically, we show that there exists PPT $\mathcal{A}$ that can distinguish between $\mathbf{Hybrid}_1^{\mathcal{A}}$ and $\mathbf{Hybrid}_2^{\mathcal{A}}$, then we can construct $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$ using $\mathcal{A}$ as a subroutine that breaks IND-CPA security of PKE as follows:

$\mathcal{B}_1(\text{PKE.pk}) :$

- $\mathcal{B}_1$ runs $\mathcal{A}_1(1^\lambda)$ to obtain (x_0, x_1, st) .
- Sample mpk and partial $\overline{\text{msk}}$ as follows:
 - $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
 - $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 - $(\widetilde{\text{crs}}, \tau) \leftarrow \text{NIZK.Sim}_1(1^\lambda)$.
 - Set $\text{mpk} = (\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}})$ and $\overline{\text{msk}} = (\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}, \text{FE.msk}, \text{SKE.sk})$.
Notice that $\overline{\text{msk}}$ is missing PKE.sk compared to a normal msk .
- $K \leftarrow \text{PRF.Setup}(1^\lambda)$.
- $\text{FE.ct}^* \leftarrow \text{FE.Enc}(\text{FE.mpk}, (x_0, K, 0, \perp))$.
- Output $((x_0, K, 0, \perp, \text{FE.ct}^*), \perp, \text{st}' = (\text{st}, \text{mpk}, \overline{\text{msk}}, K, \text{FE.ct}^*, \tau))$.

$\mathcal{B}_2(\text{PKE.ct}^*, \text{st}') :$

- Finish computing CT^* as follows:
 - $\widetilde{\pi}^* \leftarrow \text{NIZK.Sim}_2(\widetilde{\text{crs}}, \tau, z)$ where z is the following statement on FE.ct^* and PKE.ct^* :
“ PKE.ct^* correctly encrypts $(x, K, \alpha, \widehat{\text{sk}}, \text{FE.ct}^*)$, where $(x, K, \alpha, \widehat{\text{sk}})$ is encrypted in FE.ct^* ”
 - Set $\text{CT}^* = (\text{FE.ct}^*, \text{PKE.ct}^*, \pi^*)$.
- Run $\mathcal{A}_2^{\text{KeyGen}(\overline{\text{msk}}, \cdot), \mathcal{O}(\overline{\text{msk}}, \cdot, \cdot)}(\text{mpk}, \text{CT}^*, \text{st})$ while simulating the following two oracles:
 - $\text{KeyGen}(\overline{\text{msk}}, f)$:
 - * $s \leftarrow \{0, 1\}^\lambda$.
 - * $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|f(\cdot)|}$ is randomly sampled.
 - * $\text{FE.sk}_{G_f} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[f, s, \text{SKE.ct}])$.
 - * Output $\text{SK}_f = (\widetilde{\text{crs}}, \text{FE.sk}_{G_f})$.
 - $\mathcal{O}(\overline{\text{msk}}, \text{CT}, g)$:
 - * If $\text{CT} = \text{CT}^*$, return $\perp$.
 - * $s \leftarrow \{0, 1\}^\lambda$.
 - * $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|g(\cdot)|}$ is randomly sampled.
 - * $\text{FE.sk}_{G_g} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[g, s, \text{SKE.ct}])$.
 - * Parse $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$.
 - * If $\text{NIZK.Verify}(\widetilde{\text{crs}}, (\text{FE.ct}, \text{PKE.ct}), \pi) = 0$, return $\perp$.
 - * Return $\text{FE.Dec}(\text{FE.sk}_{G_g}, \text{FE.ct})$.
- If $\mathcal{A}$ output it's in $\mathbf{Hybrid}_1^A$, output 0. Otherwise, output 1.

Notice that if $\mathcal{A}$ outputs that it is in $\mathbf{Hybrid}_1^A$, that means PKE.ct^* encrypts $(x, K, 0, \perp, \text{FE.ct}^*)$, and hence $\mathcal{B}$ outputting 0 is correct. Otherwise, if $\mathcal{A}$ outputs it is in $\mathbf{Hybrid}_2^A$ where PKE.ct^* encrypts $\perp$, $\mathcal{B}$ outputting 1 is also correct. $\square$

Lemma 5.5. *If NIZK has simulation soundness, then no PPT adversary $\mathcal{A}$ can distinguish between $\text{Hybrid}_2^{\mathcal{A}}$ and $\text{Hybrid}_3^{\mathcal{A}}$ with non-negligible probability.*

Proof. We prove this by reduction to the simulation soundness of NIZK. Let $\mathcal{A}$ be an adversary that distinguishes between $\text{Hybrid}_2^{\mathcal{A}}$ and $\text{Hybrid}_3^{\mathcal{A}}$, we construct an adversary $\mathcal{B}$ for the NIZK simulation soundness game as follows.

$\mathcal{B}^{\text{NIZK.Sim}_2(\widetilde{\text{crs}}, \tau, \cdot)}(\widetilde{\text{crs}})$:

- Runs $\mathcal{A}_1(1^\lambda)$ to obtain (x_0, x_1, st) .
- Sample $(\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda)$ as follows:
 1. $(\text{PKE.pk}, \text{PKE.sk}) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 2. $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 3. $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
 4. $(\text{mpk}, \text{msk}) := ((\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}), (\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}, \text{FE.msk}, \text{PKE.sk}, \text{SKE.sk}))$.
- Compute $\text{CT}^* \leftarrow \text{Enc}(\text{mpk}, x_0)$ as follows:
 1. $K \leftarrow \text{PRF.Setup}(1^\lambda)$.
 2. $\text{FE.ct}^* \leftarrow \text{FE.Enc}(\text{FE.mpk}, (x_0, K, 0, \perp))$.
 3. $\text{PKE.ct}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, \perp)$.
 4. Query the oracle $\text{NIZK.Sim}_2(\widetilde{\text{crs}}, \tau, \cdot)$ on the following statement z on FE.ct^* and PKE.ct^* to obtain $\widetilde{\pi}^*$:

“ PKE.ct^* correctly encrypts $(x, K, \alpha, \widehat{\text{sk}}, \text{FE.ct}^*)$, where $(x, K, \alpha, \widehat{\text{sk}})$ is encrypted in FE.ct^* ”
 5. Set $\text{CT}^* = (\text{FE.ct}^*, \text{PKE.ct}^*, \pi^*)$.
- Sample a uniform bit $b \leftarrow \{0, 1\}$.
- Run $b' \leftarrow \mathcal{A}_2^{\text{KeyGen}(\text{msk}, \cdot), \mathcal{O}(\text{msk}, \cdot, \cdot)}(\text{mpk}, \text{CT}^*, \text{st})$ while simulating the following two oracles:
 - $\text{KeyGen}(\text{msk}, f)$:
 1. $s \leftarrow \{0, 1\}^\lambda$.
 2. $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|f(\cdot)|}$ is randomly sampled.
 3. $\text{FE.sk}_{G_f} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[f, s, \text{SKE.ct}])$.
 4. Output $\text{SK}_f = (\widetilde{\text{crs}}, \text{FE.sk}_{G_f})$.
 - $\mathcal{O}(\text{msk}, \text{CT}, g)$:
 1. If $\text{CT} = \text{CT}^*$, return $\perp$.
 2. $s \leftarrow \{0, 1\}^\lambda$.
 3. $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|g(\cdot)|}$ is randomly sampled.
 4. $\text{FE.sk}_{G_g} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[g, s, \text{SKE.ct}])$.
 5. Parse $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$.
 6. If $\text{NIZK.Verify}(\widetilde{\text{crs}}, (\text{FE.ct}, \text{PKE.ct}), \pi) = 0$, return $\perp$.

7. Compute $(x, K, \alpha, \widehat{\text{sk}}) \leftarrow \text{FE.Dec}(\text{FE.sk}_{\mathcal{I}}, \text{FE.ct})$ by sampling an FE functional key for the identity function $\mathcal{I}$ as $\text{FE.sk}_{\mathcal{I}} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, \mathcal{I})$.
8. Compute $(x', K', \alpha', \widehat{\text{sk}}', \text{FE.ct}') \leftarrow \text{PKE.Dec}(\text{PKE.sk}, \text{PKE.ct})$.
9. If $(x, K, \alpha, \widehat{\text{sk}}) \neq (x', K', \alpha', \widehat{\text{sk}}')$, $\mathcal{B}$ outputs $((\text{FE.ct}, \text{PKE.ct}), \pi)$ to the NIZK simulation soundness game (and continues finishing simulating the oracles for $\mathcal{A}_2$).
10. If $b = 0$, return $\text{FE.Dec}(\text{SK}_{G_g}, \text{FE.ct})$. If $b = 1$, compute and return $G[g, s, \text{SKE.ct}](x', K', \alpha', \widehat{\text{sk}}')$.

Kindly note that $b = 0$ corresponds to the case where $\mathcal{A}$ is in **Hybrid**₂^A and $b = 1$ corresponds to **Hybrid**₃^A.

Now, notice that if $\mathcal{B}$ ever outputs $((\text{FE.ct}, \text{PKE.ct}), \pi)$ when answering a $\mathcal{O}$ query (CT, g) , it *will* win the NIZK simulation soundness game. Recall that winning the simulation soundness game requires: (1) $((\text{FE.ct}, \text{PKE.ct}), \pi)$ is not returned by a prior query to NIZK.Sim_2 ; (2) $(\text{FE.ct}, \text{PKE.ct})$ is not in the language; (3) $\text{Verify}(\widetilde{\text{crs}}, (\text{FE.ct}, \text{PKE.ct}), \pi)$ outputs 1. (1) is true because $\text{CT} \neq \text{CT}^*$, otherwise $\mathcal{O}$ will return $\perp$ earlier on in Step 1; (2) is true since $\mathcal{B}$ will only output if FE.ct and PKE.ct encrypt different $(x, K, \alpha, \widehat{\text{sk}})$ tuples and hence not in the language; and (3) is true because otherwise $\mathcal{O}$ will return $\perp$ in Step 6. So indeed if $\mathcal{B}$ ever outputs $((\text{FE.ct}, \text{PKE.ct}), \pi)$, $\mathcal{B}$ will win the simulation soundness game.

So if $\mathcal{B}$ does not win, that means it never outputs anything. This implies that for all the $\mathcal{O}$ queries made by $\mathcal{A}_2$, the response is either $\perp$, or FE.ct and PKE.ct encrypt the exact same $(x, K, \alpha, \widehat{\text{sk}})$ tuple. Observe that if FE.ct and PKE.ct encrypt the same $(x, K, \alpha, \widehat{\text{sk}})$ tuple, then for $b = 0$, $\mathcal{O}$ would respond with $\text{FE.Dec}(\text{SK}_{G_g}, \text{FE.ct}) = G[g, s, \text{SKE.ct}](x, K, \alpha, \widehat{\text{sk}})$, which is exactly the same as what the oracle response would be for $b = 1$. This means, if $\mathcal{B}$ does not win, then for all $\mathcal{O}$ queries, the responses will be exactly the same *regardless of the challenge bit* b , and therefore $\mathcal{A}$ cannot possibly guess b correctly with non-negligible advantage. By contrapositive, if $\mathcal{A}$ can successfully distinguish **Hybrid**₂^A and **Hybrid**₃^A by guessing the bit b correctly, $\mathcal{B}$ would also win the simulation soundness game for NIZK.

Lastly, kindly observe that step 4d is no longer needed once we switch to **Hybrid**₃^A. $\square$

Lemma 5.6. *If SKE is a IND-CPA secure SKE scheme, then no PPT adversary $\mathcal{A}$ can distinguish between **Hybrid**₃^A and **Hybrid**₄^A with non-negligible probability.*

Proof. Notice that the only difference between **Hybrid**₃^A and **Hybrid**₄^A is that in **Hybrid**₃^A SKE.ct encrypts a random y , while in **Hybrid**₄^A, SKE.ct encrypts $\hat{y}$ that are function evaluation on x_0 and PRF outputs. This easily reduces to the left-or-right CPA security of the underlying SKE.

For completeness, we show a reduction by building an adversary $\mathcal{B}$ that breaks the CPA security of SKE by using $\mathcal{A}$ as a subroutine.

$\mathcal{B}(1^\lambda)$:

- $(x_0, x_1, \text{st}) \leftarrow \mathcal{A}_1(1^\lambda)$.
- Sample mpk and partial $\overline{\text{msk}}$ as follows:
 1. $(\text{PKE.pk}, \text{PKE.sk}) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 2. $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
 3. $(\widetilde{\text{crs}}, \tau) \leftarrow \text{NIZK.Sim}_1(1^\lambda)$.

4. $(\overline{\text{mpk}}, \overline{\text{msk}}) := ((\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}), (\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}, \text{FE.msk}, \text{PKE.sk}))$, where $\overline{\text{msk}}$ is the normal msk but missing the SKE.sk part.
- Compute $\text{CT}^* \leftarrow \text{Enc}(\text{mpk}, x_0)$ as follows:
 1. $K \leftarrow \text{PRF.Setup}(1^\lambda)$.
 2. $\text{FE.ct}^* \leftarrow \text{FE.Enc}(\text{FE.mpk}, (x_0, K, 0, \perp))$.
 3. $\text{PKE.ct}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, \perp)$.
 4. $\pi^* \leftarrow \text{NIZK.Sim}_2(\widetilde{\text{crs}}, \tau, z)$ where z is the following statement on FE.ct^* and PKE.ct^* :
“ PKE.ct^* correctly encrypts $(x, K, \alpha, \widehat{\text{sk}}, \text{FE.ct}^*)$, where $(x, K, \alpha, \widehat{\text{sk}})$ is encrypted in FE.ct^* ”
 5. Set $\text{CT}^* = (\text{FE.ct}^*, \text{PKE.ct}^*, \pi^*)$.
 - Run $\mathcal{A}_2^{\text{KeyGen}(\overline{\text{msk}}, \cdot), \mathcal{O}(\overline{\text{msk}}, \cdot, \cdot)}(\text{mpk}, \text{CT}^*, \text{st})$, while simulating the following two oracles:
 - $\text{KeyGen}(\overline{\text{msk}}, f)$:
 1. $s \leftarrow \{0, 1\}^\lambda$.
 2. $r \leftarrow \text{PRF.Eval}(K, s)$.
 3. Sample/compute $y_0 \leftarrow \{0, 1\}^{|f(\cdot)|}$ and $y_1 \leftarrow f(x_0; r)$.
 4. Submit challenge messages (y_0, y_1) and receive a challenge ciphertext SKE.ct .
 5. $\text{FE.sk}_{G_f} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[f, s, \text{SKE.ct}])$.
 6. Output $\text{SK}_f = (\widetilde{\text{crs}}, \text{FE.sk}_{G_f})$.
 - $\mathcal{O}(\overline{\text{msk}}, \text{CT}, g)$:
 1. If $\text{CT} = \text{CT}^*$, return $\perp$.
 2. $s \leftarrow \{0, 1\}^\lambda$.
 3. Sample $y_0, y_1 \leftarrow \{0, 1\}^{|g(\cdot)|}$.
 4. Submit challenge messages (y_0, y_1) and receive a challenge ciphertext SKE.ct .
 5. Parse $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$.
 6. If $\text{NIZK.Verify}(\widetilde{\text{crs}}, (\text{FE.ct}, \text{PKE.ct}), \pi) = 0$, return $\perp$.
 7. Compute $(x, K, \alpha, \widehat{\text{sk}}, \text{FE.ct}') \leftarrow \text{PKE.Dec}(\text{PKE.sk}, \text{PKE.ct})$, and return $G[g, s, \text{SKE.ct}](x, K, \alpha, \widehat{\text{sk}})$.
 - If $\mathcal{A}$ outputs it is in $\mathbf{Hybrid}_3^{\mathcal{A}}$, output 0. Otherwise, output 1.

It is easy to see that if $\mathcal{A}$ wins, $\mathcal{B}$ also wins. □

Lemma 5.7. *If FE is a selectively secure FE scheme, then no PPT adversary $\mathcal{A}$ can distinguish between $\mathbf{Hybrid}_4^{\mathcal{A}}$ and $\mathbf{Hybrid}_5^{\mathcal{A}}$ with non-negligible probability.*

Proof. We show a reduction to selective FE security. Let $\mathcal{A}$ be an adversary that can distinguish between $\mathbf{Hybrid}_4^{\mathcal{A}}$ and $\mathbf{Hybrid}_5^{\mathcal{A}}$, we construct an adversary $\mathcal{B}$ that can break selective security of the underlying FE scheme.

$\mathcal{B}(1^\lambda)$:

- $(x_0, x_1, \text{st}) \leftarrow \mathcal{A}_1(1^\lambda)$.

- Sample partial $\overline{\text{mpk}}$ and partial $\overline{\text{msk}}$ as follows:
 1. $(\text{PKE.pk}, \text{PKE.sk}) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 2. $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 3. $(\widetilde{\text{crs}}, \tau) \leftarrow \text{NIZK.Sim}_1(1^\lambda)$.
 4. Set $(\overline{\text{mpk}}, \overline{\text{msk}}) = ((\text{PKE.pk}, \widetilde{\text{crs}}), (\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}, \text{PKE.sk}, \text{SKE.sk}))$, where $\overline{\text{mpk}}$ and $\overline{\text{msk}}$ are the normal mpk and msk but missing the FE.mpk and FE.msk part.
- Compute $\text{CT}^* \leftarrow \text{Enc}(\overline{\text{mpk}}, x_0)$ as follows:
 1. $K \leftarrow \text{PRF.Setup}(1^\lambda)$.
 2. Submit challenge messages $m_0 = (x_0, K, 0, \perp)$ and $m_1 = (\perp, \perp, 1, \text{SKE.sk})$, and receive a challenge ciphertext FE.ct^* together with FE.mpk .
 3. $\text{PKE.ct}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, \perp)$.
 4. $\pi^* \leftarrow \text{NIZK.Sim}_2(\widetilde{\text{crs}}, \tau, z)$ where z is the following statement on FE.ct^* and PKE.ct^* :
 “ PKE.ct^* correctly encrypts $(x, K, \alpha, \widehat{\text{sk}}, \text{FE.ct}^*)$, where $(x, K, \alpha, \widehat{\text{sk}})$ is encrypted in FE.ct^* ”
 5. Set $\text{CT}^* = (\text{FE.ct}^*, \text{PKE.ct}^*, \pi^*)$.
- Run $\mathcal{A}_2^{\text{KeyGen}(\overline{\text{msk}}, \cdot), \mathcal{O}(\overline{\text{msk}}, \cdot, \cdot)}(\text{mpk}, \text{CT}^*, \text{st})$, while simulating the following two oracles:
 - $\text{KeyGen}(\overline{\text{msk}}, f)$:
 1. $s \leftarrow \{0, 1\}^\lambda$.
 2. $r \leftarrow \text{PRF.Eval}(K, s)$.
 3. $\hat{y} \leftarrow f(x_0; r)$.
 4. $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, \hat{y})$.
 5. Submit a function query for $G[f, s, \text{SKE.ct}]$ and receive FE.sk_{G_f} .
 6. Output $\text{SK}_f = (\widetilde{\text{crs}}, \text{FE.sk}_{G_f})$.
 - $\mathcal{O}(\overline{\text{msk}}, \text{CT}, g)$:
 1. If $\text{CT} = \text{CT}^*$, return $\perp$.
 2. $s \leftarrow \{0, 1\}^\lambda$.
 3. $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|g(\cdot)|}$ is randomly sampled.
 4. Parse $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$.
 5. If $\text{NIZK.Verify}(\widetilde{\text{crs}}, (\text{FE.ct}, \text{PKE.ct}), \pi) = 0$, return $\perp$.
 6. Compute $(x, K, \alpha, \widehat{\text{sk}}, \text{FE.ct}') \leftarrow \text{PKE.Dec}(\text{PKE.sk}, \text{PKE.ct})$, and return $G[g, s, \text{SKE.ct}](x, K, \alpha, \widehat{\text{sk}})$.
- If $\mathcal{A}$ outputs it is in **Hybrid** $_4^{\mathcal{A}}$, output 0. Otherwise, output 1.

First, notice that the only function queries $\mathcal{B}$ needs to submit are for $G[f, s, \text{SKE.ct}]$, where we have

$$G[f, s, \text{SKE.ct}](x_0, K, 0, \perp) = f(x_0; r) = G[f, s, \text{SKE.ct}](\perp, \perp, 1, \text{SKE.sk}),$$

so all the function queries are valid.

Then, it is easy to verify that if $\mathcal{A}$ wins, $\mathcal{B}$ also wins. $\square$

Lemma 5.8. *If PRF is a secure PRF, then no PPT adversary $\mathcal{A}$ can distinguish between $\mathbf{Hybrid}_5^A$ and $\mathbf{Hybrid}_6^A$ with non-negligible probability.*

Proof. We construct the following adversary $\mathcal{B}$ for the $\text{Expt}_{\mathcal{B}}^{\text{PRF}}$ game by using $\mathcal{A}$ as a subroutine.

$\mathcal{B}(1^\lambda)$:

- $(x_0, x_1, \text{st}) \leftarrow \mathcal{A}_1(1^\lambda)$.
- Sample $(\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda)$ as follows:
 1. $(\text{PKE.pk}, \text{PKE.sk}) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 2. $\text{SKE.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.
 3. $(\text{FE.mpk}, \text{FE.msk}) \leftarrow \text{FE.Setup}(1^\lambda)$.
 4. $(\widetilde{\text{crs}}, \tau) \leftarrow \text{NIZK.Sim}_1(1^\lambda)$.
 5. $(\text{mpk}, \text{msk}) := ((\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}), (\text{FE.mpk}, \text{PKE.pk}, \widetilde{\text{crs}}, \text{FE.msk}, \text{PKE.sk}, \text{SKE.sk}))$.
- Compute $\text{CT}^* \leftarrow \text{Enc}(\text{mpk}, x_0)$ as follows:
 1. $\text{FE.ct}^* \leftarrow \text{FE.Enc}(\text{FE.mpk}, (\perp, \perp, 1, \text{SKE.sk}))$.
 2. $\text{PKE.ct}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, \perp)$.
 3. $\pi^* \leftarrow \text{NIZK.Sim}_2(\widetilde{\text{crs}}, \tau, z)$ where z is the following statement on FE.ct^* and PKE.ct^* :
 “ PKE.ct^* correctly encrypts $(x, K, \alpha, \widehat{\text{sk}}, \text{FE.ct}^*)$, where $(x, K, \alpha, \widehat{\text{sk}})$ is encrypted in FE.ct^* ”
 4. Set $\text{CT}^* = (\text{FE.ct}^*, \text{PKE.ct}^*, \pi^*)$.
- Run $\mathcal{A}_2^{\text{KeyGen}(\text{msk}, \cdot), \mathcal{O}(\text{msk}, \cdot, \cdot)}(\text{mpk}, \text{CT}^*, \text{st})$, while simulating the following two oracles:
 - $\text{KeyGen}(\text{msk}, f)$:
 1. $s \leftarrow \{0, 1\}^\lambda$.
 2. Submit s and receive r .
 3. $\hat{y} \leftarrow f(x_0; r)$.
 4. $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, \hat{y})$.
 5. $\text{FE.sk}_{G_f} \leftarrow \text{FE.KeyGen}(\text{FE.msk}, G[f, s, \text{SKE.ct}])$.
 6. Output $\text{SK}_f = (\widetilde{\text{crs}}, \text{FE.sk}_{G_f})$.
 - $\mathcal{O}(\text{msk}, \text{CT}, g)$:
 1. If $\text{CT} = \text{CT}^*$, return $\perp$.
 2. $s \leftarrow \{0, 1\}^\lambda$.
 3. $\text{SKE.ct} \leftarrow \text{SKE.Enc}(\text{SKE.sk}, y)$, where $y \leftarrow \{0, 1\}^{|g(\cdot)|}$ is randomly sampled.
 4. Parse $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$.
 5. If $\text{NIZK.Verify}(\widetilde{\text{crs}}, (\text{FE.ct}, \text{PKE.ct}), \pi) = 0$, return $\perp$.
 6. Compute $(x, K, \alpha, \widehat{\text{sk}}, \text{FE.ct}') \leftarrow \text{PKE.Dec}(\text{PKE.sk}, \text{PKE.ct})$, and return $G[g, s, \text{SKE.ct}](x, K, \alpha, \widehat{\text{sk}})$.
- If $\mathcal{A}$ outputs it is in $\mathbf{Hybrid}_5^A$, output 0. Otherwise, output 1.

Note that the r $\mathcal{B}$ receives will either be $\text{PRF.Eval}(K, s)$ or a uniformly random value, depending

on the challenge bit. And these cases correspond exactly to **Hybrid**₅^A and **Hybrid**₆^A. It's easy to see that if $\mathcal{A}$ wins, $\mathcal{B}$ also wins. $\square$

Lemma 5.9. *No adversary $\mathcal{A}$ can distinguish between **Hybrid**₆^A and **Hybrid**₇^A with non-negligible probability.*

Proof. Note that the only difference between **Hybrid**₆^A and **Hybrid**₇^A is that we switch from $f(x_0; \hat{r})$ to $f(x_1; \hat{r})$, where $\hat{r}$ is a uniformly random value. Recall that the experiment requires that for all f queried by $\mathcal{A}_2$ to the **KeyGen** oracle, the distributions of $f(x_0)$ and $f(x_1)$ are statistically indistinguishable. Therefore, no adversary can distinguish between them with non-negligible probability as desired. $\square$

The proofs of the hybrid arguments for **Hybrid**₇^A to **Hybrid**₁₃^A follow analogously from the lemmas above, yielding us the final theorem result.

Theorem 5.2. *If PKE is a CPA-secure PKE, SKE is a CPA-secure SKE, NIZK is a NIZK with simulation soundness, FE is a selectively-secure FE scheme, and PRF is a secure PRF, then Construction 1 is IND-secure per Definition 5.1.*

Proof. The lemmas above show a sequence of a polynomial number of hybrid experiments where no PPT adversary can distinguish one from the next with non-negligible probability. The first hybrid **Hybrid**₀^A corresponds to the $\text{Expt}_{\mathcal{A}}^{\text{rFE}}$ game where $b = 0$, and the last one **Hybrid**₁₃^A corresponds to the case where $b = 1$. The security of the indistinguishability game follows. $\square$

5.4 Issue with Counterexample Construction (Construction 1)

Now we highlight why construction 1 is problematic against a malicious encryptor. The high level intuition is that a malicious encryptor can cause the randomized function evaluation to use a randomness of its own choice.

Specifically, let $\text{PRF}' = (\text{Setup}, \text{Eval})$ be a secure PRF, and consider the following PRF construction $\text{PRF} = (\text{Setup}, \text{Eval})$:

- **Setup**(1^λ): Compute $K' \leftarrow \text{PRF}'.\text{Setup}(1^\lambda)$, and output $K = (K', 0, \perp)$.
- **Eval**(K, s): First, parse $K = (K', b, r)$. If $b = 0$, output $\text{PRF}'.\text{Eval}(K', s)$. Otherwise, output r .

Essentially, it is a PRF with a “trapdoor” mode. Normally, when $b = 0$, it behaves like the underlying PRF' . But when $b = 1$, it completely ignores the seed s , and simply outputs some hardcoded randomness r . Notice that by security of PRF' , the construction PRF is also secure.

Now imagine using this PRF construction as the PRF in Construction 1. Here is what a malicious encryptor $\mathcal{A}$ can do:

Malicious Encryptor $\mathcal{A}(\text{mpk}, x)$:

1. Parse $\text{mpk} = (\text{FE.mpk}, \text{PKE.pk}, \text{crs})$.
2. $K' \leftarrow \text{PRF}'.\text{Setup}(1^\lambda)$.
3. Pick any preferred randomness r , and set $K = (K', 1, r)$.
4. $\text{FE.ct} \leftarrow \text{FE.Enc}(\text{FE.mpk}, (x, K, 0, \perp))$.
5. $\text{PKE.ct} \leftarrow \text{PKE.Enc}(\text{PKE.pk}, (x, K, 0, \perp, \text{FE.ct}))$.

6. $\pi \leftarrow \text{NIZK.Prove}(\text{crs}, z, w)$ where z is the statement that PKE.ct correctly encrypts $(x, K, \alpha, \widehat{\text{sk}}, \text{FE.ct})$, where $(x, K, \alpha, \widehat{\text{sk}})$ is encrypted in FE.ct .
7. Output $\text{CT} = (\text{FE.ct}, \text{PKE.ct}, \pi)$.

Notice that decrypting this malicious CT yields $G[f, s, \text{SKE.ct}](x, K, 0, \perp)$. But in this case PRF.Eval simply outputs the hardcoded r which is arbitrarily chosen by the malicious encryptor. In other words, the malicious encryptor can set the randomness used to evaluate the function f to any favorable value it wants.

6 Constructing Adaptively Secure rMIFE

In this section we present our construction of rMIFE. It is inspired by [GJO16], and we build upon the adaptively secure deterministic MIFE construction of Goldwasser et al. [GGG⁺14].

For our construction, we utilize a subexponentially-secure indistinguishability obfuscation ($i\mathcal{O}$), a plain PKE, a puncturable PRF and injective One Way Functions (OWFs), with parameters specified in the following subsection. The definitions of these tools can be found in Section 3.

6.1 Parameters

- PKE
 - Security parameter λ .
 - Ciphertexts of length s on inputs x_i .
- $i\mathcal{O} = i\mathcal{O}$ with $(1, 2^{-3ns-\lambda_{i\mathcal{O}}})$ weak extractability. This means for any two equivalent circuits, the security gap of the obfuscation is bounded by $2^{-3ns-\lambda_{i\mathcal{O}}}$ (any algorithm that distinguishes obfuscations of two circuits with more than this gap can be used to extract a differing point).
 - Security parameter $\lambda_{i\mathcal{O}} = \lambda$.
- $\text{PRF1} = \text{Puncturable PRF}$ with security $2^{-\lambda_{\text{PRF1}}^{\text{cPRF1}}}$. This is the PRF using K_i as keys.
 - Security parameter $\lambda_{\text{PRF1}} > (2ns + \lambda_{i\mathcal{O}})^{(1/\text{cPRF1})}$.
- $\text{PRF2} = \text{Puncturable PRF}$ with security $2^{-\lambda_{\text{PRF2}}^{\text{cPRF2}}}$. This is the PRF using K_f as keys.
 - Security parameter $\lambda_{\text{PRF2}} > (2ns + \lambda_{i\mathcal{O}})^{(1/\text{cPRF2})}$.
- $\text{InjOWF} = \text{Injective OWF}$ with security $2^{-\lambda_{\text{OWF}}^{\text{cOWF}}}$.
 - Security parameter $\lambda_{\text{OWF}} > (3ns + \lambda_{i\mathcal{O}})^{1/\text{cOWF}}$.
 - Output length $> \max\{(5ns + 2\lambda_{i\mathcal{O}})^{1/\text{cOWF}}, (3ns + \lambda_{i\mathcal{O}})^{1/\text{cPRF1}}, (3ns + \lambda_{i\mathcal{O}})^{1/\text{cPRF2}}\}$.

6.2 Construction

Now we present our main construction of rMIFE.

Construction 2. Let λ be the security parameter and n be the number of inputs. Let $i\mathcal{O}$ be an indistinguishability obfuscation scheme, $\text{PKE} = (\text{Setup}, \text{Enc}, \text{Dec})$ be a plain model PKE, $\text{PRF1}, \text{PRF2} = (\text{Setup}, \text{Eval}, \text{Punc})$ be puncturable PRFs, InjOWF be an injective OWF. We construct our $\text{rMIFE} = (\text{Setup}, \text{Enc}, \text{KeyGen}, \text{Dec})$ as follows:

- $\text{Setup}(1^\lambda, n)$:

1. For $i \in [n]$,
 - (a) $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - (b) For $b \in \{0, 1\}$, $(\text{pk}_i^b, \text{sk}_i^b) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 - (c) $\tilde{E}_i = i\mathcal{O}(1^\lambda, E_i[\text{pk}_i^0, \text{pk}_i^1, K_i])$.
 - (d) $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.
2. $\text{MSK} = \{\text{sk}_i^0, \text{sk}_i^1, K_i\}_{i \in [n]}$.
3. Output $(\text{MSK}, \{\text{EK}_i\}_{i \in [n]})$.

$E_i[\text{pk}_i^0, \text{pk}_i^1, K_i](c_i^0, c_i^1, x_i, r_i^0, r_i^1)$:

1. For $b \in \{0, 1\}$, if $c_i^b \neq \text{PKE.Enc}(\text{pk}_i^b, x_i; r_i^b)$, output $\perp$.
2. Output $z_i = \text{PRF1.Eval}(K_i, (c_i^0, c_i^1))$.

- $\text{Enc}(\text{EK}_i, x_i)$:

1. Parse $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.
2. $r_i^0, r_i^1 \leftarrow \{0, 1\}^\lambda$.
3. For $b \in \{0, 1\}$, $c_i^b = \text{PKE.Enc}(\text{pk}_i^b, x_i; r_i^b)$.
4. $z_i = \tilde{E}_i(c_i^0, c_i^1, x_i, r_i^0, r_i^1)$.
5. Output $\text{CT}_i = (c_i^0, c_i^1, z_i)$.

- $\text{KeyGen}(\text{MSK}, f)$:

1. Parse $\text{MSK} = (\{\text{sk}_i^0, \text{sk}_i^1, K_i\}_{i \in [n]})$.
2. Sample $K_f \leftarrow \text{PRF2.Setup}(1^\lambda)$.
3. $\tilde{G}_f \leftarrow i\mathcal{O}(1^\lambda, G[f, \{\text{sk}_i^0, K_i\}_{i \in [n]}, K_f, \text{InjOWF}])$.
4. Output $\text{SK}_f = \tilde{G}_f$.

$G[f, \{\text{sk}_i^0, K_i\}_{i \in [n]}, K_f, \text{InjOWF}]((c_i^0, c_i^1, z_i)_{i \in [n]})$:

1. For $i \in [n]$,
 - (a) If $\text{InjOWF}(z_i) \neq \text{InjOWF}(\text{PRF1.Eval}(K_i, c_i^0, c_i^1))$, output $\perp$.
 - (b) $x_i = \text{PKE.Dec}(\text{sk}_i^0, c_i^0)$.
2. $r_f = \text{PRF2.Eval}(K_f, (c_i^0, c_i^1)_{i \in [n]})$.
3. Output $y = f(x_1, x_2, \dots, x_n; r_f)$.

- $\text{Dec}(\text{SK}_f, \{\text{CT}_i\}_{i \in [n]})$.

1. Parse $\text{SK}_f = \tilde{G}_f$, and for $i \in [n]$, parse $\text{CT}_i = (c_i^0, c_i^1, z_i)$.
2. Output $y = \tilde{G}_f(\{c_i^0, c_i^1, z_i\}_{i \in [n]})$.

Correctness follows from correctness of the underlying schemes. We next argue our construction is secure against both malicious decryptors and malicious encryptors.

6.3 Security against malicious decryptors

We first prove security against malicious decryptors.

Theorem 6.1. *If PKE is CPA-secure, $i\mathcal{O}$ is a subexponentially-secure indistinguishable obfuscation scheme, PRFs and InjOWF are secure with the parameters outlines in the Parameters section, then Construction 2 is IND secure against malicious decryptors for $\epsilon = 2^{-2ns-\lambda_{i\mathcal{O}}}$ -distinguishable distributions.*

We prove this through a sequence of hybrids.

Sequence of Hybrids

Hybrid₀^A(1^λ): Real world experiment with $b = 0$.

1. Setup:

- (a) For $i \in [n]$,
 - i. $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - ii. For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 - iii. $\tilde{E}_i = i\mathcal{O}(1^\lambda, E_i[\text{pk}_i^0, \text{pk}_i^1, K_i])$.
 - iv. $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.

2. $\mathcal{A}$ may make any number of the following queries in any order.

• Function Query:

- (a) $\mathcal{A}$ outputs a function f_ℓ .
- (b) $K_{f_\ell} \leftarrow \text{PRF2.Setup}(1^\lambda)$.
- (c) $\tilde{G}_{f_\ell} \leftarrow i\mathcal{O}(1^\lambda, G[f_\ell, \{\text{sk}_i^0, K_i\}_{i \in [n]}, K_{f_\ell}, \text{InjOWF}])$.
- (d) Send $\text{SK}_{f_\ell} = \tilde{G}_{f_\ell}$ to $\mathcal{A}$.

• Encryption Key Query:

- (a) $\mathcal{A}$ outputs an index $i \in [n]$.
- (b) Send EK_i to $\mathcal{A}$.

• Challenge Message Query:

- (a) $\mathcal{A}$ outputs $(X_j^0, X_j^1) = ((x_{j,1}^0, \dots, x_{j,n}^0), (x_{j,1}^1, \dots, x_{j,n}^1))$.
- (b) For $i \in [n]$,
 - i. $r_{j,i}^0, r_{j,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_{j,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{j,i}^0; r_{j,i}^0)$.
 - iii. $c_{j,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{j,i}^0; r_{j,i}^1)$.
 - iv. $z_{j,i} = \text{PRF1.Eval}(K_i, (c_{j,i}^0, c_{j,i}^1))$.
 - v. $\text{CT}_{j,i} = (c_{j,i}^0, c_{j,i}^1, z_{j,i})$.
- (c) Send $\{\text{CT}_{j,i}\}_{i \in [n]}$ to $\mathcal{A}$.

3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

Hybrid₁^A(1^λ): We encrypt $x_{j,i}^1$ into $c_{j,i}^1$, that is we compute $c_{j,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{j,i}^1; r_{j,i}^1)$. This follows by PKE security.

1. **Setup:**

- (a) For $i \in [n]$,
 - i. $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - ii. For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 - iii. $\tilde{E}_i = i\mathcal{O}(1^\lambda, E_i[\text{pk}_i^0, \text{pk}_i^1, K_i])$.
 - iv. $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.

2. $\mathcal{A}$ may make any number of the following queries in any order.

• **Function Query:**

- (a) $\mathcal{A}$ outputs a function f_ℓ .
- (b) $K_{f_\ell} \leftarrow \text{PRF2.Setup}(1^\lambda)$.
- (c) $\tilde{G}_{f_\ell} \leftarrow i\mathcal{O}(1^\lambda, G[f_\ell, \{\text{sk}_i^0, K_i\}_{i \in [n]}, K_{f_\ell}, \text{InjOWF}])$.
- (d) Send $\text{SK}_{f_\ell} = \tilde{G}_{f_\ell}$ to $\mathcal{A}$.

• **Encryption Key Query:**

- (a) $\mathcal{A}$ outputs an index $i \in [n]$.
- (b) Send EK_i to $\mathcal{A}$.

• **Challenge Message Query:**

- (a) $\mathcal{A}$ outputs $(X_j^0, X_j^1) = ((x_{j,1}^0, \dots, x_{j,n}^0), (x_{j,1}^1, \dots, x_{j,n}^1))$.
- (b) For $i \in [n]$,
 - i. $r_{j,i}^0, r_{j,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_{j,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{j,i}^0; r_{j,i}^0)$.
 - iii. $c_{j,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{j,i}^1; r_{j,i}^1)$.
 - iv. $z_{j,i} = \text{PRF1.Eval}(K_i, (c_{j,i}^0, c_{j,i}^1))$.
 - v. $\text{CT}_{j,i} = (c_{j,i}^0, c_{j,i}^1, z_{j,i})$.
- (c) Send $\{\text{CT}_{j,i}\}_{i \in [n]}$ to $\mathcal{A}$.

3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

Hybrid_{2,w,0}⁴(1^λ) _{$w \in [0, 2^{2sn}]$} : Here s denotes the length of the PKE ciphertexts. In this sequence of hybrids, we consider only one secret key query from the adversary for some function f . Multiple key queries can be handled via a standard hybrid argument. While generating the secret key queried by the adversary we construct the program G_f , by switching to computing x_i using sk_i^1 and c_i^1 for all inputs with $(c_i^0, c_i^1)_{i \in [n]} < w$. For $w = 0$, this is indistinguishable from the previous hybrid by $i\mathcal{O}$ since the program G in the two hybrids is exactly identical in functionality.

1. **Setup:**

- (a) For $i \in [n]$,
 - i. $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - ii. For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 - iii. $\tilde{E}_i = i\mathcal{O}(1^\lambda, E_i[\text{pk}_i^0, \text{pk}_i^1, K_i])$.
 - iv. $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.

2. $\mathcal{A}$ may make any number of the following queries in any order.

- **Function Query:**

- $\mathcal{A}$ outputs a function f .
- $K_f \leftarrow \text{PRF2.Setup}(1^\lambda)$.
- $\tilde{G}_f \leftarrow i\mathcal{O}(1^\lambda, G'[f, \{\text{sk}_i^0, \text{sk}_i^1, K_i\}_{i \in [n]}, w, K_f, \text{InjOWF}])$.
- Send $\text{SK}_f = \tilde{G}_f$ to $\mathcal{A}$.

- **Encryption Key Query:**

- $\mathcal{A}$ outputs an index $i \in [n]$.
- Send EK_i to $\mathcal{A}$.

- **Challenge Message Query:**

- $\mathcal{A}$ outputs $(X_j^0, X_j^1) = ((x_{j,1}^0, \dots, x_{j,n}^0), (x_{j,1}^1, \dots, x_{j,n}^1))$.
- For $i \in [n]$,
 - $r_{j,i}^0, r_{j,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - $c_{j,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{j,i}^0; r_{j,i}^0)$.
 - $c_{j,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{j,i}^1; r_{j,i}^1)$.
 - $z_{j,i} = \text{PRF1.Eval}(K_i, (c_{j,i}^0, c_{j,i}^1))$.
 - $\text{CT}_{j,i} = (c_{j,i}^0, c_{j,i}^1, z_{j,i})$.
- Send $\{\text{CT}_{j,i}\}_{i \in [n]}$ to $\mathcal{A}$.

3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

$G'[f, \{\text{sk}_i^0, \text{sk}_i^1, K_i\}_{i \in [n]}, w, K_f, \text{InjOWF}](c_i^0, c_i^1, z_i)_{i \in [n]}$:

- For $i \in [n]$,
 - If $\text{InjOWF}(z_i) \neq \text{InjOWF}(\text{PRF1.Eval}(K_i, c_i^0, c_i^1))$, output $\perp$.
 - If $(c_i^0, c_i^1)_{i \in [n]} \geq w$, $x_i = \text{PKE.Dec}(\text{sk}_i^0, c_i^0)$.
 - If $(c_i^0, c_i^1)_{i \in [n]} < w$, $x_i = \text{PKE.Dec}(\text{sk}_i^1, c_i^1)$.
- $r_f = \text{PRF2.Eval}(K_f, (c_i^0, c_i^1)_{i \in [n]})$.
- Output $y = f(x_1, x_2, \dots, x_n; r_f)$.

Hybrid $_{2,w,1}^A(1^\lambda)_{w \in [0, 2^{2sn}]}$: Puncture the PRF1 keys K_i at values associated with w .

1. **Setup:**

- For $i \in [n]$,
 - $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
- Compute Values for w :**
 - Parse $w = (d_i^0, d_i^1)_{i \in [n]}$.
 - For $i \in [n]$, $\alpha \in \{0, 1\}$, $x_i^\alpha = \text{PKE.Dec}(\text{sk}_i^\alpha, d_i^\alpha)$.
 - Define $A_w = \{i \in [n] : x_i^0 = x_i^1\}$.
 - For $i \in A_w$, $u_i = \text{PRF1.Eval}(K_i, (d_i^0, d_i^1))$.

v. For $i \in [n]$, $K_i[d_i^0, d_i^1] = \text{PRF1.Punc}(K_i, (d_i^0, d_i^1))$.

(c) Compute:

- i. For $i \in A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E'_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1, u_i])$.
- ii. For $i \in [n] \setminus A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E''_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1])$.

(d) For $i \in [n]$, set $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.

2. $\mathcal{A}$ may make any number of the following queries in any order.

• **Function Query:**

- (a) $\mathcal{A}$ outputs a function f .
- (b) $K_f \leftarrow \text{PRF2.Setup}(1^\lambda)$.
- (c) $\tilde{G}_f \leftarrow i\mathcal{O}(1^\lambda, G''[f, \{\text{sk}_i^0, \text{sk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1\}_{i \in [n]}, \{\text{InjOWF}(u_i)\}_{i \in A_w}, w, K_f, \text{InjOWF}])$.
- (d) Send $\text{SK}_f = \tilde{G}_f$ to $\mathcal{A}$.

• **Encryption Key Query:**

- (a) $\mathcal{A}$ outputs an index $i \in [n]$.
- (b) Send EK_i to $\mathcal{A}$.

• **Challenge Message Query:**

- (a) $\mathcal{A}$ outputs $(X_j^0, X_j^1) = ((x_{j,1}^0, \dots, x_{j,n}^0), (x_{j,1}^1, \dots, x_{j,n}^1))$.
- (b) For $i \in [n]$,
 - i. $r_{j,i}^0, r_{j,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_{j,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{j,i}^0; r_{j,i}^0)$.
 - iii. $c_{j,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{j,i}^1; r_{j,i}^1)$.
 - iv. $z_{j,i} = \text{PRF1.Eval}(K_i, (c_{j,i}^0, c_{j,i}^1))$.
 - v. $\text{CT}_{j,i} = (c_{j,i}^0, c_{j,i}^1, z_{j,i})$.
- (c) Send $\{\text{CT}_{j,i}\}_{i \in [n]}$ to $\mathcal{A}$.

3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

$E'_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1, u_i](c_i^0, c_i^1, x_i, r_i^0, r_i^1)$:

1. For $b \in \{0, 1\}$, if $c_i^b \neq \text{PKE.Enc}(\text{pk}_i^b, x_i; r_i^b)$, output $\perp$.
2. If $(c_i^0, c_i^1) = (d_i^0, d_i^1)$, output u_i .
3. Else, output $z_i = \text{PRF1.Eval}(K_i[d_i^0, d_i^1], (c_i^0, c_i^1))$.

$E''_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1](c_i^0, c_i^1, x_i, r_i^0, r_i^1)$:

1. For $b \in \{0, 1\}$, if $c_i^b \neq \text{PKE.Enc}(\text{pk}_i^b, x_i; r_i^b)$, output $\perp$.
2. Output $z_i = \text{PRF1.Eval}(K_i[d_i^0, d_i^1], (c_i^0, c_i^1))$.

$G'''[f, \{\text{sk}_i^0, \text{sk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1\}_{i \in [n]}, \{\text{InjOWF}(u_i)\}_{i \in A_w}, w, K_f, \text{InjOWF}]$
 $((c_i^0, c_i^1, z_i)_{i \in [n]}):$

1. For $i \in [n]$,
 - (a) If $(c_i^0, c_i^1) = (d_i^0, d_i^1)$, if $\text{InjOWF}(z_i) \neq \text{InjOWF}(u_i)$, output $\perp$.
 - (b) Else, if $\text{InjOWF}(z_i) \neq \text{InjOWF}(\text{PRF1.Eval}(K_i[d_i^0, d_i^1], c_i^0, c_i^1))$, output $\perp$.
 - (c) If $(c_i^0, c_i^1)_{i \in [n]} \geq w$, $x_i = \text{PKE.Dec}(\text{sk}_i^0, c_i^0)$.
 - (d) If $(c_i^0, c_i^1)_{i \in [n]} < w$, $x_i = \text{PKE.Dec}(\text{sk}_i^1, c_i^1)$.
2. $r_f = \text{PRF2.Eval}(K_f, (c_i^0, c_i^1)_{i \in [n]})$.
3. Output $y = f(x_1, x_2, \dots, x_n; r_f)$.

Hybrid $_{2,w,2}^A(1^\lambda)$: Choose u_i uniformly at random. This follows by PRF1 security.

1. Setup:

- (a) For $i \in [n]$,
 - i. $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - ii. For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
- (b) **Compute Values for w :**
 - i. Parse $w = (d_i^0, d_i^1)_{i \in [n]}$.
 - ii. For $i \in [n]$, $\alpha \in \{0, 1\}$, $x_i^\alpha = \text{PKE.Dec}(\text{sk}_i^\alpha, d_i^\alpha)$.
 - iii. Define $A_w = \{i \in [n] : x_i^0 = x_i^1\}$.
 - iv. For $i \in A_w$, $u_i \leftarrow \{0, 1\}^\lambda$.
 - v. For $i \in [n]$, $K_i[d_i^0, d_i^1] = \text{PRF1.Punc}(K_i, (d_i^0, d_i^1))$.
- (c) Compute:
 - i. For $i \in A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E'_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1, u_i])$.
 - ii. For $i \in [n] \setminus A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E'_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1])$.
- (d) For $i \in [n]$, set $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.

2. $\mathcal{A}$ may make any number of the following queries in any order.

• **Function Query:**

- (a) $\mathcal{A}$ outputs a function f .
- (b) $K_f \leftarrow \text{PRF2.Setup}(1^\lambda)$.
- (c) $\tilde{G}_f \leftarrow i\mathcal{O}(1^\lambda, G''[f, \{\text{sk}_i^0, \text{sk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1\}_{i \in [n]}, \{\text{InjOWF}(u_i)\}_{i \in A_w}, w, K_f, \text{InjOWF}])$.
- (d) Send $\text{SK}_f = \tilde{G}_f$ to $\mathcal{A}$.

• **Encryption Key Query:**

- (a) $\mathcal{A}$ outputs an index $i \in [n]$.
- (b) Send EK_i to $\mathcal{A}$.

• **Challenge Message Query:**

- (a) $\mathcal{A}$ outputs $(X_j^0, X_j^1) = ((x_{j,1}^0, \dots, x_{j,n}^0), (x_{j,1}^1, \dots, x_{j,n}^1))$.

- (b) For $i \in [n]$,
 - i. $r_{j,i}^0, r_{j,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_{j,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{j,i}^0; r_{j,i}^0)$.
 - iii. $c_{j,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{j,i}^1; r_{j,i}^1)$.
 - iv. $z_{j,i} = \text{PRF1.Eval}(K_i, (c_{j,i}^0, c_{j,i}^1))$.
 - v. $\text{CT}_{j,i} = (c_{j,i}^0, c_{j,i}^1, z_{j,i})$.
- (c) Send $\{\text{CT}_{j,i}\}_{i \in [n]}$ to $\mathcal{A}$.

3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

Hybrid $_{2,w,3}^A(1^\lambda)$: Puncture each K_f at w . This step also follows from $i\mathcal{O}$.

1. **Setup:**

- (a) For $i \in [n]$,
 - i. $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - ii. For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
- (b) **Compute Values for w :**
 - i. Parse $w = (d_i^0, d_i^1)_{i \in [n]}$.
 - ii. For $i \in [n]$, $\alpha \in \{0, 1\}$, $x_i^\alpha = \text{PKE.Dec}(\text{sk}_i^\alpha, d_i^\alpha)$.
 - iii. Define $A_w = \{i \in [n] : x_i^0 = x_i^1\}$.
 - iv. For $i \in A_w$, $u_i \leftarrow \{0, 1\}^\lambda$.
 - v. For $i \in [n]$, $K_i[d_i^0, d_i^1] = \text{PRF1.Punc}(K_i, (d_i^0, d_i^1))$.
- (c) Compute:
 - i. For $i \in A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E'_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1, u_i])$.
 - ii. For $i \in [n] \setminus A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E''_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1])$.
- (d) For $i \in [n]$, set $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.

2. $\mathcal{A}$ may make any number of the following queries in any order.

• **Function Query:**

- (a) $\mathcal{A}$ outputs a function f .
- (b) $K_f \leftarrow \text{PRF2.Setup}(1^\lambda)$.
- (c) $K_f[w] = \text{PRF2.Punc}(K_f, w)$.
- (d) $r^* \leftarrow \text{PRF2.Eval}(K_f, w)$.
- (e) For $i \in [n]$, $x_i^0 \leftarrow \text{PKE.Dec}(\text{sk}_i^0, d_i^0)$.
- (f) $y^* = f(x_1^0, x_2^0, \dots, x_n^0; r^*)$.
- (g) $\tilde{G}_f \leftarrow i\mathcal{O}(1^\lambda, G'''[f, \{\text{sk}_i^0, \text{sk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1\}_{i \in [n]}, \{\text{InjOWF}(u_i)\}_{i \in A_w}, w, K_f[w], y^*, \text{InjOWF}])$.
- (h) Send $\text{SK}_f = \tilde{G}_f$ to $\mathcal{A}$.

• **Encryption Key Query:**

- (a) $\mathcal{A}$ outputs an index $i \in [n]$.
- (b) Send EK_i to $\mathcal{A}$.

• **Challenge Message Query:**

- (a) $\mathcal{A}$ outputs $(X_j^0, X_j^1) = ((x_{j,1}^0, \dots, x_{j,n}^0), (x_{j,1}^1, \dots, x_{j,n}^1))$.
- (b) For $i \in [n]$,
 - i. $r_{j,i}^0, r_{j,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_{j,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{j,i}^0; r_{j,i}^0)$.
 - iii. $c_{j,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{j,i}^1; r_{j,i}^1)$.
 - iv. $z_{j,i} = \text{PRF1.Eval}(K_i, (c_{j,i}^0, c_{j,i}^1))$.
 - v. $\text{CT}_{j,i} = (c_{j,i}^0, c_{j,i}^1, z_{j,i})$.
- (c) Send $\{\text{CT}_{j,i}\}_{i \in [n]}$ to $\mathcal{A}$.

3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

$G'''[f, \{\text{sk}_i^0, \text{sk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1\}_{i \in [n]}, \{\text{InjOWF}(u_i)\}_{i \in A_w}, w, K_f[w], y^*, \text{InjOWF}((c_i^0, c_i^1, z_i)_{i \in [n])}]$

1. For $i \in [n]$,
 - (a) If $(c_i^0, c_i^1) = (d_i^0, d_i^1)$, if $\text{InjOWF}(z_i) \neq \text{InjOWF}(u_i)$, output $\perp$.
 - (b) Else, if $\text{InjOWF}(z_i) \neq \text{InjOWF}(\text{PRF1.Eval}(K_i[d_i^0, d_i^1], c_i^0, c_i^1))$, output $\perp$.
 - (c) If $(c_i^0, c_i^1)_{i \in [n]} > w$, $x_i = \text{PKE.Dec}(\text{sk}_i^0, c_i^0)$.
 - (d) If $(c_i^0, c_i^1)_{i \in [n]} = w$, output y^* .
 - (e) If $(c_i^0, c_i^1)_{i \in [n]} < w$, $x_i = \text{PKE.Dec}(\text{sk}_i^1, c_i^1)$.
2. $r_f = \text{PRF2.Eval}(K_f[w], (c_i^0, c_i^1)_{i \in [n]})$.
3. Output $y = f(x_1, x_2, \dots, x_n; r_f)$.

Hybrid $_{2,w,4}^A(1^\lambda)$: Now replace r^* with a uniform value. This follows from puncturable PRF2 security.

1. **Setup:**

- (a) For $i \in [n]$,
 - i. $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - ii. For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
- (b) **Compute Values for w :**
 - i. Parse $w = (d_i^0, d_i^1)_{i \in [n]}$.
 - ii. For $i \in [n]$, $\alpha \in \{0, 1\}$, $x_i^\alpha = \text{PKE.Dec}(\text{sk}_i^\alpha, d_i^\alpha)$.
 - iii. Define $A_w = \{i \in [n] : x_i^0 = x_i^1\}$.
 - iv. For $i \in A_w$, $u_i \leftarrow \{0, 1\}^\lambda$.
 - v. For $i \in [n]$, $K_i[d_i^0, d_i^1] = \text{PRF1.Punc}(K_i, (d_i^0, d_i^1))$.
- (c) **Compute:**
 - i. For $i \in A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E'_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1, u_i])$.
 - ii. For $i \in [n] \setminus A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E''_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1])$.
- (d) For $i \in [n]$, set $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.

2. $\mathcal{A}$ may make any number of the following queries in any order.

• **Function Query:**

- (a) $\mathcal{A}$ outputs a function f .
- (b) $K_f \leftarrow \text{PRF2.Setup}(1^\lambda)$.
- (c) $K_f[w] = \text{PRF2.Punc}(K_f, w)$.
- (d) $r^* \leftarrow \{0, 1\}^\lambda$.
- (e) For $i \in [n]$, $x_i^0 \leftarrow \text{PKE.Dec}(\text{sk}_i^0, d_i^0)$.
- (f) $y^* = f(x_1^0, x_2^0, \dots, x_n^0; r^*)$.
- (g) $\tilde{G}_f \leftarrow i\mathcal{O}(1^\lambda, G'''[f, \{\text{sk}_i^0, \text{sk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1\}_{i \in [n]}, \{\text{InjOWF}(u_i)\}_{i \in A_w}, w, K_f[w], y^*, \text{InjOWF}])$.
- (h) Send $\text{SK}_f = \tilde{G}_f$ to $\mathcal{A}$.

• **Encryption Key Query:**

- (a) $\mathcal{A}$ outputs an index $i \in [n]$.
- (b) Send EK_i to $\mathcal{A}$.

• **Challenge Message Query:**

- (a) $\mathcal{A}$ outputs $(X_j^0, X_j^1) = ((x_{j,1}^0, \dots, x_{j,n}^0), (x_{j,1}^1, \dots, x_{j,n}^1))$.
- (b) For $i \in [n]$,
 - i. $r_{j,i}^0, r_{j,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_{j,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{j,i}^0; r_{j,i}^0)$.
 - iii. $c_{j,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{j,i}^1; r_{j,i}^1)$.
 - iv. $z_{j,i} = \text{PRF1.Eval}(K_i, (c_{j,i}^0, c_{j,i}^1))$.
 - v. $\text{CT}_{j,i} = (c_{j,i}^0, c_{j,i}^1, z_{j,i})$.
- (c) Send $\{\text{CT}_{j,i}\}_{i \in [n]}$ to $\mathcal{A}$.

3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

Hybrid_{2,w,5}^A(1^λ): In Function Queries, we now compute $x_i^1 \leftarrow \text{PKE.Dec}(\text{sk}_i^1, d_i^1)$ instead of $x_i^0 \leftarrow \text{PKE.Dec}(\text{sk}_i^0, d_i^0)$, and update y^* to be evaluation on the x_i^1 's accordingly. This step follows from $i\mathcal{O}$, InjOWF and $\mathcal{I}$ -randomized-compatibility.

1. **Setup:**

- (a) For $i \in [n]$,
 - i. $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - ii. For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
- (b) **Compute Values for w :**
 - i. Parse $w = (d_i^0, d_i^1)_{i \in [n]}$.
 - ii. For $i \in [n]$, $\alpha \in \{0, 1\}$, $x_i^\alpha = \text{PKE.Dec}(\text{sk}_i^\alpha, d_i^\alpha)$.
 - iii. Define $A_w = \{i \in [n] : x_i^0 = x_i^1\}$.
 - iv. For $i \in A_w$, $u_i \leftarrow \{0, 1\}^\lambda$.
 - v. For $i \in [n]$, $K_i[d_i^0, d_i^1] = \text{PRF1.Punc}(K_i, (d_i^0, d_i^1))$.
- (c) Compute:

- i. For $i \in A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E'_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1, u_i])$.
 - ii. For $i \in [n] \setminus A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E''_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1])$.
 - (d) For $i \in [n]$, set $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.
2. $\mathcal{A}$ may make any number of the following queries in any order.
- **Function Query:**
 - (a) $\mathcal{A}$ outputs a function f .
 - (b) $K_f \leftarrow \text{PRF2.Setup}(1^\lambda)$.
 - (c) $K_f[w] = \text{PRF2.Punc}(K_f, w)$.
 - (d) $r^* \leftarrow \{0, 1\}^\lambda$.
 - (e) For $i \in [n]$, $x_i^1 \leftarrow \text{PKE.Dec}(\text{sk}_i^1, d_i^1)$.
 - (f) $y^* = f(x_1^1, x_2^1, \dots, x_n^1; r^*)$.
 - (g) $\tilde{G}_f \leftarrow i\mathcal{O}(1^\lambda, G'''[f, \{\text{sk}_i^0, \text{sk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1\}_{i \in [n]}, \{\text{InjOWF}(u_i)\}_{i \in A_w}, w, K_f[w], y^*, \text{InjOWF}])$.
 - (h) Send $\text{SK}_f = \tilde{G}_f$ to $\mathcal{A}$.
 - **Encryption Key Query:**
 - (a) $\mathcal{A}$ outputs an index $i \in [n]$.
 - (b) Send EK_i to $\mathcal{A}$.
 - **Challenge Message Query:**
 - (a) $\mathcal{A}$ outputs $(X_j^0, X_j^1) = ((x_{j,1}^0, \dots, x_{j,n}^0), (x_{j,1}^1, \dots, x_{j,n}^1))$.
 - (b) For $i \in [n]$,
 - i. $r_{j,i}^0, r_{j,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_{j,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{j,i}^0; r_{j,i}^0)$.
 - iii. $c_{j,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{j,i}^1; r_{j,i}^1)$.
 - iv. $z_{j,i} = \text{PRF1.Eval}(K_i, (c_{j,i}^0, c_{j,i}^1))$.
 - v. $\text{CT}_{j,i} = (c_{j,i}^0, c_{j,i}^1, z_{j,i})$.
 - (c) Send $\{\text{CT}_{j,i}\}_{i \in [n]}$ to $\mathcal{A}$.
3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

Hybrid_{2,w,6}^A(1^λ): Now we undo the changes introduced in **Hybrid**_{2,w,4}^A by changing r^* back to PRF2 evaluation.

1. **Setup:**

- (a) For $i \in [n]$,
 - i. $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - ii. For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
- (b) **Compute Values for w :**
 - i. Parse $w = (d_i^0, d_i^1)_{i \in [n]}$.
 - ii. For $i \in [n]$, $\alpha \in \{0, 1\}$, $x_i^\alpha = \text{PKE.Dec}(\text{sk}_i^\alpha, d_i^\alpha)$.
 - iii. Define $A_w = \{i \in [n] : x_i^0 = x_i^1\}$.
 - iv. For $i \in A_w$, $u_i \leftarrow \{0, 1\}^\lambda$.

- v. For $i \in [n]$, $K_i[d_i^0, d_i^1] = \text{PRF1.Punc}(K_i, (d_i^0, d_i^1))$.
- (c) Compute:
- For $i \in A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E'_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1, u_i])$.
 - For $i \in [n] \setminus A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E''_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1])$.
- (d) For $i \in [n]$, set $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.
2. $\mathcal{A}$ may make any number of the following queries in any order.
- **Function Query:**
 - $\mathcal{A}$ outputs a function f .
 - $K_f \leftarrow \text{PRF2.Setup}(1^\lambda)$.
 - $K_f[w] = \text{PRF2.Punc}(K_f, w)$.
 - $r^* \leftarrow \text{PRF2.Eval}(K_f, w)$.
 - For $i \in [n]$, $x_i^1 \leftarrow \text{PKE.Dec}(\text{sk}_i^1, d_i^1)$.
 - $y^* = f(x_1^1, x_2^1, \dots, x_n^1; r^*)$.
 - $\tilde{G}_f \leftarrow i\mathcal{O}(1^\lambda, G'''[f, \{\text{sk}_i^0, \text{sk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1\}_{i \in [n]}, \{\text{InjOWF}(u_i)\}_{i \in A_w}, w, K_f[w], y^*, \text{InjOWF}])$.
 - Send $\text{SK}_f = \tilde{G}_f$ to $\mathcal{A}$.
 - **Encryption Key Query:**
 - $\mathcal{A}$ outputs an index $i \in [n]$.
 - Send EK_i to $\mathcal{A}$.
 - **Challenge Message Query:**
 - $\mathcal{A}$ outputs $(X_j^0, X_j^1) = ((x_{j,1}^0, \dots, x_{j,n}^0), (x_{j,1}^1, \dots, x_{j,n}^1))$.
 - For $i \in [n]$,
 - $r_{j,i}^0, r_{j,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - $c_{j,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{j,i}^0; r_{j,i}^0)$.
 - $c_{j,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{j,i}^1; r_{j,i}^1)$.
 - $z_{j,i} = \text{PRF1.Eval}(K_i, (c_{j,i}^0, c_{j,i}^1))$.
 - $\text{CT}_{j,i} = (c_{j,i}^0, c_{j,i}^1, z_{j,i})$.
 - Send $\{\text{CT}_{j,i}\}_{i \in [n]}$ to $\mathcal{A}$.
3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

Hybrid $_{2,w,7}^A(1^\lambda)$: Now we undo the changes introduced in **Hybrid** $_{2,w,3}^A$ by removing the puncturing on K_f . Notice that results in program G'' , but now with index $w + 1$, since we are now running $\text{PKE.Dec}(\text{sk}_i^1, c_i^1)$ for w . This step follows from $i\mathcal{O}$.

1. **Setup:**

- For $i \in [n]$,
 - $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
- Compute Values for w :**
 - Parse $w = (d_i^0, d_i^1)_{i \in [n]}$.

- ii. For $i \in [n]$, $\alpha \in \{0, 1\}$, $x_i^\alpha = \text{PKE.Dec}(\text{sk}_i^\alpha, d_i^\alpha)$.
 - iii. Define $A_w = \{i \in [n] : x_i^0 = x_i^1\}$.
 - iv. For $i \in A_w$, $u_i \leftarrow \{0, 1\}^\lambda$.
 - v. For $i \in [n]$, $K_i[d_i^0, d_i^1] = \text{PRF1.Punc}(K_i, (d_i^0, d_i^1))$.
- (c) Compute:
- i. For $i \in A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E'_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1, u_i])$.
 - ii. For $i \in [n] \setminus A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E''_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1])$.
- (d) For $i \in [n]$, set $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.
2. $\mathcal{A}$ may make any number of the following queries in any order.
- **Function Query:**
 - (a) $\mathcal{A}$ outputs a function f .
 - (b) $K_f \leftarrow \text{PRF2.Setup}(1^\lambda)$.
 - (c) ~~$K_f[w] \leftarrow \text{PRF2.Punc}(K_f, w)$.~~
 - (d) ~~$r^* \leftarrow \text{PRF2.Eval}(K_f, w)$.~~
 - (e) ~~For $i \in [n]$, $x_i^1 \leftarrow \text{PKE.Dec}(\text{sk}_i^1, d_i^1)$.~~
 - (f) ~~$y^* = f(x_1^1, x_2^1, \dots, x_n^1; r^*)$.~~
 - (g) $\tilde{G}_f \leftarrow i\mathcal{O}(1^\lambda, \textcolor{red}{G}''[f, \{\text{sk}_i^0, \text{sk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1\}_{i \in [n]}, \{\text{InjOWF}(u_i)\}_{i \in A_w}, w + 1, K_f[\textcolor{red}{w}], \textcolor{red}{y}^*, \text{InjOWF}])$.
 - (h) Send $\text{SK}_f = \tilde{G}_f$ to $\mathcal{A}$.
 - **Encryption Key Query:**
 - (a) $\mathcal{A}$ outputs an index $i \in [n]$.
 - (b) Send EK_i to $\mathcal{A}$.
 - **Challenge Message Query:**
 - (a) $\mathcal{A}$ outputs $(X_j^0, X_j^1) = ((x_{j,1}^0, \dots, x_{j,n}^0), (x_{j,1}^1, \dots, x_{j,n}^1))$.
 - (b) For $i \in [n]$,
 - i. $r_{j,i}^0, r_{j,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_{j,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{j,i}^0; r_{j,i}^0)$.
 - iii. $c_{j,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{j,i}^1; r_{j,i}^1)$.
 - iv. $z_{j,i} = \text{PRF1.Eval}(K_i, (c_{j,i}^0, c_{j,i}^1))$.
 - v. $\text{CT}_{j,i} = (c_{j,i}^0, c_{j,i}^1, z_{j,i})$.
 - (c) Send $\{\text{CT}_{j,i}\}_{i \in [n]}$ to $\mathcal{A}$.

3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

Hybrid_{2,w,8}^A(1^λ): Now we undo the changes introduced in **Hybrid**_{2,w,2}^A by changing u_i 's back to PRF1 outputs. This follows from PRF1 security.

1. **Setup:**

- (a) For $i \in [n]$,
 - i. $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - ii. For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.

(b) **Compute Values for w :**

- i. Parse $w = (d_i^0, d_i^1)_{i \in [n]}$.
- ii. For $i \in [n]$, $\alpha \in \{0, 1\}$, $x_i^\alpha = \text{PKE.Dec}(\text{sk}_i^\alpha, d_i^\alpha)$.
- iii. Define $A_w = \{i \in [n] : x_i^0 = x_i^1\}$.
- iv. For $i \in A_w$, $u_i \leftarrow \text{PRF1.Eval}(K_i, (d_i^0, d_i^1))$.
- v. For $i \in [n]$, $K_i[d_i^0, d_i^1] = \text{PRF1.Punc}(K_i, (d_i^0, d_i^1))$.

(c) **Compute:**

- i. For $i \in A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E'_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1, u_i])$.
- ii. For $i \in [n] \setminus A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E''_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1])$.

(d) For $i \in [n]$, set $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.

2. $\mathcal{A}$ may make any number of the following queries in any order.

• **Function Query:**

- (a) $\mathcal{A}$ outputs a function f .
- (b) $K_f \leftarrow \text{PRF2.Setup}(1^\lambda)$.
- (c) $\tilde{G}_f \leftarrow i\mathcal{O}(1^\lambda, G''[f, \{\text{sk}_i^0, \text{sk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1\}_{i \in [n]}, \{\text{InjOWF}(u_i)\}_{i \in A_w}, w+1, K_f, \text{InjOWF}])$.
- (d) Send $\text{SK}_f = \tilde{G}_f$ to $\mathcal{A}$.

• **Encryption Key Query:**

- (a) $\mathcal{A}$ outputs an index $i \in [n]$.
- (b) Send EK_i to $\mathcal{A}$.

• **Challenge Message Query:**

- (a) $\mathcal{A}$ outputs $(X_j^0, X_j^1) = ((x_{j,1}^0, \dots, x_{j,n}^0), (x_{j,1}^1, \dots, x_{j,n}^1))$.
- (b) For $i \in [n]$,
 - i. $r_{j,i}^0, r_{j,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_{j,i}^0 = \text{PKE.Enc}(\text{pk}_{j,i}^0, x_{j,i}^0; r_{j,i}^0)$.
 - iii. $c_{j,i}^1 = \text{PKE.Enc}(\text{pk}_{j,i}^1, x_{j,i}^1; r_{j,i}^1)$.
 - iv. $z_{j,i} = \text{PRF1.Eval}(K_i, (c_{j,i}^0, c_{j,i}^1))$.
 - v. $\text{CT}_{j,i} = (c_{j,i}^0, c_{j,i}^1, z_{j,i})$.
- (c) Send $\{\text{CT}_{j,i}\}_{i \in [n]}$ to $\mathcal{A}$.

3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

Hybrid $_{2,w,9}^A(1^\lambda)$: Now we undo the changes introduced in **Hybrid** $_{2,w,1}^A$ by removing the puncturing on K_i 's (and correspondingly change G'' back to G' , and E'' and E' back to E). Notice that this hybrid is now exactly the same as **Hybrid** $_{2,w+1,0}^A$.

1. **Setup:**

- (a) For $i \in [n]$,
 - i. $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - ii. For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 - iii. $\tilde{E}_i = i\mathcal{O}(1^\lambda, E_i[\text{pk}_i^0, \text{pk}_i^1, K_i])$.
 - iv. $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.

- (b) **Compute Values for w :**
- i. ~~Parse $w = (d_i^0, d_i^1)_{i \in [n]}$.~~
 - ii. ~~For $i \in [n]$, $\alpha \in \{0, 1\}$, $x_i^\alpha = \text{PKE.Dec}(\text{sk}_i^\alpha, d_i^\alpha)$.~~
 - iii. ~~Define $A_w = \{i \in [n] : x_i^0 = x_i^1\}$.~~
 - iv. ~~For $i \in A_w$, $u_i \leftarrow \text{PRF1.Eval}(K_i, (d_i^0, d_i^1))$.~~
 - v. ~~For $i \in [n]$, $K_i[d_i^0, d_i^1] = \text{PRF1.Punc}(K_i, (d_i^0, d_i^1))$.~~
- (c) **Compute:**
- i. ~~For $i \in A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E'_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1, u_i])$.~~
 - ii. ~~For $i \in [n] \setminus A_w$, $\tilde{E}_i = i\mathcal{O}(1^\lambda, E''_i[\text{pk}_i^0, \text{pk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1])$.~~

2. $\mathcal{A}$ may make any number of the following queries in any order.

• **Function Query:**

- (a) $\mathcal{A}$ outputs a function f .
- (b) $K_f \leftarrow \text{PRF2.Setup}(1^\lambda)$.
- (c) $\tilde{G}_f \leftarrow i\mathcal{O}(1^\lambda, \textcolor{red}{G}'[f, \{\text{sk}_i^0, \text{sk}_i^1, K_i[d_i^0, d_i^1], d_i^0, d_i^1\}_{i \in [n]}, \{\text{InjOWF}(u_i)\}_{i \in A_w}, w+1, K_f, \text{InjOWF}])$.
- (d) Send $\text{SK}_f = \tilde{G}_f$ to $\mathcal{A}$.

• **Encryption Key Query:**

- (a) $\mathcal{A}$ outputs an index $i \in [n]$.
- (b) Send EK_i to $\mathcal{A}$.

• **Challenge Message Query:**

- (a) $\mathcal{A}$ outputs $(X_j^0, X_j^1) = ((x_{j,1}^0, \dots, x_{j,n}^0), (x_{j,1}^1, \dots, x_{j,n}^1))$.
- (b) For $i \in [n]$,
 - i. $r_{j,i}^0, r_{j,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_{j,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{j,i}^0; r_{j,i}^0)$.
 - iii. $c_{j,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{j,i}^1; r_{j,i}^1)$.
 - iv. $z_{j,i} = \text{PRF1.Eval}(K_i, (c_{j,i}^0, c_{j,i}^1))$.
 - v. $\text{CT}_{j,i} = (c_{j,i}^0, c_{j,i}^1, z_{j,i})$.
- (c) Send $\{\text{CT}_{j,i}\}_{i \in [n]}$ to $\mathcal{A}$.

3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

Hybrid₃^A(1^λ): Now we remove sk_i^0 and w from G' . This step follows by $i\mathcal{O}$.

1. **Setup:**

- (a) For $i \in [n]$,
 - i. $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - ii. For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 - iii. $\tilde{E}_i = i\mathcal{O}(1^\lambda, E_i[\text{pk}_i^0, \text{pk}_i^1, K_i])$.
 - iv. $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.

2. $\mathcal{A}$ may make any number of the following queries in any order.

• **Function Query:**

- (a) $\mathcal{A}$ outputs a function f .
- (b) $K_f \leftarrow \text{PRF2.Setup}(1^\lambda)$.
- (c) $\tilde{G}_f \leftarrow i\mathcal{O}(1^\lambda, G^\dagger[f, \{\text{sk}_i^0, \text{sk}_i^1, K_i\}_{i \in [n]}, w+1, K_f, \text{InjOWF}])$.
- (d) Send $\text{SK}_f = \tilde{G}_f$ to $\mathcal{A}$.
- **Encryption Key Query:**
 - (a) $\mathcal{A}$ outputs an index $i \in [n]$.
 - (b) Send EK_i to $\mathcal{A}$.
- **Challenge Message Query:**
 - (a) $\mathcal{A}$ outputs $(X_j^0, X_j^1) = ((x_{j,1}^0, \dots, x_{j,n}^0), (x_{j,1}^1, \dots, x_{j,n}^1))$.
 - (b) For $i \in [n]$,
 - i. $r_{j,i}^0, r_{j,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_{j,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{j,i}^0; r_{j,i}^0)$.
 - iii. $c_{j,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{j,i}^1; r_{j,i}^1)$.
 - iv. $z_{j,i} = \text{PRF1.Eval}(K_i, (c_{j,i}^0, c_{j,i}^1))$.
 - v. $\text{CT}_{j,i} = (c_{j,i}^0, c_{j,i}^1, z_{j,i})$.
 - (c) Send $\{\text{CT}_{j,i}\}_{i \in [n]}$ to $\mathcal{A}$.

3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

(Difference from G highlighted)

$G^\dagger[f, \{\text{sk}_i^1, K_i\}_{i \in [n]}, K_f, \text{InjOWF}]((c_i^0, c_i^1, z_i)_{i \in [n]}):$

1. For $i \in [n]$,
 - (a) If $\text{InjOWF}(z_i) \neq \text{InjOWF}(\text{PRF1.Eval}(K_i, c_i^0, c_i^1))$, output $\perp$.
 - (b) $x_i = \text{PKE.Dec}(\text{sk}_i^1, c_i^1)$.
2. $r_f = \text{PRF2.Eval}(K_f, (c_i^0, c_i^1)_{i \in [n]})$.
3. Output $y = f(x_1, x_2, \dots, x_n; r_f)$.

Hybrid₄^A(1^λ): Now we change $c_{j,i}^0$ to an encryption of $x_{j,i}^1$. This follows by PKE security.

1. **Setup:**

- (a) For $i \in [n]$,
 - i. $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - ii. For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 - iii. $\tilde{E}_i = i\mathcal{O}(1^\lambda, E_i[\text{pk}_i^0, \text{pk}_i^1, K_i])$.
 - iv. $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.

2. $\mathcal{A}$ may make any number of the following queries in any order.

- **Function Query:**
 - (a) $\mathcal{A}$ outputs a function f .
 - (b) $K_f \leftarrow \text{PRF2.Setup}(1^\lambda)$.

- (c) $\tilde{G}_f \leftarrow i\mathcal{O}(1^\lambda, G^\dagger[f, \{\text{sk}_i^1, \textcolor{red}{K}_i\}_{i \in [n]}, K_f, \text{InjOWF}])$.
- (d) Send $\text{SK}_f = \tilde{G}_f$ to $\mathcal{A}$.
- **Encryption Key Query:**
 - (a) $\mathcal{A}$ outputs an index $i \in [n]$.
 - (b) Send EK_i to $\mathcal{A}$.
- **Challenge Message Query:**
 - (a) $\mathcal{A}$ outputs $(X_j^0, X_j^1) = ((x_{j,1}^0, \dots, x_{j,n}^0), (x_{j,1}^1, \dots, x_{j,n}^1))$.
 - (b) For $i \in [n]$,
 - i. $r_{j,i}^0, r_{j,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - ii. $\textcolor{red}{c}_{j,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{j,i}^1; r_{j,i}^0)$.
 - iii. $c_{j,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{j,i}^1; r_{j,i}^1)$.
 - iv. $z_{j,i} = \text{PRF1.Eval}(K_i, (c_{j,i}^0, c_{j,i}^1))$.
 - v. $\text{CT}_{j,i} = (c_{j,i}^0, c_{j,i}^1, z_{j,i})$.
 - (c) Send $\{\text{CT}_{j,i}\}_{i \in [n]}$ to $\mathcal{A}$.

3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

Notice that now we have successfully switched $c_{j,i}^0$ and $c_{j,i}^1$ from encrypting $x_{j,i}^0$ to encrypting $x_{j,i}^1$. This is almost the real experiment with $b = 1$, apart from the fact that we are using $G^\dagger$ instead of G in answering the function queries. The remaining hybrids will follow a similar process as we've been doing to change $G^\dagger$ back to G .

Proof of Hybrid Arguments

Lemma 6.2. *If PKE is IND-CPA secure, then no PPT adversary can distinguish between Hybrid_0^A and Hybrid_1^A with non-negligible probability.*

Proof. Note that the only difference between the two hybrids is that $c_{j,i}^1$ encrypts $x_{j,i}^0$ in Hybrid_0^A and $x_{j,i}^1$ in Hybrid_1^A . By the CPA security of PKE, without the private key, no PPT adversary should be able to tell which message is encrypted (and hence distinguish between the two hybrids). $\square$

Lemma 6.3. *If $i\mathcal{O}$ is an indistinguishability obfuscation, then no PPT adversary can distinguish between Hybrid_1^A and $\text{Hybrid}_{2,0,0}^A$ with non-negligible probability.*

Proof. Here we change from $i\mathcal{O}$ of the program G to $i\mathcal{O}$ of the program G' . In order to invoke the security of $i\mathcal{O}$, we need to show that G and G' are functionally equivalent. Note that for $w = 0$, we always have $(c_i^0, c_i^1) \geq w$. And hence G' always behaves exactly the same as G by decrypting c_i^0 . $\square$

Lemma 6.4. *If $i\mathcal{O}$ has $(1, 2^{-3ns-\lambda_{i\mathcal{O}}})$ weak extractability, then for any distinguisher $\mathcal{D}$, we have $|\Pr[\mathcal{D}(\text{Hybrid}_{2,w,0}^A) - \mathcal{D}(\text{Hybrid}_{2,w,1}^A)]| < O((n + p(\lambda_{i\mathcal{O}})) \cdot 2^{-3ns-\lambda_{i\mathcal{O}}})$ for some polynomial p .*

Proof. Again here we wish to invoke $i\mathcal{O}$ security. Notice that here we are changing the program E_i to E' and E_i'' , and G' to G'' . We will argue about their functional equivalence.

- E'_i : Notice if $(c_i^0, c_i^1) = (d_i^0, d_i^1)$, E'_i outputs $u_i = \text{PRF1.Eval}(K_i, (d_i^0, d_i^1))$, which is exactly the output of E_i . If $(c_i^0, c_i^1) \neq (d_i^0, d_i^1)$, then E'_i can successfully use the PRF key punctured at (d_i^0, d_i^1) to evaluate on the point (c_i^0, c_i^1) , yielding $\text{PRF1.Eval}(K_i[d_i^0, d_i^1], (c_i^0, c_i^1)) = \text{PRF1.Eval}(K_i, (c_i^0, c_i^1))$, which is the same as the output of E_i .
- E''_i : Notice that E''_i are only handed out for $i \notin A_w$, meaning $x_i^0 \neq x_i^1$. Under this case both E_i and E''_i would output $\perp$ and hence are equivalent.
- G'' : Notice that the only differences are the cases where G' and G'' will output $\perp$. G' outputs $\perp$ if $\text{InjOWF}(z_i) \neq \text{InjOWF}(\text{PRF1.Eval}(K_i, (c_i^0, c_i^1)))$. This correspond to the two cases in G'' . If $(c_i^0, c_i^1) \neq (d_i^0, d_i^1)$, G'' uses the punctured key to perform the exact same check, and hence have the exact functionality. If $(c_i^0, c_i^1) = (d_i^0, d_i^1)$, then G'' will be checking $\text{InjOWF}(z_i) = \text{InjOWF}(u_i) = \text{InjOWF}(\text{PRF1.Eval}(K_i, (d_i^0, d_i^1)))$, giving us again the exact same check as in G' . Therefore G and G'' have the exact same functionality.

Therefore, all these switches reduce to $i\mathcal{O}$ security. Since we make n switches for E and $p(\lambda_{i\mathcal{O}})$ switches for G , the distinguisher's advantage is upper bounded by $(n + p(\lambda_{i\mathcal{O}})) \cdot 2^{-3ns - \lambda_{i\mathcal{O}}}$. $\square$

Lemma 6.5. *If PRF1 has security $2^{-\lambda_{\text{PRF1}}^{\text{cPRF1}}}$ with $\lambda_{\text{PRF1}} \geq (2ns + \lambda_{i\mathcal{O}})^{1/\text{cPRF1}}$, then for any distinguisher $\mathcal{D}$, $|\Pr[\mathcal{D}(\text{Hybrid}_{2,w,1}^A) - \mathcal{D}(\text{Hybrid}_{2,w,2}^A)]| < O(n \cdot 2^{-2ns - \lambda_{i\mathcal{O}}})$.*

Proof. This step follows directly from the security of the puncturable PRF. Since we are only giving out the key punctured at (d_i^0, d_i^1) , by puncturable PRF security we can replace the PRF output on that point with a uniformly random value. Here we make $O(n)$ of such switches, and the sub-exponential security of PRF1 guarantees that each switch can be spotted with advantage at most $2^{-\lambda_{\text{PRF1}}^{\text{cPRF1}}} = 2^{-2ns - \lambda_{i\mathcal{O}}}$, therefore, the distinguisher's advantage is upper bounded by $O(n \cdot 2^{-2ns - \lambda_{i\mathcal{O}}})$. $\square$

Lemma 6.6. *If $i\mathcal{O}$ has $(1, 2^{-3ns - \lambda_{i\mathcal{O}}})$ weak extractability, then for any distinguisher $\mathcal{D}$, we have $|\Pr[\mathcal{D}(\text{Hybrid}_{2,w,2}^A) - \mathcal{D}(\text{Hybrid}_{2,w,3}^A)]| < O(p(\lambda_{i\mathcal{O}}) \cdot 2^{-3ns - \lambda_{i\mathcal{O}}})$ for some polynomial p .*

Proof. Again, here we show functional equivalence to invoke $i\mathcal{O}$ security. Note that the only change is the behavior of the program G'' for cases where $(c_i^0, c_i^1) = w$.

Originally, in $\text{Hybrid}_{2,w,2}^A$, G'' will compute $x_i = \text{PKE.Dec}(\text{sk}_i^0, c_i^0)$ for all $i \in [n]$ and output $y = f(x_1, x_2, \dots, x_n; r)$ with $r = \text{PRF2.Eval}(K_f, (c_i^0, c_i^1))$.

But now in $\text{Hybrid}_{2,w,3}^A$, G''' will compute $r^* = \text{PRF2.Eval}(K_f, w) = r$, $x_i^0 = \text{PKE.Dec}(\text{sk}_i^0, d_i^0) = \text{PKE.Dec}(\text{sk}_i^0, c_i^0)$ for all i and eventually $y^* = f(x_1^0, x_2^0, \dots, x_n^0; r^*)$. By inspection, indeed we have $y = y^*$.

Notice here the number of switches we make is equal to the number of function queries, which is bounded by some polynomial p . Therefore, the distinguisher's advantage is bounded by $O(\text{poly}(\lambda_{i\mathcal{O}}) \cdot 2^{-3ns - \lambda_{i\mathcal{O}}})$. $\square$

Lemma 6.7. *If PRF2 has security $2^{-\lambda_{\text{PRF2}}^{\text{cPRF2}}}$ with $\lambda_{\text{PRF2}} \geq (2ns + \lambda_{i\mathcal{O}})^{1/\text{cPRF2}}$, then for any distinguisher $\mathcal{D}$, $|\Pr[\mathcal{D}(\text{Hybrid}_{2,w,3}^A) - \mathcal{D}(\text{Hybrid}_{2,w,4}^A)]| < O(p(\lambda_{i\mathcal{O}}) \cdot 2^{-2ns - \lambda_{i\mathcal{O}}})$ for some polynomial p .*

Proof. Here we are switching PRF2 output on point w with random with the key also punctured on w . We are making $p(\lambda_{i\mathcal{O}})$ switches, so the distinguisher's advantage is upper bounded by $O(p(\lambda_{i\mathcal{O}}) \cdot 2^{-2ns - \lambda_{i\mathcal{O}}})$. $\square$

Lemma 6.8. *If $i\mathcal{O}$ has $(1, 2^{-3ns-\lambda_{i\mathcal{O}}})$ weak extractability, PRFs and InjOWF satisfy the requirements outlined in the Parameters section (section 6.1), then for any distinguisher $\mathcal{D}$, we have $|\Pr[\mathcal{D}(\mathbf{Hybrid}_{2,w,4}^A) - \mathcal{D}(\mathbf{Hybrid}_{2,w,5}^A)]| < O(p(\lambda_{i\mathcal{O}}) \cdot 2^{-2ns-\lambda_{i\mathcal{O}}})$ for some polynomial p .*

Proof. Assume towards contradiction that there exists a distinguisher $\mathcal{D}$ that can distinguish between $\mathbf{Hybrid}_{2,w,4}^A$ and $\mathbf{Hybrid}_{2,w,5}^A$ with probability at least $2^{-2ns-\lambda_{i\mathcal{O}}}$. Note that the only difference between $\mathbf{Hybrid}_{2,w,4}^A$ and $\mathbf{Hybrid}_{2,w,5}^A$ is how y^* is computed, which is then used in the generation of the circuit $\tilde{G}_f$. Since $\tilde{G}_f$ is an $i\mathcal{O}$ circuit, we break into the following cases:

1. **Case 1:** The circuit G''' in $\mathbf{Hybrid}_{2,w,5}^A$ is functionally equivalent to the G''' circuit in $\mathbf{Hybrid}_{2,w,4}^A$.
2. **Case 2:** The two circuits above are not functionally equivalent. Notice that this means the two y^* values are different in $\mathbf{Hybrid}_{2,w,4}^A$ and $\mathbf{Hybrid}_{2,w,5}^A$. This means $f(x_1^0, x_2^0, \dots, x_n^0; r^*) \neq f(x_1^1, x_2^1, \dots, x_n^1; r^*)$.

For Case 1, this reduces directly to the $(1, 2^{-3ns-\lambda_{i\mathcal{O}}})$ weak extractability of the underlying $i\mathcal{O}$ scheme. If any distinguisher can distinguish between the two hybrids in this case, it can also win the $i\mathcal{O}$ weak extractability game, but such winning probability is bounded by $2^{-3ns-\lambda_{i\mathcal{O}}}$. So in case 1, the probability of successfully distinguishing these two hybrids is at most $2^{-3ns-\lambda_{i\mathcal{O}}}$.

Case 2 happens when we have $f(x_1^0, x_2^0, \dots, x_n^0; r^*) \neq f(x_1^1, x_2^1, \dots, x_n^1; r^*)$. But notice that by the randomized compatibility requirement, we require their output distributions to be computationally indistinguishable with a sub-exponentially small distinguishing advantage. In this case, the argument proceeds as follow:

1. The only way for the two circuits to yield potentially different outputs is when step 1(d) in G''' is executed. So we will need to have $(c_i^0, c_i^1)_{i \in [n]} = w = (d_i^0, d_i^1)_{i \in [n]}$.
2. In order to proceed to step 1(d), we must fail the check in step 1(a), because otherwise, the circuit will output $\perp$. This means $\text{InjOWF}(z_i) = \text{InjOWF}(u_i)$ for all $i \in [n]$. By security of the injective OWF, it translates to that for each i , with all but $2^{-3ns-\lambda_{i\mathcal{O}}}$ probability, $z_i = u_i$, because otherwise we would have successfully inverted the injective OWF. So union bounding over all i 's, we have that with all but $n \cdot 2^{-3ns-\lambda_{i\mathcal{O}}}$, $z_i = u_i$ for all i .
3. Then, we can invoke the randomized compatibility requirement. In order for a distinguisher to distinguish between these two hybrids, it must now distinguish between $f(x_1^0, x_2^0, \dots, x_n^0; r^*)$ and $f(x_1^1, x_2^1, \dots, x_n^1; r^*)$. But, by randomized compatibility, for a PPT adversary $\mathcal{A}$, the probability that it can distinguish between the random outputs of $f(x_1^0, x_2^0, \dots, x_n^0)$ and $f(x_1^1, x_2^1, \dots, x_n^1)$ is at most $2^{-2ns-\lambda_{i\mathcal{O}}}$. This means the distinguisher's success probability is upper bounded by the adversary's success probability of $2^{-2ns-\lambda_{i\mathcal{O}}}$.

Therefore, the distinguisher's success probability in Case 2 is upper bounded by the probability of inverting the injective OWF and the probability of breaking the randomized compatibility, and is hence at most $2^{-2ns-\lambda_{i\mathcal{O}}}$ as desired.

We have shown that in both cases, the distinguisher succeeds with probability at most $2^{-2ns-\lambda_{i\mathcal{O}}}$, which concludes the proof. $\square$

The rest of the hybrid arguments follow analogously from the lemmas above.

Notice that to go from **Hybrid**₁^A to **Hybrid**₃^A, we have to go through an exponential $\Theta(2^{2ns})$ number of sub-hybrids. But with the lemmas above, we still have

$$\begin{aligned} |\Pr[\mathcal{D}(\mathbf{Hybrid}_1^A) - \mathcal{D}(\mathbf{Hybrid}_3^A)]| &= |\Pr[\mathcal{D}(\mathbf{Hybrid}_{2,0,0}^A) - \mathcal{D}(\mathbf{Hybrid}_{2,n+1,0}^A)]| \\ &< 2^{2ns} \left(2((n + p(\lambda)) \cdot 2^{-3ns-\lambda} + n \cdot 2^{-2ns-\lambda} \right. \\ &\quad \left. + p(\lambda) \cdot 2^{-3ns-\lambda} + p(\lambda) \cdot 2^{-2ns-\lambda}) + p(\lambda) \cdot 2^{-2ns-\lambda} \right) \\ &< q(\lambda) \cdot 2^{2ns} \cdot 2^{-2ns-\lambda} = \text{negl}(\lambda), \end{aligned}$$

where q is some polynomial and $\lambda = \lambda_{i\mathcal{O}}$.

Therefore, we have shown a sequence of polynomial number of hybrids (not counting sub-hybrids; above we have shown the exponential number of sub-hybrids with sub-exponential distinguisher advantages between each pair yielding a negligible advantage between the two larger hybrids), where no PPT distinguisher can distinguish adjacent ones with non-negligible probability. Hence we have shown that Construction 2 is secure against malicious decryptors.

6.4 Security against malicious encryptors

Lastly, we prove its security against malicious encryptors.

Theorem 6.9. *If $i\mathcal{O}$ is correct, and PRF2 is a secure puncturable PRF, then Construction 2 is secure against malicious encryptors.*

Proof. We prove this through a simple sequence of hybrids.

Sequence of Hybrids

Hybrid₀^A(1^λ): Real world experiment with $b = 0$.

1. Setup:

- (a) For $i \in [n]$,
 - i. $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - ii. For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 - iii. $\tilde{E}_i = i\mathcal{O}(1^\lambda, E_i[\text{pk}_i^0, \text{pk}_i^1, K_i])$.
 - iv. $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.

2. $\mathcal{A}$ may make any number of the following queries in any order.

• Functional Key Query:

- (a) $\mathcal{A}$ outputs a function f_ℓ .
- (b) Sample $K_{f_\ell} \leftarrow \text{PRF2.Setup}(1^\lambda)$.
- (c) Compute $\tilde{G}_{f_\ell} \leftarrow i\mathcal{O}(1^\lambda, G[f_\ell, \{\text{sk}_i^0, K_i\}_{i \in [n]}, K_{f_\ell}, \text{InjOWF}])$.
- (d) Send $\text{SK}_{f_\ell} = \tilde{G}_{f_\ell}$ to $\mathcal{A}$.

• Encryption Key Query:

- (a) $\mathcal{A}$ outputs an index $i \in [n]$.
- (b) Send EK_i to $\mathcal{A}$.

• Function Store Query:

- (a) $\mathcal{A}$ submits a function g_k .
 - (b) If (g_k, sk_{g_k}) already exists in KeyStore, which was initialized as KeyStore = $\emptyset$ at the beginning of experiment, output 0.
 - (c) Sample $K_{g_k} \leftarrow \text{PRF2.Setup}(1^\lambda)$.
 - (d) Compute $\tilde{G}_{g_k} \leftarrow i\mathcal{O}(1^\lambda, G[g_k, \{sk_i^0, K_i\}_{i \in [n]}, K_{g_k}, \text{InjOWF}])$.
 - (e) Store the pair $(g_k, sk_{g_k} = \tilde{G}_{g_k})$ in KeyStore and output 1.
- **Encryption Query:**
 - (a) $\mathcal{A}$ outputs $X_t = (x_{t,1}, \dots, x_{t,n})$.
 - (b) For $i \in [n]$,
 - i. $r_{t,i}^0, r_{t,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_{t,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{t,i}; r_{t,i}^0)$.
 - iii. $c_{t,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{t,i}; r_{t,i}^1)$.
 - iv. $z_{t,i} = \tilde{E}_i(c_{t,i}^0, c_{t,i}^1, x_{t,i}, r_{t,i}^0, r_{t,i}^1)$.
 - v. $\text{CT}_{t,i} = (c_{t,i}^0, c_{t,i}^1, z_{t,i})$.
 - (c) Send $\{\text{CT}_{t,i}\}_{i \in [n]}$ to $\mathcal{A}$.
 - **Challenge Decryption Query:**
 - (a) $\mathcal{A}$ submits $\text{CT}_j = (\text{CT}_{j,1}, \dots, \text{CT}_{j,n})$.
 - (b) If $(\text{CT}_j, \{y_{g_k,j}\})$ already exists in DecStore, which is initialized as DecStore = $\emptyset$ at the beginning of the experiment, output $\{y_{g_k,j}\}$.
 - (c) For all $(g_k, sk_{g_k} = \tilde{G}_{g_k}) \in \text{KeyStore}$, compute $y_{g_k,j} = \tilde{G}_{g_k}(\text{CT}_{j,1}, \dots, \text{CT}_{j,n})$.
 - (d) Store $(\text{CT}_j, \{y_{g_k,j}\}) \in \text{DecStore}$ and output $\{y_{g_k,j}\}$.

3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

Hybrid₁^A(1^λ): Now when processing function store queries, instead of computing and storing (g_k, sk_{g_k}) , store (g_k, K_{g_k}) and later when decryption oracle uses KeyStore, calculate sk_{g_k} using the stored K_{g_k} values. This step follows from the correctness guarantee of the $i\mathcal{O}$ program.

1. **Setup:**

- (a) For $i \in [n]$,
 - i. $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - ii. For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 - iii. $\tilde{E}_i = i\mathcal{O}(1^\lambda, E_i[\text{pk}_i^0, \text{pk}_i^1, K_i])$.
 - iv. $\text{EK}_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.

2. $\mathcal{A}$ may make any number of the following queries in any order.

- **Functional Key Query:**
 - (a) $\mathcal{A}$ outputs a function f_ℓ .
 - (b) Sample $K_{f_\ell} \leftarrow \text{PRF2.Setup}(1^\lambda)$.
 - (c) Compute $\tilde{G}_{f_\ell} \leftarrow i\mathcal{O}(1^\lambda, G[f_\ell, \{sk_i^0, K_i\}_{i \in [n]}, K_{f_\ell}, \text{InjOWF}])$.
 - (d) Send $\text{SK}_{f_\ell} = \tilde{G}_{f_\ell}$ to $\mathcal{A}$.
- **Encryption Key Query:**

- (a) $\mathcal{A}$ outputs an index $i \in [n]$.
- (b) Send EK_i to $\mathcal{A}$.
- **Function Store Query:**
 - (a) $\mathcal{A}$ submits a function g_k .
 - (b) If (g_k, K_{g_k}) already exists in KeyStore, which was initialized as KeyStore = $\emptyset$ at the beginning of experiment, output 0.
 - (c) Sample $K_{g_k} \leftarrow \text{PRF2.Setup}(1^\lambda)$.
 - (d) ~~Compute $\tilde{G}_{g_k} \leftarrow i\mathcal{O}(1^\lambda, G[g_k, \{\text{sk}_i^0, K_i\}_{i \in [n]}, K_{g_k}, \text{InjOWF}])$.~~
 - (e) Store the pair (g_k, K_{g_k}) in KeyStore and output 1.
- **Encryption Query:**
 - (a) $\mathcal{A}$ outputs $X_t = (x_{t,1}, \dots, x_{t,n})$.
 - (b) For $i \in [n]$,
 - i. $r_{t,i}^0, r_{t,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_{t,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{t,i}; r_{t,i}^0)$.
 - iii. $c_{t,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{t,i}; r_{t,i}^1)$.
 - iv. $z_{t,i} = \tilde{E}_i(c_{t,i}^0, c_{t,i}^1, x_{t,i}, r_{t,i}^0, r_{t,i}^1)$.
 - v. $\text{CT}_{t,i} = (c_{t,i}^0, c_{t,i}^1, z_{t,i})$.
 - (c) Send $\{\text{CT}_{t,i}\}_{i \in [n]}$ to $\mathcal{A}$.
- **Challenge Decryption Query:**
 - (a) $\mathcal{A}$ submits $\text{CT}_j = (\text{CT}_{j,1}, \dots, \text{CT}_{j,n})$.
 - (b) If $(\text{CT}_j, \{y_{g_k,j}\})$ already exists in DecStore, which is initialized as DecStore = $\emptyset$ at the beginning of the experiment, output $\{y_{g_k,j}\}$.
 - (c) For all $(g_k, K_{g_k}) \in \text{KeyStore}$:
 - i. ~~Compute $\tilde{G}_{g_k} \leftarrow i\mathcal{O}(1^\lambda, G[g_k, \{\text{sk}_i^0, K_i\}_{i \in [n]}, K_{g_k}, \text{InjOWF}])$.~~
 - ii. Compute $y_{g_k,j} = \tilde{G}_{g_k}(\text{CT}_{j,1}, \dots, \text{CT}_{j,n})$.
 - (d) Store $(\text{CT}_j, \{y_{g_k,j}\}) \in \text{DecStore}$ and output $\{y_{g_k,j}\}$.

3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

Hybrid₂^A(1^λ): Now we change the PRF2 keys to be lazily and uniformly sampled in the Decryption oracle. This step is a syntactical change.

1. **Setup:**

- (a) For $i \in [n]$,
 - i. $K_i \leftarrow \text{PRF1.Setup}(1^\lambda)$.
 - ii. For $\alpha \in \{0, 1\}$, $(\text{pk}_i^\alpha, \text{sk}_i^\alpha) \leftarrow \text{PKE.Setup}(1^\lambda)$.
 - iii. $\tilde{E}_i = i\mathcal{O}(1^\lambda, E_i[\text{pk}_i^0, \text{pk}_i^1, K_i])$.
 - iv. $EK_i = (\text{pk}_i^0, \text{pk}_i^1, \tilde{E}_i)$.

2. $\mathcal{A}$ may make any number of the following queries in any order.

• **Functional Key Query:**

- (a) $\mathcal{A}$ outputs a function f_ℓ .

- (b) Sample $K_{f_\ell} \leftarrow \text{PRF2.Setup}(1^\lambda)$.
- (c) Compute $\tilde{G}_{f_\ell} \leftarrow i\mathcal{O}(1^\lambda, G[f_\ell, \{\text{sk}_i^0, K_i\}_{i \in [n]}, K_{f_\ell}, \text{InjOWF}])$.
- (d) Send $\text{SK}_{f_\ell} = \tilde{G}_{f_\ell}$ to $\mathcal{A}$.
- **Encryption Key Query:**
 - (a) $\mathcal{A}$ outputs an index $i \in [n]$.
 - (b) Send EK_i to $\mathcal{A}$.
- **Function Store Query:**
 - (a) $\mathcal{A}$ submits a function g_k .
 - (b) If (g_k, K_{g_k}) already exists in **KeyStore**, which was initialized as **KeyStore** = $\emptyset$ at the beginning of experiment, output 0.
 - (c) **Sample $K_{g_k} \leftarrow \text{PRF2.Setup}(1^\lambda)$.**
 - (d) Store the pair $(g_k, \perp)$ in **KeyStore** and output 1.
- **Encryption Query:**
 - (a) $\mathcal{A}$ outputs $X_t = (x_{t,1}, \dots, x_{t,n})$.
 - (b) For $i \in [n]$,
 - i. $r_{t,i}^0, r_{t,i}^1 \leftarrow \{0, 1\}^\lambda$.
 - ii. $c_{t,i}^0 = \text{PKE.Enc}(\text{pk}_i^0, x_{t,i}; r_{t,i}^0)$.
 - iii. $c_{t,i}^1 = \text{PKE.Enc}(\text{pk}_i^1, x_{t,i}; r_{t,i}^1)$.
 - iv. $z_{t,i} = \tilde{E}_i(c_{t,i}^0, c_{t,i}^1, x_{t,i}, r_{t,i}^0, r_{t,i}^1)$.
 - v. $\text{CT}_{t,i} = (c_{t,i}^0, c_{t,i}^1, z_{t,i})$.
 - (c) Send $\{\text{CT}_{t,i}\}_{i \in [n]}$ to $\mathcal{A}$.
- **Challenge Decryption Query:**
 - (a) $\mathcal{A}$ submits $\text{CT}_j = (\text{CT}_{j,1}, \dots, \text{CT}_{j,n})$.
 - (b) If $(\text{CT}_j, \{y_{g_k,j}\})$ already exists in **DecStore**, which is initialized as **DecStore** = $\emptyset$ at the beginning of the experiment, output $\{y_{g_k,j}\}$.
 - (c) For all $(g_k, K_{g_k}) \in \text{KeyStore}$:
 - i. **If $K_{g_k} = \perp$, sample $K_{g_k} \leftarrow \{0, 1\}^\lambda$ and update (g_k, K_{g_k}) in **KeyStore**.**
 - ii. Compute $\tilde{G}_{g_k} \leftarrow i\mathcal{O}(1^\lambda, G[g_k, \{\text{sk}_i^0, K_i\}_{i \in [n]}, K_{g_k}, \text{InjOWF}])$.
 - iii. Compute $y_{g_k,j} = \tilde{G}_{g_k}(\text{CT}_{j,1}, \dots, \text{CT}_{j,n})$.
 - (d) Store $(\text{CT}_j, \{y_{g_k,j}\}) \in \text{DecStore}$ and output $\{y_{g_k,j}\}$.

3. **Output:** $\mathcal{A}$ outputs a bit b' . Output b' if the queries are compatible, and 0 otherwise.

Notice now when we answer decryption queries, we effectively sample a uniformly random PRF2 key K_{g_k} , and evaluate it on the ciphertext to obtain the randomness. Notice that a malicious encryptor cannot affect our uniform choice of PRF2 key in any way, therefore by PRF2 security, the PRF2 output will always be close to uniformly random. Consequently, the functions will be evaluated using a uniform randomness that the malicious encryptor cannot tamper with. $\square$

7 Acknowledgements

This research was supported in part from a Simons Investigator Award, DARPA SIEVE award, NTT Research, NSF grant2333935, BSF grant 2022370, a Xerox Faculty Research Award, a Google Faculty Research Award, an Okawa Foundation Research Grant, and the Symantec Chair of Computer Science. This material is based upon work supported by the Defense Advanced Research Projects Agency through Award HR00112020024.

8 References

- [ABF⁺13] Joël Alwen, Manuel Barbosa, Pooya Farshim, Rosario Gennaro, S. Dov Gordon, Stefano Tessaro, and David A. Wilson. On the relationship between functional encryption, obfuscation, and fully homomorphic encryption. In Martijn Stam, editor, *14th IMA International Conference on Cryptography and Coding*, volume 8308 of *LNCS*, pages 65–84. Springer, Berlin, Heidelberg, December 2013.
- [ABG19] Michel Abdalla, Fabrice Benhamouda, and Romain Gay. From single-input to multi-client inner-product functional encryption. In Steven D. Galbraith and Shiho Moriai, editors, *ASIACRYPT 2019, Part III*, volume 11923 of *LNCS*, pages 552–582. Springer, Cham, December 2019.
- [ABKW19] Michel Abdalla, Fabrice Benhamouda, Markulf Kohlweiss, and Hendrik Waldner. Decentralizing inner-product functional encryption. In Dongdai Lin and Kazue Sako, editors, *PKC 2019, Part II*, volume 11443 of *LNCS*, pages 128–157. Springer, Cham, April 2019.
- [ABM⁺20] Michel Abdalla, Florian Bourse, Hugo Marival, David Pointcheval, Azam Soleimanian, and Hendrik Waldner. Multi-client inner-product functional encryption in the random-oracle model. In Clemente Galdi and Vladimir Kolesnikov, editors, *SCN 20*, volume 12238 of *LNCS*, pages 525–545. Springer, Cham, September 2020.
- [ACF⁺18] Michel Abdalla, Dario Catalano, Dario Fiore, Romain Gay, and Bogdan Ursu. Multi-input functional encryption for inner products: Function-hiding realizations and constructions without pairings. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 597–627. Springer, Cham, August 2018.
- [ACF⁺20] Shweta Agrawal, Michael Clear, Ophir Frieder, Sanjam Garg, Adam O’Neill, and Justin Thaler. Ad hoc multi-input functional encryption. In Thomas Vidick, editor, *ITCS 2020*, volume 151, pages 40:1–40:41. LIPIcs, January 2020.
- [ACGU20] Michel Abdalla, Dario Catalano, Romain Gay, and Bogdan Ursu. Inner-product functional encryption with fine-grained access control. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part III*, volume 12493 of *LNCS*, pages 467–497. Springer, Cham, December 2020.
- [AFS21] Miguel Ambrona, Dario Fiore, and Claudio Soriente. Controlled functional encryption revisited: Multi-authority extensions and efficient schemes for quadratic functions. *PoPETs*, 2021(1):21–42, January 2021.
- [AGRW17] Michel Abdalla, Romain Gay, Mariana Raykova, and Hoeteck Wee. Multi-input inner-product functional encryption from pairings. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part I*, volume 10210 of *LNCS*, pages 601–626. Springer, Cham, April / May 2017.
- [AGT21] Shweta Agrawal, Rishab Goyal, and Junichi Tomida. Multi-input quadratic functional encryption from pairings. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 208–238, Virtual Event, August 2021. Springer, Cham.

- [AGT22] Shweta Agrawal, Rishab Goyal, and Junichi Tomida. Multi-input quadratic functional encryption: Stronger security, broader functionality. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part I*, volume 13747 of *LNCS*, pages 711–740. Springer, Cham, November 2022.
- [AGVW13] Shweta Agrawal, Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Functional encryption: New perspectives and lower bounds. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 500–518. Springer, Berlin, Heidelberg, August 2013.
- [AGW20] Michel Abdalla, Junqing Gong, and Hoeteck Wee. Functional encryption for attribute-weighted sums from k -Lin. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part I*, volume 12170 of *LNCS*, pages 685–716. Springer, Cham, August 2020.
- [AJ15] Prabhanjan Ananth and Abhishek Jain. Indistinguishability obfuscation from compact functional encryption. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part I*, volume 9215 of *LNCS*, pages 308–326. Springer, Berlin, Heidelberg, August 2015.
- [AJS18] Prabhanjan Ananth, Aayush Jain, and Amit Sahai. Indistinguishability obfuscation without multilinear maps: iO from LWE, bilinear maps, and weak pseudorandomness. Cryptology ePrint Archive, Report 2018/615, 2018.
- [ARYY23] Shweta Agrawal, Mélissa Rossi, Anshu Yadav, and Shota Yamada. Constant input attribute based (and predicate) encryption from evasive and tensor LWE. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 532–564. Springer, Cham, August 2023.
- [ATY23] Shweta Agrawal, Junichi Tomida, and Anshu Yadav. Attribute-based multi-input FE (and more) for attribute-weighted sums. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 464–497. Springer, Cham, August 2023.
- [AW17] Shashank Agrawal and David J. Wu. Functional encryption: Deterministic to randomized functions from simple assumptions. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part II*, volume 10211 of *LNCS*, pages 30–61. Springer, Cham, April / May 2017.
- [AYY22] Shweta Agrawal, Anshu Yadav, and Shota Yamada. Multi-input attribute based encryption and predicate encryption. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 590–621. Springer, Cham, August 2022.
- [BCP14] Elette Boyle, Kai-Min Chung, and Rafael Pass. On extractability obfuscation. In Yehuda Lindell, editor, *TCC 2014*, volume 8349 of *LNCS*, pages 52–73. Springer, Berlin, Heidelberg, February 2014.
- [BGI⁺01] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil P. Vadhan, and Ke Yang. On the (im)possibility of obfuscating programs. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 1–18. Springer, Berlin, Heidelberg, August 2001.

- [BGI14] Elette Boyle, Shafi Goldwasser, and Ioana Ivan. Functional signatures and pseudorandom functions. In Hugo Krawczyk, editor, *PKC 2014*, volume 8383 of *LNCS*, pages 501–519. Springer, Berlin, Heidelberg, March 2014.
- [BGJS15] Saikrishna Badrinarayanan, Divya Gupta, Abhishek Jain, and Amit Sahai. Multi-input functional encryption for unbounded arity functions. In Tetsu Iwata and Jung Hee Cheon, editors, *ASIACRYPT 2015, Part I*, volume 9452 of *LNCS*, pages 27–51. Springer, Berlin, Heidelberg, November / December 2015.
- [BGJS16] Saikrishna Badrinarayanan, Vipul Goyal, Aayush Jain, and Amit Sahai. Verifiable functional encryption. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 557–587. Springer, Berlin, Heidelberg, December 2016.
- [BJK⁺18] Zvika Brakerski, Aayush Jain, Ilan Komargodski, Alain Passelègue, and Daniel Wichs. Non-trivial witness encryption and null-iO from standard assumptions. In Dario Catalano and Roberto De Prisco, editors, *SCN 18*, volume 11035 of *LNCS*, pages 425–441. Springer, Cham, September 2018.
- [BKMT21] Pramod Bhatotia, Markulf Kohlweiss, Lorenzo Martinico, and Yiannis Tselekounis. Steel: Composable hardware-based stateful and randomised functional encryption. In Juan Garay, editor, *PKC 2021, Part II*, volume 12711 of *LNCS*, pages 709–736. Springer, Cham, May 2021.
- [BKS16] Zvika Brakerski, Ilan Komargodski, and Gil Segev. Multi-input functional encryption in the private-key setting: Stronger security from weaker assumptions. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 852–880. Springer, Berlin, Heidelberg, May 2016.
- [BKSW18] Saikrishna Badrinarayanan, Dakshita Khurana, Amit Sahai, and Brent Waters. Upgrading to functional encryption. In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018, Part I*, volume 11239 of *LNCS*, pages 629–658. Springer, Cham, November 2018.
- [BLR⁺15] Dan Boneh, Kevin Lewi, Mariana Raykova, Amit Sahai, Mark Zhandry, and Joe Zimmerman. Semantically secure order-revealing encryption: Multi-input functional encryption without obfuscation. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 563–594. Springer, Berlin, Heidelberg, April 2015.
- [BSW11] Dan Boneh, Amit Sahai, and Brent Waters. Functional encryption: Definitions and challenges. In Yuval Ishai, editor, *TCC 2011*, volume 6597 of *LNCS*, pages 253–273. Springer, Berlin, Heidelberg, March 2011.
- [BW13] Dan Boneh and Brent Waters. Constrained pseudorandom functions and their applications. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part II*, volume 8270 of *LNCS*, pages 280–300. Springer, Berlin, Heidelberg, December 2013.
- [BZ16] Mark Bun and Mark Zhandry. Order-revealing encryption and the hardness of private learning. In Eyal Kushilevitz and Tal Malkin, editors, *TCC 2016-A, Part I*, volume 9562 of *LNCS*, pages 176–206. Springer, Berlin, Heidelberg, January 2016.

- [CDG⁺18a] J  r  my Chotard, Edouard Dufour Sans, Romain Gay, Duong Hieu Phan, and David Pointcheval. Decentralized multi-client functional encryption for inner product. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part II*, volume 11273 of *LNCS*, pages 703–732. Springer, Cham, December 2018.
- [CDG⁺18b] J  r  my Chotard, Edouard Dufour Sans, Romain Gay, Duong Hieu Phan, and David Pointcheval. Multi-client functional encryption with repetition for inner product. Cryptology ePrint Archive, Report 2018/1021, 2018.
- [CDSG⁺20] J  r  my Chotard, Edouard Dufour-Sans, Romain Gay, Duong Hieu Phan, and David Pointcheval. Dynamic decentralized functional encryption. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part I*, volume 12170 of *LNCS*, pages 747–775. Springer, Cham, August 2020.
- [CGJS15] Nishanth Chandran, Vipul Goyal, Aayush Jain, and Amit Sahai. Functional encryption: Decentralised and delegatable. Cryptology ePrint Archive, Report 2015/1017, 2015.
- [CLWW16] Nathan Chenette, Kevin Lewi, Stephen A. Weis, and David J. Wu. Practical order-revealing encryption with limited leakage. In Thomas Peyrin, editor, *FSE 2016*, volume 9783 of *LNCS*, pages 474–493. Springer, Berlin, Heidelberg, March 2016.
- [CMR17] Brent Carmer, Alex J. Malozemoff, and Mariana Raykova. 5Gen-C: Multi-input functional encryption and program obfuscation for arithmetic circuits. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 747–764. ACM Press, October / November 2017.
- [DI13] Angelo De Caro and Vincenzo Iovino. On the power of rewinding simulators in functional encryption. Cryptology ePrint Archive, Report 2013/752, 2013.
- [DIJ⁺13] Angelo De Caro, Vincenzo Iovino, Abhishek Jain, Adam O’Neill, Omer Paneth, and Giuseppe Persiano. On the achievability of simulation-based security for functional encryption. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 519–535. Springer, Berlin, Heidelberg, August 2013.
- [DIPV17] Yvo Desmedt, Vincenzo Iovino, Giuseppe Persiano, and Ivan Visconti. Controlled homomorphic encryption: Definition and construction. In Michael Brenner, Kurt Rohloff, Joseph Bonneau, Andrew Miller, Peter Y. A. Ryan, Vanessa Teague, Andrea Bracciali, Massimiliano Sala, Federico Pintore, and Markus Jakobsson, editors, *FC 2017 Workshops*, volume 10323 of *LNCS*, pages 107–129. Springer, Cham, April 2017.
- [DOT18] Pratish Datta, Tatsuaki Okamoto, and Junichi Tomida. Full-hiding (unbounded) multi-input inner product functional encryption from the k -Linear assumption. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part II*, volume 10770 of *LNCS*, pages 245–277. Springer, Cham, March 2018.
- [FFMV23] Danilo Francati, Daniele Friolo, Giulio Malavolta, and Daniele Venturi. Multi-key and multi-input predicate encryption from learning with errors. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part III*, volume 14006 of *LNCS*, pages 573–604. Springer, Cham, April 2023.

- [FFMV24] Danilo Francati, Daniele Friolo, Giulio Malavolta, and Daniele Venturi. Multi-key and multi-input predicate encryption (for conjunctions) from learning with errors. *Journal of Cryptology*, 37(3):24, July 2024.
- [GGG⁺14] Shafi Goldwasser, S. Dov Gordon, Vipul Goyal, Abhishek Jain, Jonathan Katz, Feng-Hao Liu, Amit Sahai, Elaine Shi, and Hong-Sheng Zhou. Multi-input functional encryption. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 578–602. Springer, Berlin, Heidelberg, May 2014.
- [GGH⁺13] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. In *54th FOCS*, pages 40–49. IEEE Computer Society Press, October 2013.
- [GGHW14] Sanjam Garg, Craig Gentry, Shai Halevi, and Daniel Wichs. On the implausibility of differing-inputs obfuscation and extractable witness encryption with auxiliary input. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 518–535. Springer, Berlin, Heidelberg, August 2014.
- [GGHZ16] Sanjam Garg, Craig Gentry, Shai Halevi, and Mark Zhandry. Functional encryption without obfuscation. In Eyal Kushilevitz and Tal Malkin, editors, *TCC 2016-A, Part II*, volume 9563 of *LNCS*, pages 480–511. Springer, Berlin, Heidelberg, January 2016.
- [GJKS15] Vipul Goyal, Abhishek Jain, Venkata Koppula, and Amit Sahai. Functional encryption for randomized functionalities. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015, Part II*, volume 9015 of *LNCS*, pages 325–351. Springer, Berlin, Heidelberg, March 2015.
- [GJO16] Vipul Goyal, Aayush Jain, and Adam O’Neill. Multi-input functional encryption with unbounded-message security. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 531–556. Springer, Berlin, Heidelberg, December 2016.
- [ITZ16] Vincenzo Iovino, Qiang Tang, and Karol Zebrowski. On the power of public-key function-private functional encryption. In Sara Foresti and Giuseppe Persiano, editors, *CANS 16*, volume 10052 of *LNCS*, pages 585–593. Springer, Cham, November 2016.
- [JLS21] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from well-founded assumptions. In Samir Khuller and Virginia Vassilevska Williams, editors, *53rd ACM STOC*, pages 60–73. ACM Press, June 2021.
- [JLS22] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from LPN over $\mathbb{F}_p$, DLIN, and PRGs in NC^0 . In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 670–699. Springer, Cham, May / June 2022.
- [KLM⁺18] Sam Kim, Kevin Lewi, Avradip Mandal, Hart Montgomery, Arnab Roy, and David J. Wu. Function-hiding inner product encryption is practical. In Dario Catalano and Roberto De Prisco, editors, *SCN 18*, volume 11035 of *LNCS*, pages 544–562. Springer, Cham, September 2018.

- [KNT18] Fuyuki Kitagawa, Ryo Nishimaki, and Keisuke Tanaka. Obfustopia built on secret-key functional encryption. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 603–648. Springer, Cham, April / May 2018.
- [KPTZ13] Aggelos Kiayias, Stavros Papadopoulos, Nikos Triandopoulos, and Thomas Zacharias. Delegatable pseudorandom functions and applications. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013*, pages 669–684. ACM Press, November 2013.
- [KS17] Ilan Komargodski and Gil Segev. From minicrypt to obfustopia via private-key functional encryption. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part I*, volume 10210 of *LNCS*, pages 122–151. Springer, Cham, April / May 2017.
- [KSY15] Ilan Komargodski, Gil Segev, and Eylon Yogev. Functional encryption for randomized functionalities in the private-key setting from minimal assumptions. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015, Part II*, volume 9015 of *LNCS*, pages 352–377. Springer, Berlin, Heidelberg, March 2015.
- [Lin17] Huijia Lin. Indistinguishability obfuscation from SXDH on 5-linear maps and locality-5 PRGs. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 599–629. Springer, Cham, August 2017.
- [LT19] Benoît Libert and Radu Titiu. Multi-client functional encryption for linear functions in the standard model from LWE. In Steven D. Galbraith and Shiho Moriai, editors, *ASIACRYPT 2019, Part III*, volume 11923 of *LNCS*, pages 520–551. Springer, Cham, December 2019.
- [LW16] Kevin Lewi and David J. Wu. Order-revealing encryption: New constructions, applications, and lower bounds. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 1167–1178. ACM Press, October 2016.
- [LZ20] Muhua Liu and Ping Zhang. An adaptively secure functional encryption for randomized functions. *The Computer Journal*, 63(1):1247–1258, 2020.
- [NPP22] Ky Nguyen, Duong Hieu Phan, and David Pointcheval. Multi-client functional encryption with fine-grained access control. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part I*, volume 13791 of *LNCS*, pages 95–125. Springer, Cham, December 2022.
- [NPP23a] Dinh Duy Nguyen, Duong Hieu Phan, and David Pointcheval. Verifiable decentralized multi-client functional encryption for inner product. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part V*, volume 14442 of *LNCS*, pages 33–65. Springer, Singapore, December 2023.
- [NPP23b] Ky Nguyen, Duong Hieu Phan, and David Pointcheval. Optimal security notion for decentralized multi-client functional encryption. In Mehdi Tibouchi and Xiaofeng Wang, editors, *ACNS 2023, Part II*, volume 13906 of *LNCS*, pages 336–365. Springer, Cham, June 2023.

- [O’N10] Adam O’Neill. Definitional issues in functional encryption. Cryptology ePrint Archive, Report 2010/556, 2010.
- [RS92] Charles Rackoff and Daniel R. Simon. Non-interactive zero-knowledge proof of knowledge and chosen ciphertext attack. In Joan Feigenbaum, editor, *CRYPTO’91*, volume 576 of *LNCS*, pages 433–444. Springer, Berlin, Heidelberg, August 1992.
- [Sah99] Amit Sahai. Non-malleable non-interactive zero knowledge and adaptive chosen-ciphertext security. In *40th FOCS*, pages 543–553. IEEE Computer Society Press, October 1999.
- [SV23] Elaine Shi and Nikhil Vanjani. Multi-client inner product encryption: Function-hiding instantiations without random oracles. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 622–651. Springer, Cham, May 2023.
- [SW14] Amit Sahai and Brent Waters. How to use indistinguishability obfuscation: deniable encryption, and more. In David B. Shmoys, editor, *46th ACM STOC*, pages 475–484. ACM Press, May / June 2014.
- [Tom19] Junichi Tomida. Tightly secure inner product functional encryption: Multi-input and function-hiding constructions. In Steven D. Galbraith and Shiho Moriai, editors, *ASIACRYPT 2019, Part III*, volume 11923 of *LNCS*, pages 459–488. Springer, Cham, December 2019.
- [WW24a] Brent Waters and David J. Wu. Adaptively-sound succinct arguments for NP from indistinguishability obfuscation. In Bojan Mohar, Igor Shinkar, and Ryan O’Donnell, editors, *56th ACM STOC*, pages 387–398. ACM Press, June 2024.
- [WW24b] Brent Waters and David J. Wu. A pure indistinguishability obfuscation approach to adaptively-sound SNARGs for NP. Cryptology ePrint Archive, Report 2024/933, 2024.